Computer Networks (2025-2026)

Part 4: Network Layer

Jeroen Famaey

Andrei Belogaev

{jeroen.famaey, andrei.belogaev}@uantwerpen.be

University of Antwerp
Faculty of Science

Table of contents

In this part, we'll learn exactly how the network layer can provide its host-to-host communication service. We'll see that unlike the transport and application layers, *there is a piece of the network layer in each and every host and router in the network.* Because of this, network-layer protocols are among the most challenging (and therefore among the most interesting!) in the protocol stack. We'll see that the network layer can be decomposed into two interacting parts, the **data plane** and the **control plane**.

We'll first cover the data plane functions of the network layer—the *per-router* functions in the network layer that determine how a datagram (that is, a network-layer packet) arriving on one of a router's input links is forwarded to one of that router's output links. We'll cover both traditional IP forwarding (where forwarding is based on a datagram's destination address) and generalized forwarding (where forwarding and other functions may be performed using values in several different fields in the datagram's header). We'll study the IPv4 and IPv6 protocols and addressing in detail.

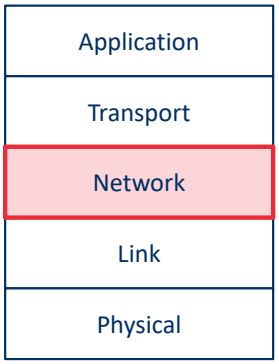
Next, we'll cover the control plane functions of the network layer—the *network-wide* logic that controls how a datagram is routed among routers along an end-to-end path from the source host to the destination host. We'll cover routing algorithms, as well as routing protocols, such as OSPF and BGP, that are in widespread use in today's Internet. Traditionally, these control-plane routing protocols and data-plane forwarding functions have been implemented together, monolithically, within a router. Software-defined networking (SDN) explicitly separates the data plane and control plane by implementing these control plane functions as a separate service, typically in a remote "controller." We'll also cover SDN controllers.

Network layer overview

University of Antwerp
Faculty of Science

Sections 4.1 and 4.2

Network layer overview

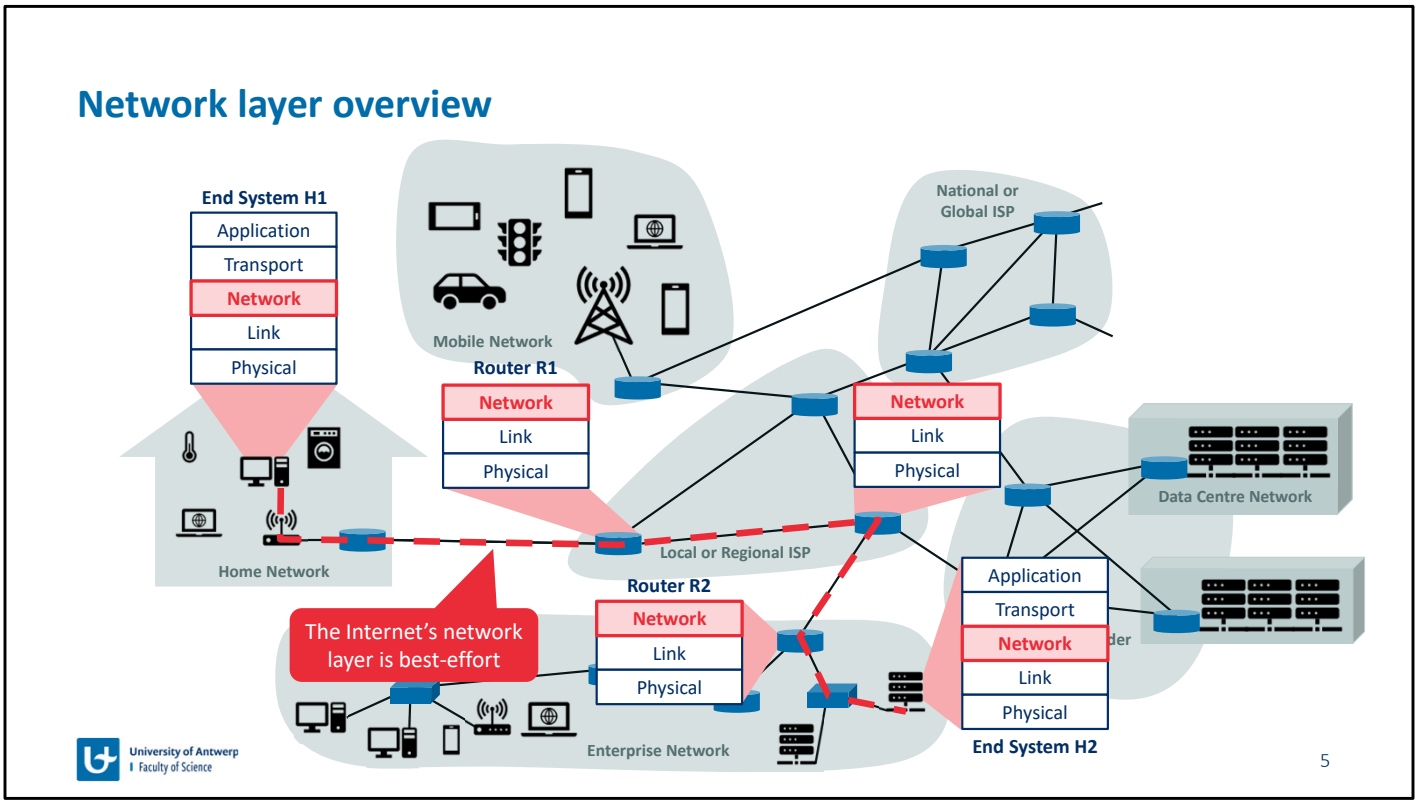


Network Layer

- Responsible for moving network-layer packet from one host to another
- Network-layer packets are called **datagrams**
- Functions
  - Addressing (Internet Protocol (IP) versions 4 and 6)
  - Routing (many intra- and inter-domain routing protocols)
- The Internet's network layer is connectionless and unreliable

The Internet's network layer is responsible for moving network-layer packets known as **datagrams** from one host to another. The Internet transport-layer protocol (TCP or UDP) in a source host passes a transport-layer segment and a destination address to the network layer, just as you would give the postal service a letter with a destination address. The network layer then provides the service of delivering the segment to the transport layer in the destination host.

The Internet's network layer includes the celebrated IP protocol, which defines the fields in the datagram as well as how the end systems and routers act on these fields. There is only one IP protocol, and all Internet components that have a network layer must run the IP protocol. The Internet's network layer also contains routing protocols that determine the routes that datagrams take between sources and destinations. The Internet has many routing protocols. The Internet is a network of networks, and within a network, the network administrator can run any routing protocol desired. Although the network layer contains both the IP protocol and numerous routing protocols, it is often simply referred to as the IP layer, reflecting the fact that IP is the glue that binds the Internet together.

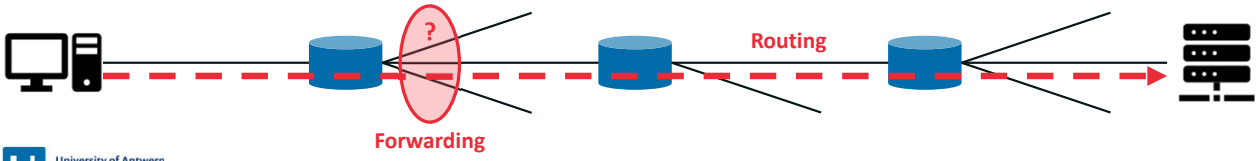


The figure shows a simple network with two hosts, H1 and H2, and several routers on the path between H1 and H2. Let's suppose that H1 is sending information to H2, and consider the role of the network layer in these hosts and in the intervening routers. The network layer in H1 takes segments from the transport layer in H1, encapsulates each segment into a datagram, and then sends the datagrams to its nearby router, R1. At the receiving host, H2, the network layer receives the datagrams from its nearby router R2, extracts the transport-layer segments, and delivers the segments up to the transport layer at H2. The primary data-plane role of each router is to forward datagrams from its input links to its output links; the primary role of the network control plane is to coordinate these local, per-router forwarding actions so that datagrams are ultimately transferred end-to-end, along paths of routers between source and destination hosts. Note that the routers are shown with a truncated protocol stack, that is, with no upper layers above the network layer, because routers do not run application- and transport-layer protocols.

The Internet's network layer provides a single service, known as **best-effort service**. With best-effort service, packets are neither guaranteed to be received in the order in which they were sent, nor is their eventual delivery even guaranteed. There is no guarantee on the end-to-end delay nor is there a minimal bandwidth guarantee. It might appear that *best-effort service* is a euphemism for *no service at all*—a network that delivered *no* packets to the destination would satisfy the definition of best-effort delivery service! Interestingly, in spite of these well-developed alternatives, the Internet's basic best-effort service model combined with adequate bandwidth provisioning and bandwidth-adaptive application-level protocols such as the DASH protocol, have arguably proven to be more than "good enough" to enable an amazing range of applications, including Netflix, video-over-IP, Skype, and Facetime.

Forwarding and routing

- **Forwarding** (data plane)
  - Move packet from the router's input link to the appropriate output link
  - Local decision taken at very short time frame (nanoseconds)
  - Typically implemented in hardware
- **Routing** (control plane)
  - Determine the route or path taken by packets from sender to receiver
  - Network-wide process that takes place on longer time scales (seconds)
  - Typically implemented in software



University of Antwerp
Faculty of Science

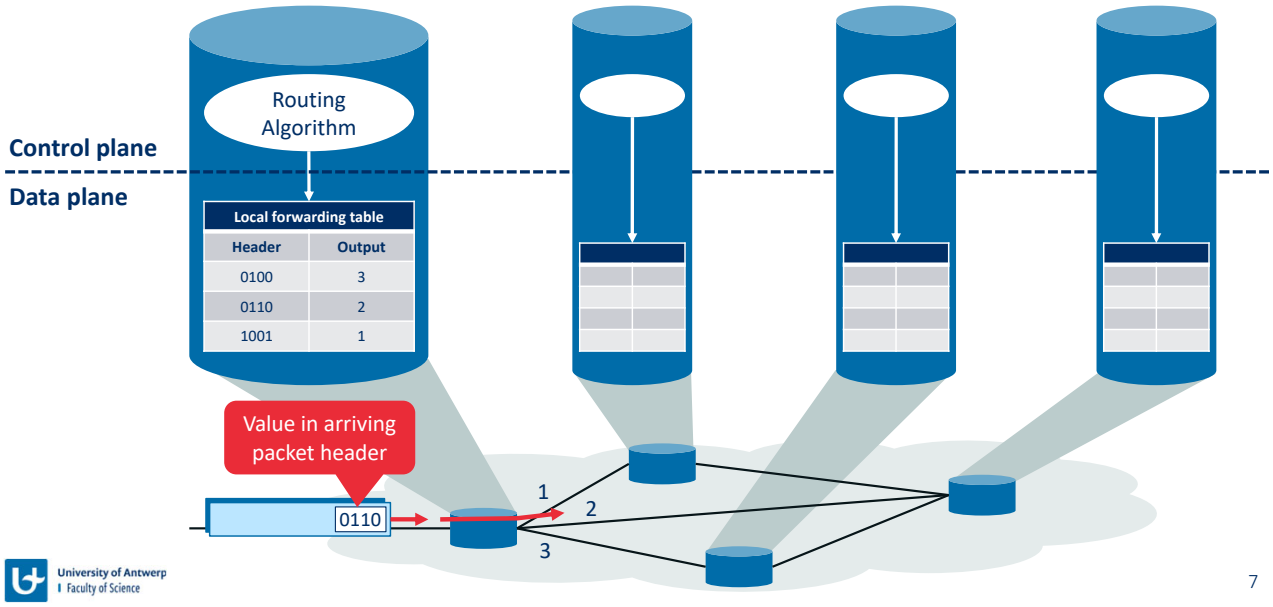
6

The primary role of the network layer is deceptively simple—to move packets from a sending host to a receiving host. To do so, two important network-layer functions can be identified:

- *Forwarding.* When a packet arrives at a router's input link, the router must move the packet to the appropriate output link. As we will see, forwarding is but one function (albeit the most common and important one!) implemented in the data plane. In the more general case, a packet might also be blocked from exiting a router (for example, if the packet originated at a known malicious sending host, or if the packet were destined to a forbidden destination host), or might be duplicated and sent over multiple outgoing links.
- *Routing.* The network layer must determine the route or path taken by packets as they flow from a sender to a receiver. The algorithms that calculate these paths are referred to as **routing algorithms**. Routing is implemented in the control plane of the network layer.

The terms *forwarding* and *routing* are often used interchangeably by authors discussing the network layer. We'll use these terms much more precisely. **Forwarding** refers to the router-local action of transferring a packet from an input link interface to the appropriate output link interface. Forwarding takes place at very short timescales (typically a few nanoseconds), and thus is typically implemented in hardware. **Routing** refers to the network-wide process that determines the end-to-end paths that packets take from source to destination. Routing takes place on much longer timescales (typically seconds), and as we will see is often implemented in software.

Traditional control plane

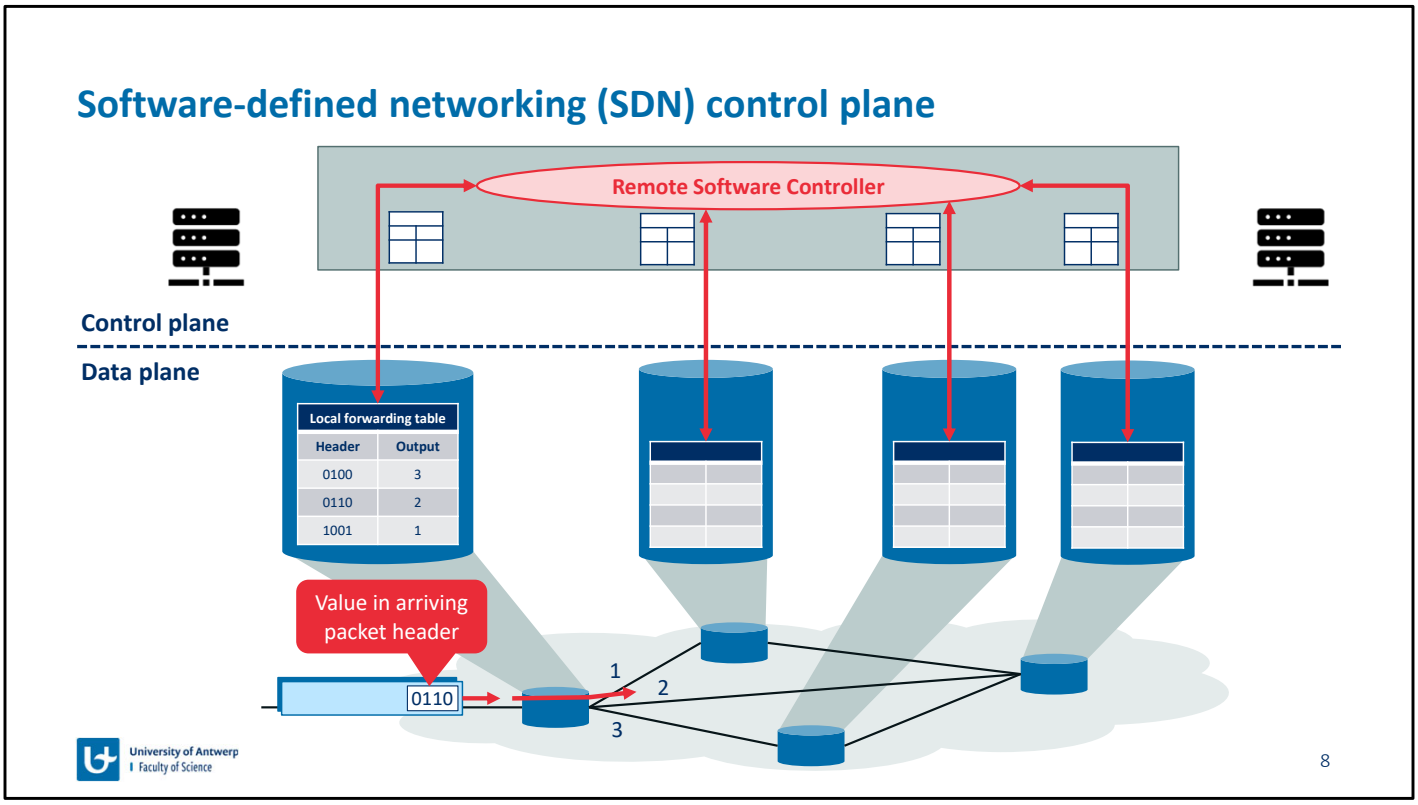


University of Antwerp
Faculty of Science

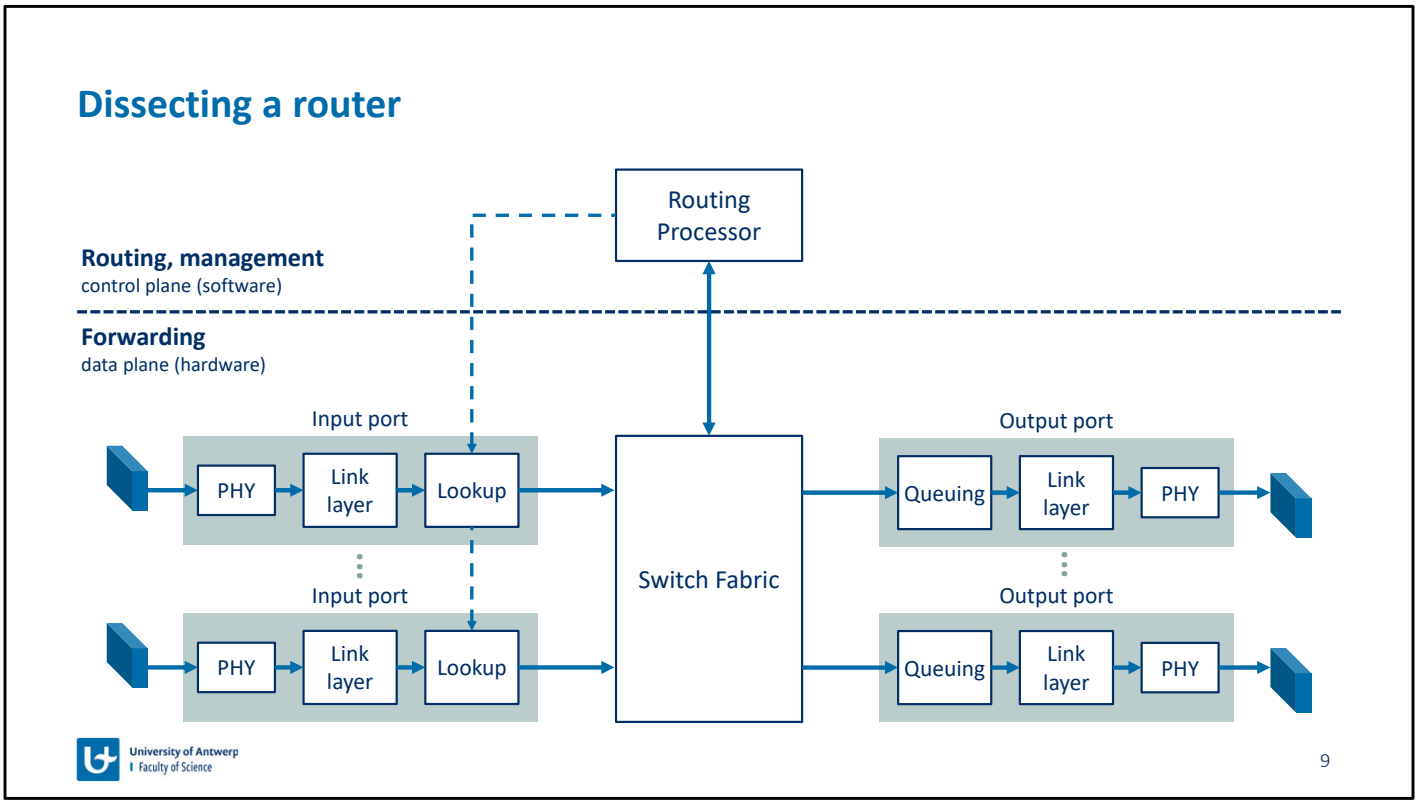
7

A key element in every network router is its **forwarding table**. A router forwards a packet by examining the value of one or more fields in the arriving packet's header, and then using these header values to index into its forwarding table. The value stored in the forwarding table entry for those values indicates the outgoing link interface at that router to which that packet is to be forwarded. For example, a packet with header field value of 0110 arrives to a router. The router indexes into its forwarding table and determines that the output link interface for this packet is interface 2. The router then internally forwards the packet to interface 2.

But now you are undoubtedly wondering how a router's forwarding tables are configured in the first place. This is a crucial issue, one that exposes the important interplay between forwarding (in data plane) and routing (in control plane). The routing algorithm determines the contents of the routers' forwarding tables. In this example, a routing algorithm runs in each and every router and both forwarding and routing functions are contained within a router. As we'll see, the routing algorithm function in one router communicates with the routing algorithm function in other routers to compute the values for its forwarding table.



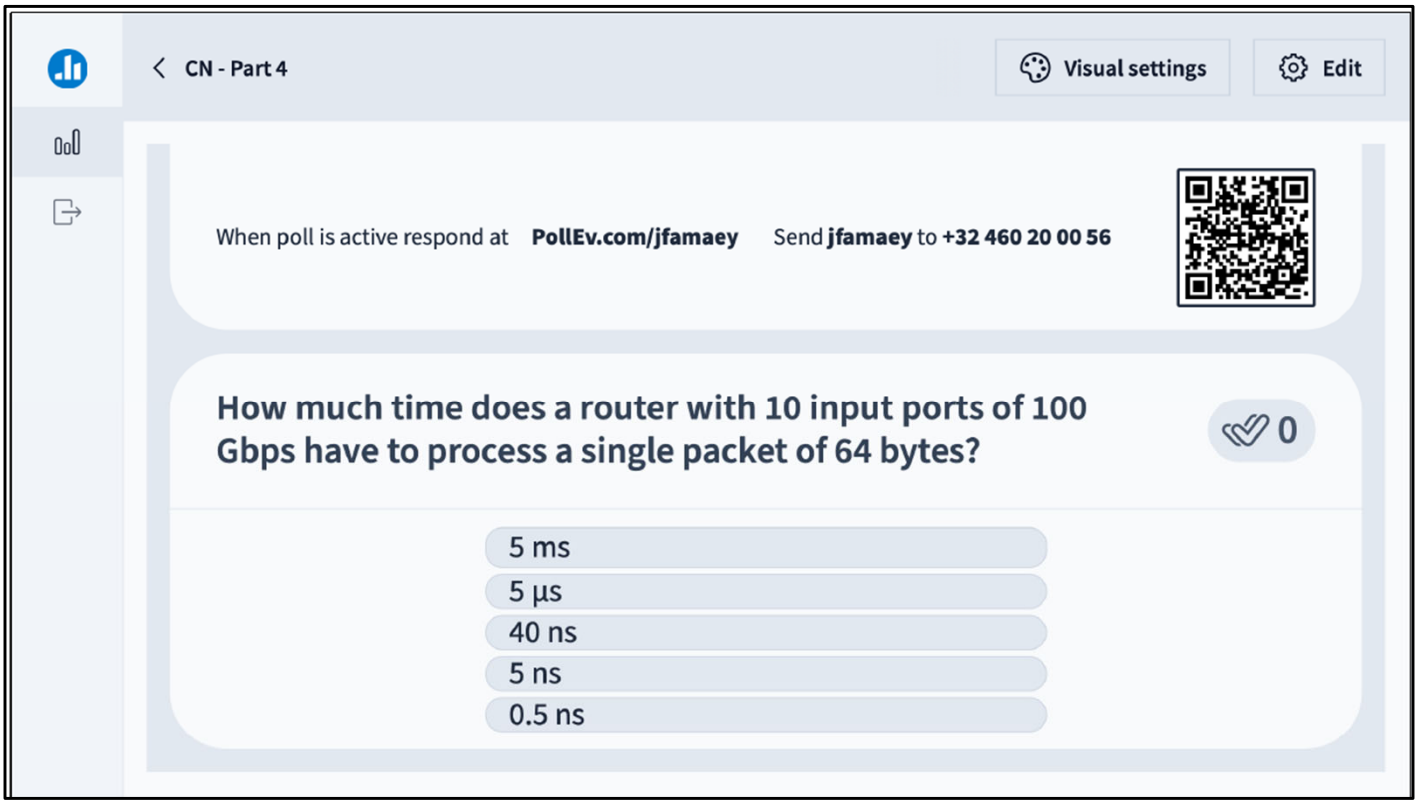
This figure shows an alternative approach in which a physically separate, **remote controller** computes and distributes the forwarding tables to be used by each and every router. Note that the data plane components of both approaches are identical. Here, however, control-plane routing functionality is separated from the physical router—the routing device performs forwarding only, while the remote controller computes and distributes forwarding tables. The remote controller might be implemented in a remote data centre with high reliability and redundancy, and might be managed by the ISP or some third party. How might the routers and the remote controller communicate? By exchanging messages containing forwarding tables and other pieces of routing information. The control-plane approach shown here is at the heart of **software-defined networking (SDN)**, where the network is "software-defined" because the controller that computes forwarding tables and interacts with routers is implemented in software.



Four router components can be identified:

- ***Input ports***. An input port performs several key functions. It performs the physical layer function of terminating an incoming physical link at a router; this is shown in the leftmost box of an input port and the rightmost box of an output port. An input port also performs link-layer functions needed to interoperate with the link layer at the other side of the incoming link; this is represented by the middle boxes in the input and output ports. Perhaps most crucially, a lookup function is also performed at the input port; this will occur in the rightmost box of the input port. It is here that the forwarding table is consulted to determine the router output port to which an arriving packet will be forwarded via the switching fabric. Control packets (for example, packets carrying routing protocol information) are forwarded from an input port to the routing processor. Note that the term "port" here—referring to the physical input and output router interfaces—is distinctly different from the software ports associated with network applications and sockets.
- ***Switching fabric***. The switching fabric connects the router's input ports to its output ports. This switching fabric is completely contained within the router—a network inside of a network router!
- ***Output ports***. An output port stores packets received from the switching fabric and transmits these packets on the outgoing link by performing the necessary link-layer and physical-layer functions. When a link is bidirectional (that is, carries traffic in both directions), an output port will typically be paired with the input port for that link on the same line card.
- ***Routing processor***. The routing processor performs control-plane functions. In traditional routers, it executes the routing protocols, maintains routing tables and

attached link state information, and computes the forwarding table for the router. In SDN routers, the routing processor is responsible for communicating with the remote controller in order to (among other activities) receive forwarding table entries computed by the remote controller, and install these entries in the router's input ports.


CN - Part 4
Visual settings
Edit
When poll is active respond at PollEv.com/jfamaey Send jfamaey to +32 460 20 00 56
How much time does a router with 10 input ports of 100 Gbps have to process a single packet of 64 bytes?
0
5 ms
5 µs
40 ns
5 ns
0.5 ns

Destination-based forwarding

- The forwarding table cannot contain an entry for each destination address (4 billion entries for IPv4!)
- Instead, forwarding table entries consist of **ranges**, represented by prefixes
- If a packet matches multiple prefixes, longest prefix matching is used

Destination address range		Prefix	Link Interface
11001000 00010111 00010000 00000000	11001000 00010111 00010111 11111111	11001000 00010111 00010*** ********	0
11001000 00010111 00011000 00000000	11001000 00010111 00011000 11111111	11001000 00010111 00011000 ********	1
11001000 00010111 00011001 00000000	11001000 00010111 00011111 11111111	11001000 00010111 00011*** ********	2
Otherwise		Otherwise	3

University of Antwerp
Faculty of Science

Let's consider the case that the output port to which an incoming packet is to be switched is based on the packet's destination address. In the case of 32-bit IP addresses, a brute-force implementation of the forwarding table would have one entry for every possible destination address. Since there are more than 4 billion possible addresses, this option is totally out of the question.

As an example of how this issue of scale can be handled, let's suppose that our router has four links, numbered 0 through 3, and that packets are to be forwarded to the link interfaces according to the destination address ranges listed in the table. Clearly, for this example, it is not necessary to have 4 billion entries in the router's forwarding table. We could, for example, have a forwarding table with just four prefix entries. With this style of forwarding table, the router matches a **prefix** of the packet's destination address with the entries in the table; if there's a match, the router forwards the packet to a link associated with the match.

Exercise: Prefix matching

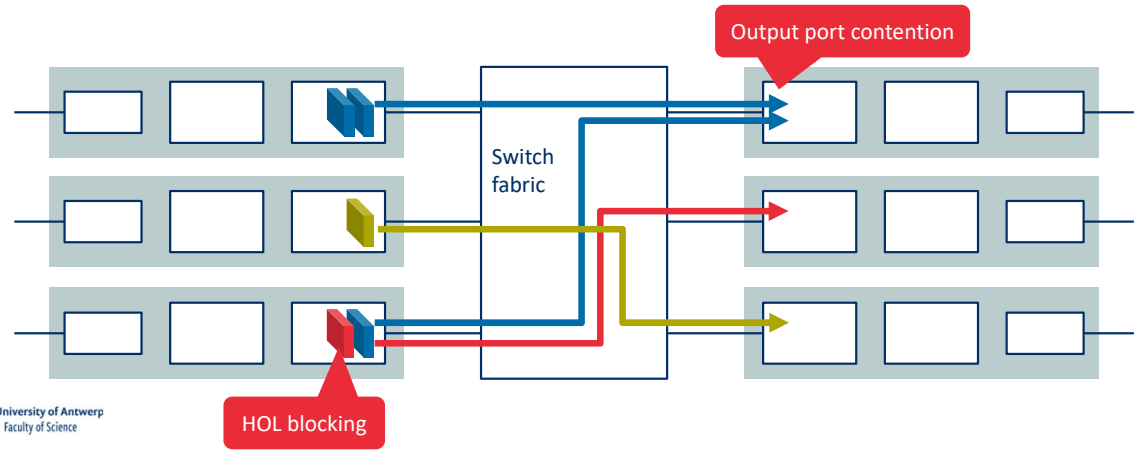
Exercise: On which interface is each of the packets forwarded?

Destination address range		Prefix	Link Interface
11001000 00010111 00010000 00000000	11001000 00010111 00010111 11111111	11001000 00010111 00010*** ********	0
11001000 00010111 00011000 00000000	11001000 00010111 00011000 11111111	11001000 00010111 00011000 ********	1
11001000 00010111 00011001 00000000	11001000 00010111 00011111 11111111	11001000 00010111 00011*** ********	2
Otherwise		Otherwise	3

- Packet 1: `11001000 00010111 00010110 10100001`
- Packet 2: `11001000 00010111 00011000 10101010`

Queuing in routers

- When input or output queues (buffers) become full, **packet loss** will occur
  - **Head-of-the-line (HOL) blocking** occurs when a packet in an input queue must wait because it is blocked by another packet at the head of the line
  - **Output port contention** occurs when multiple packets are destined for the same output line at the same time



University of Antwerp
Faculty of Science

15

If we consider input and output port functionality and the configurations, it's clear that packet queues may form at both the input ports *and* the output ports. The location and extent of queueing (either at the input port queues or the output port queues) will depend on the traffic load, the relative speed of the switching fabric, and the line speed. Let's now consider these queues in a bit more detail, since as these queues grow large, the router's memory can eventually be exhausted and **packet loss** will occur when no memory is available to store arriving packets.

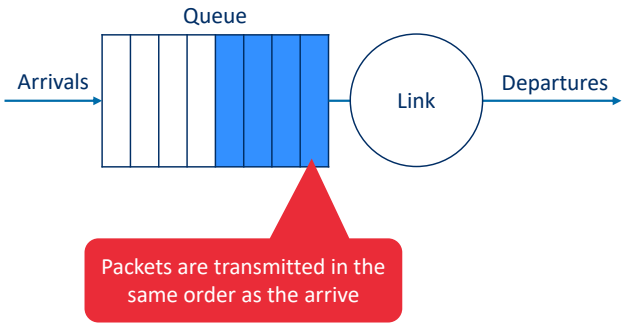
What happens if the switch fabric is not fast enough (relative to the input line speeds) to transfer *all* arriving packets through the fabric without delay? In this case, packet queuing can also occur at the input ports, as packets must join input port queues to wait their turn to be transferred through the switching fabric to the output port. Consider an example where two packets (dark blue coloured) at the front of their input queues are destined for the same upper-right output port. Suppose that the switch fabric chooses to transfer the packet from the front of the upper-left queue. In this case, the darkly shaded packet in the lower-left queue must wait. But not only must this darkly shaded packet wait, so too must the lightly shaded packet that is queued behind that packet in the lower-left queue, even though there is *no* contention for the middle-right output port (the destination for the lightly shaded packet). This phenomenon is known as **head-of-the-line (HOL) blocking** in an input-queued switch—a queued packet in an input queue must wait for transfer through the fabric (even though its output port is free) because it is blocked by another packet at the head of the line.

Let's next consider whether queueing can occur at a switch's output ports. Suppose that *the switching speed* is *N* times faster than *line speed* and that packets arriving at each of the *N*

input ports are destined to the same output port. In this case, in the time it takes to send a single packet onto the outgoing link, *N* new packets will arrive at this output port (one from each of the *N* input ports). Since the output port can transmit only a single packet in a unit of time (the packet transmission time), the *N* arriving packets will have to queue (wait) for transmission over the outgoing link. Then *N* more packets can possibly arrive in the time it takes to transmit just one of the *N* packets that had just previously been queued. And so on. Thus, packet queues can form at the output ports even when the switching fabric is *N* times faster than the port line speeds.

Packet scheduling techniques in router queues

First-in-first-out (FIFO)



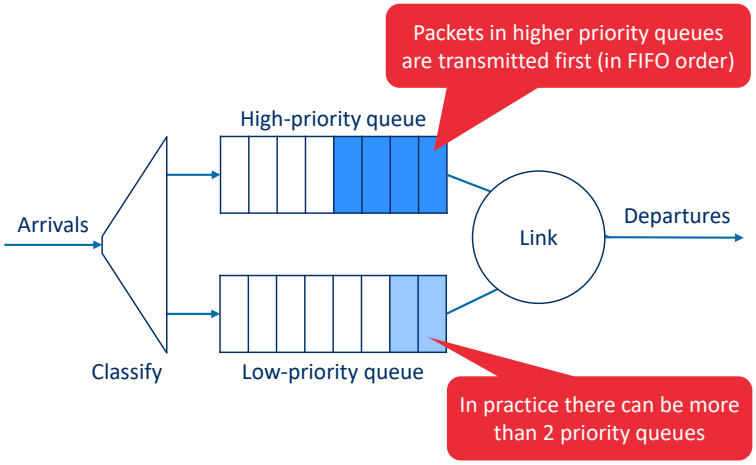
University of Antwerp
Faculty of Science

16

The FIFO (also known as first-come-first-served, or FCFS) scheduling discipline selects packets for link transmission in the same order in which they arrived at the output link queue.

Packet scheduling techniques in router queues

Priority queuing



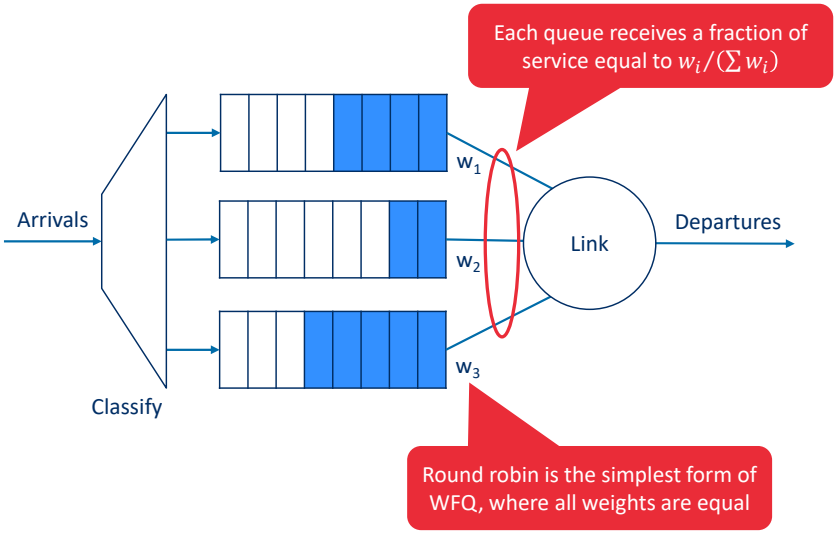
University of Antwerp
Faculty of Science

17

Under priority queuing, packets arriving at the output link are classified into priority classes upon arrival at the queue. In practice, a network operator may configure a queue so that packets carrying network management information (for example, as indicated by the source or destination TCP/UDP port number) receive priority over user traffic; additionally, real-time voice-over-IP packets might receive priority over non-real-time traffic such e-mail packets. Each priority class typically has its own queue. When choosing a packet to transmit, the priority queuing discipline will transmit a packet from the highest priority class that has a nonempty queue (that is, has packets waiting for transmission). The choice among packets in the same priority class is typically done in a FIFO manner.

Packet scheduling techniques in router queues

Round robin and weighted fair queuing (WFQ)



University of Antwerp
Faculty of Science

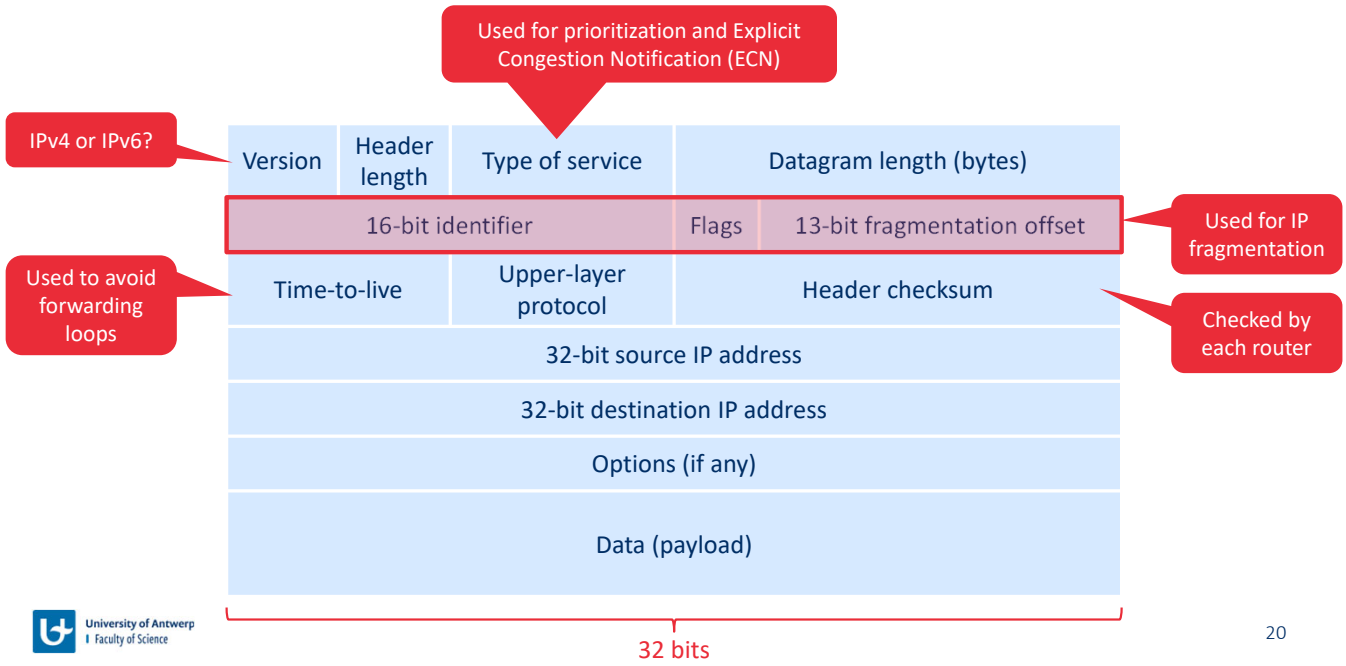
18

Under the **round robin queuing discipline**, packets are sorted into classes as with priority queuing. However, rather than there being a strict service priority among classes, a round robin scheduler alternates service among the classes. In the simplest form of round robin scheduling, a class 1 packet is transmitted, followed by a class 2 packet, followed by a class 1 packet, followed by a class 2 packet, and so on.

A generalized form of round robin queuing that has been widely implemented in routers is the so-called **weighted fair queuing (WFQ) discipline**. Here, arriving packets are classified and queued in the appropriate per-class waiting area. As in round robin scheduling, a WFQ scheduler will serve classes in a circular manner—first serving class 1, then serving class 2, then serving class 3, and then (assuming there are three classes) repeating the service pattern. WFQ differs from round robin in that each class may receive a differential amount of service in any interval of time. Specifically, each class, *i*, is assigned a weight, w_i. Under WFQ, during any interval of time during which there are class *i* packets to send, class *i* will then be guaranteed to receive a fraction of service equal to $w_i/(\sum w_i)$.

The Internet Protocol (IP)

IPv4 datagram format – RFC 791



The key fields in the IPv4 datagram are the following:

- ***Version number.*** These 4 bits specify the IP protocol version of the datagram. By looking at the version number, the router can determine how to interpret the remainder of the IP datagram. Different versions of IP use different datagram formats.
- ***Header length.*** Because an IPv4 datagram can contain a variable number of options (which are included in the IPv4 datagram header), these 4 bits are needed to determine where in the IP datagram the payload (for example, the transport-layer segment being encapsulated in this datagram) actually begins. Most IP datagrams do not contain options, so the typical IP datagram has a 20-byte header. The value represents the length of the header in 32-bit words.
- ***Type of service.*** The type of service (TOS) bits were included in the IPv4 header to allow different types of IP datagrams to be distinguished from each other. For example, it might be useful to distinguish real-time datagrams (such as those used by an IP telephony application) from non-real-time traffic (e.g., FTP). The specific level of service to be provided is a policy issue determined and configured by the network administrator for that router. We also learned before that two of the TOS bits are used for Explicit Congestion Notification.
- ***Datagram length.*** This is the total length of the IP datagram (header plus data), measured in bytes. Since this field is 16 bits long, the theoretical maximum size of the IP datagram is 65,535 bytes. However, datagrams are rarely larger than 1,500 bytes, which allows an IP datagram to fit in the payload field of a maximally sized Ethernet frame.
- ***Identifier, flags, fragmentation offset.*** These three fields have to do with so-called IP fragmentation, when a large IP datagram is broken into several smaller IP datagrams which are then forwarded independently to the destination, where they are

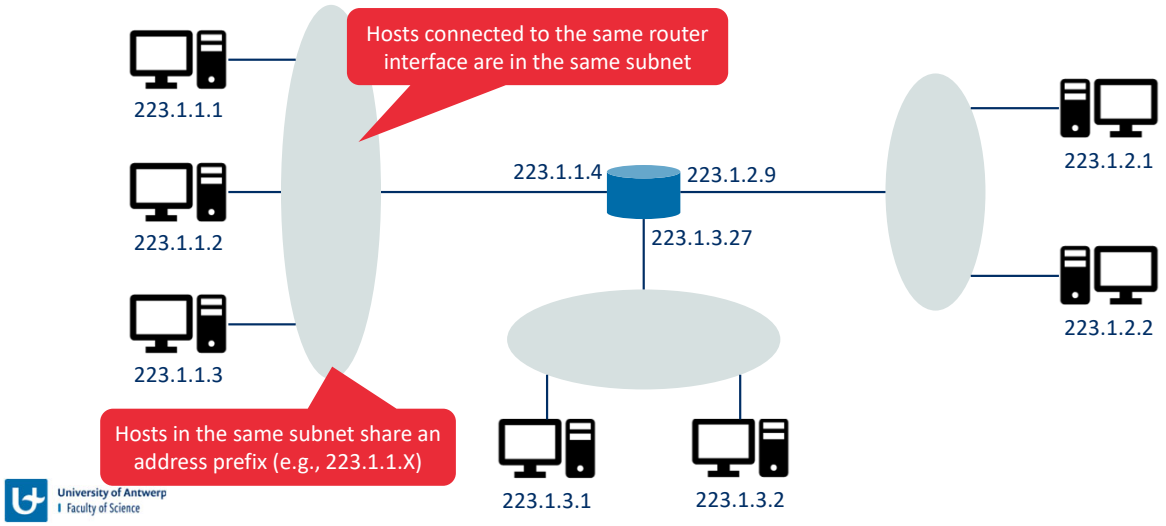
reassembled before their payload data is passed up to the transport layer at the destination host. Interestingly, the new version of IP, IPv6, does not allow for fragmentation.

- ***Time-to-live.*** The time-to-live (TTL) field is included to ensure that datagrams do not circulate forever (due to, for example, a long-lived routing loop) in the network. This field is decremented by one each time the datagram is processed by a router. If the TTL field reaches 0, a router must drop that datagram.
- ***Upper-layer protocol.*** This field is typically used only when an IP datagram reaches its final destination. The value of this field indicates the specific transport-layer protocol to which the data portion of this IP datagram should be passed. For example, a value of 6 indicates that the data portion is passed to TCP, while a value of 17 indicates that the data is passed to UDP. For a list of all possible values, see [IANA Protocol Numbers 2016]. Note that the protocol number in the IP datagram has a role that is analogous to the role of the port number field in the transport-layer segment. The protocol number is the glue that binds the network and transport layers together, whereas the port number is the glue that binds the transport and application layers together. We'll see in Chapter 6 that the link-layer frame also has a special field that binds the link layer to the network layer.
- ***Header checksum.*** The header checksum aids a router in detecting bit errors in a received IP datagram. The header checksum is computed by treating each 2 bytes in the header as a number and summing these numbers using 1s complement arithmetic. The 1s complement of this sum, known as the Internet checksum, is stored in the checksum field. A router computes the header checksum for each received IP datagram and detects an error condition if the checksum carried in the datagram header does not equal the computed checksum. Routers typically discard datagrams for which an error has been detected. Note that the checksum must be recomputed and stored again at each router, since the TTL field, and possibly the options field as well, will change.
- ***Source and destination IP addresses.*** When a source creates a datagram, it inserts its IP address into the source IP address field and inserts the address of the ultimate destination into the destination IP address field. Often the source host determines the destination address via a DNS lookup.
- ***Options.*** The options fields allow an IP header to be extended. Header options were meant to be used rarely—hence the decision to save overhead by not including the information in options fields in every datagram header. However, the mere existence of options does complicate matters—since datagram headers can be of variable length, one cannot determine a priori where the data field will start. Also, since some datagrams may require options processing and others may not, the amount of time needed to process an IP datagram at a router can vary greatly. These considerations become particularly important for IP processing in high-performance routers and hosts. For these reasons and others, IP options were not included in the IPv6 header.
- ***Data (payload).*** Finally, we come to the last and most important field—the *raison d'etre* for the datagram in the first place! In most circumstances, the data field of the IP datagram contains the transport-layer segment (TCP or UDP) to be delivered to the destination. However, the data field can carry other types of data, such as ICMP messages.

A question often asked at this point is, why does TCP/IP perform error checking at both the transport and network layers? There are several reasons for this repetition. First, note that only the IP header is checksummed at the IP layer, while the TCP/UDP checksum is computed over the entire TCP/UDP segment. Second, TCP/UDP and IP do not necessarily both have to belong to the same protocol stack. TCP can, in principle, run over a different network-layer protocol (for example, ATM) and IP can carry data that will not be passed to TCP/UDP.

IPv4 addressing

Between each host and physical link is an **interface**, that has its own IP address



University of Antwerp
Faculty of Science

21

A host typically has only a single link into the network; when IP in the host wants to send a datagram, it does so over this link. The boundary between the host and the physical link is called an **interface**. Now consider a router and its interfaces. Because a router's job is to receive a datagram on one link and forward the datagram on some other link, a router necessarily has two or more links to which it is connected. The boundary between the router and any one of its links is also called an interface. A router thus has multiple interfaces, one for each of its links. Because every host and router is capable of sending and receiving IP datagrams, IP requires each host and router interface to have its own IP address. *Thus, an IP address is technically associated with an interface, rather than with the host or router containing that interface.*

Each IP address is 32 bits long (equivalently, 4 bytes), and there are thus a total of 2^{32} (or approximately 4 billion) possible IP addresses. These addresses are typically written in so-called **dotted-decimal notation**, in which each byte of the address is written in its decimal form and is separated by a period (dot) from other bytes in the address. For example, consider the IP address 193.32.216.9. The 193 is the decimal equivalent of the first 8 bits of the address; the 32 is the decimal equivalent of the second 8 bits of the address, and so on. Thus, the address 193.32.216.9 in binary notation is 11000001 00100000 11011000 00001001.

Each interface on every host and router in the global Internet must have an IP address that is globally unique (except for interfaces behind NATs). These addresses cannot be chosen in a willy-nilly manner, however. A portion of an interface's IP address will be determined by the subnet to which it is connected.

The figure provides an example of IP addressing and interfaces. In this figure, one router (with three interfaces) is used to interconnect seven hosts. Take a close look at the IP addresses assigned to the host and router interfaces, as there are several things to notice. The three hosts in the upper-left portion, and the router interface to which they are connected, all have an IP address of the form 223.1.1.xxx. That is, they all have the same leftmost 24 bits in their IP address. These four interfaces are also interconnected to each other by a network *that contains no routers*.

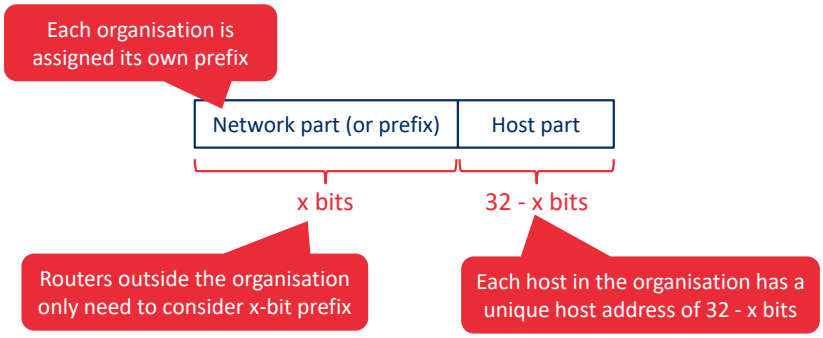
In IP terms, this network interconnecting three host interfaces and one router interface forms a **subnet** [RFC 950]. IP addressing assigns an address to this subnet: 223.1.1.0/24, where the /24 ("slash-24") notation, sometimes known as a **subnet mask**, indicates that the leftmost 24 bits of the 32-bit quantity define the subnet address. The 223.1.1.0/24 subnet thus consists of the three host interfaces (223.1.1.1, 223.1.1.2, and 223.1.1.3) and one router interface (223.1.1.4). Any additional hosts attached to the 223.1.1.0/24 subnet would be *required* to have an address of the form 223.1.1.xxx. There are two additional subnets shown in the figure: the 223.1.2.0/24 network and the 223.1.3.0/24 subnet.

Classless Interdomain Routing (CIDR) – RFC 4632

- CIDR determines the Internet's address assignment strategy
- Each 32-bit IP address is split in two parts (x indicates the number of bits in the first part, also referred to as subnet

a.b.c.d/x

- IP broadcast address: 255.255.255.255 (the message is delivered to all hosts in the subnet)



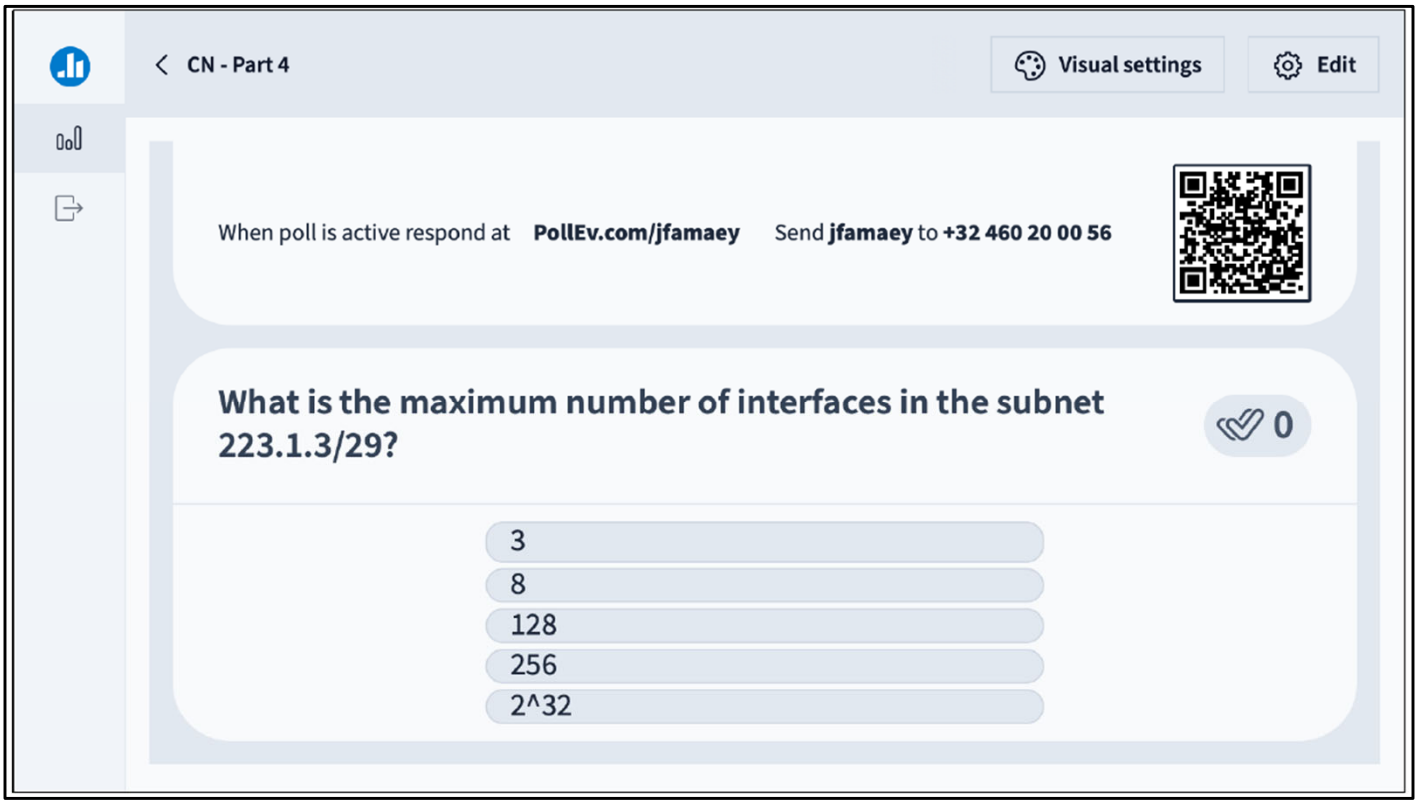
University of Antwerp
Faculty of Science

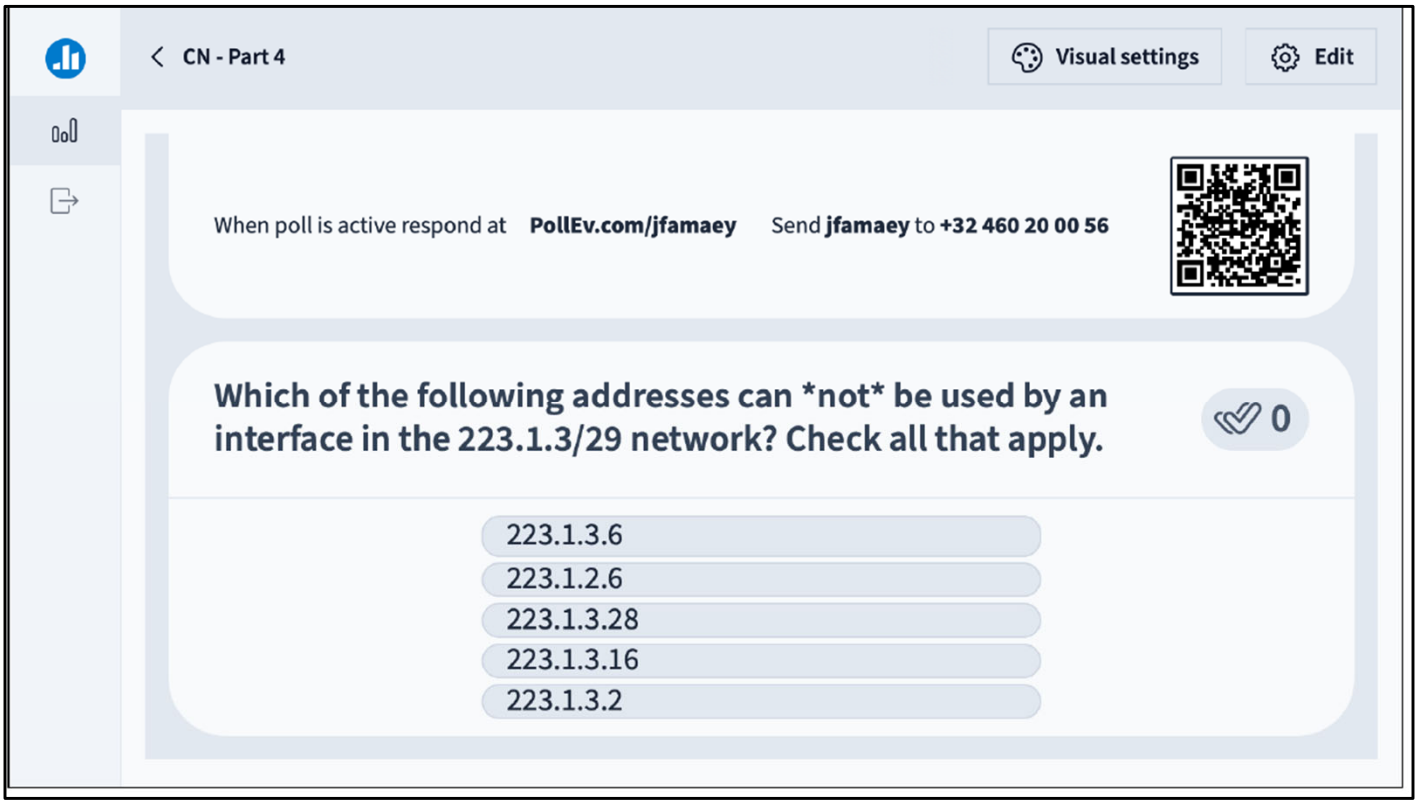
22

The Internet's address assignment strategy is known as **Classless Interdomain Routing (CIDR**—pronounced *cider*) [RFC 4632]. CIDR generalizes the notion of subnet addressing. As with subnet addressing, the 32-bit IP address is divided into two parts and again has the dotted-decimal form *a.b.c.d/x*, where *x* indicates the number of bits in the first part of the address.

The *x* most significant bits of an address of the form *a.b.c.d/x* constitute the network portion of the IP address, and are often referred to as the **prefix** (or *network prefix*) of the address. An organization is typically assigned a block of contiguous addresses, that is, a range of addresses with a common prefix. In this case, the IP addresses of devices within the organization will share the common prefix. When a router outside the organization forwards a datagram whose destination address is inside the organization, only the leading *x* bits of the address need be considered. This considerably reduces the size of the forwarding table in these routers, since a *single* entry of the form *a.b.c.d/x* will be sufficient to forward packets to *any* destination within the organization. The remaining 32-*x* bits of an address can be thought of as distinguishing among the devices *within* the organization, all of which have the same network prefix. These are the bits that will be considered when forwarding packets at routers *within* the organization.

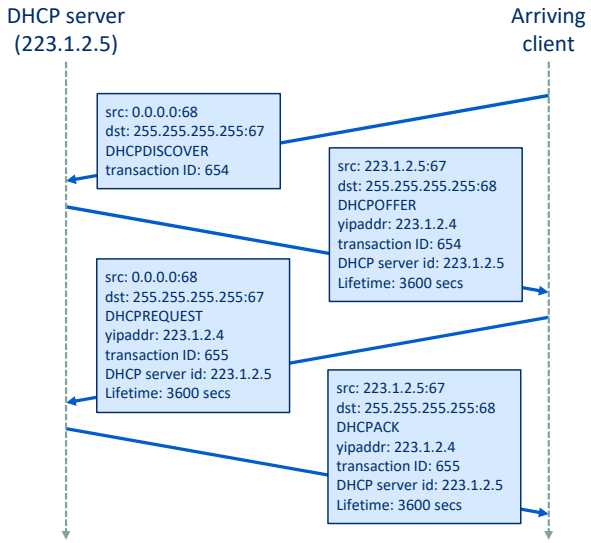
We would be remiss if we did not mention yet another type of IP address, the IP broadcast address 255.255.255.255. When a host sends a datagram with destination address 255.255.255.255, the message is delivered to all hosts on the same subnet. Routers optionally forward the message into neighbouring subnets as well (although they usually don't).


CN - Part 4
Visual settings
Edit
When poll is active respond at PollEv.com/jfamaey Send jfamaey to +32 460 20 00 56
What is the maximum number of interfaces in the subnet 223.1.3/29?
0
3
8
128
256
2^32


CN - Part 4
Visual settings
Edit
When poll is active respond at PollEv.com/jfamaey Send jfamaey to +32 460 20 00 56
Which of the following addresses can *not* be used by an interface in the 223.1.3/29 network? Check all that apply.
0
223.1.3.6
223.1.2.6
223.1.3.28
223.1.3.16
223.1.3.2

Dynamic Host Configuration Protocol (DHCP)

- A DHCP server offers several functions to hosts within a subnet
  - Automatically assign IP addresses to hosts
  - Provide additional information (subnet mask, IP address of first-hop router, local DNS server)
- DHCP consists of 4 steps
  - Discovery
  - Service offer(s)
  - Request
  - Acknowledgment
- An assigned IP address has a lifetime (i.e., amount of time it is valid), which can be extended with a renewal request



University of Antwerp
Faculty of Science

26

A system administrator will typically manually configure the IP addresses into the router (often remotely, with a network management tool). Host addresses can also be configured manually, but typically this is done using the **Dynamic Host Configuration Protocol (DHCP)** [RFC 2131]. DHCP allows a host to obtain (be allocated) an IP address automatically. A network administrator can configure DHCP so that a given host receives the same IP address each time it connects to the network, or a host may be assigned a **temporary IP address** that will be different each time the host connects to the network. In addition to host IP address assignment, DHCP also allows a host to learn additional information, such as its subnet mask, the address of its first-hop router (often called the default gateway), and the address of its local DNS server.

For a newly arriving host, the DHCP protocol is a four-step process. In the figure, yiaddr (as in "your Internet address") indicates the address being allocated to the newly arriving client. The four steps are:

- ***DHCP server discovery.*** The first task of a newly arriving host is to find a DHCP server with which to interact. This is done using a **DHCP discover message**, which a client sends within a UDP packet to port 67. The UDP packet is encapsulated in an IP datagram. But to whom should this datagram be sent? The host doesn't even know the IP address of the network to which it is attaching, much less the address of a DHCP server for this network. Given this, the DHCP client creates an IP datagram containing its DHCP discover message along with the broadcast destination IP address of 255.255.255.255 and a "this host" source IP address of 0.0.0.0. The DHCP client passes the IP datagram to the link layer, which then broadcasts this frame to all nodes attached to the subnet.
- ***DHCP server offer(s).*** A DHCP server receiving a DHCP discover message responds to the

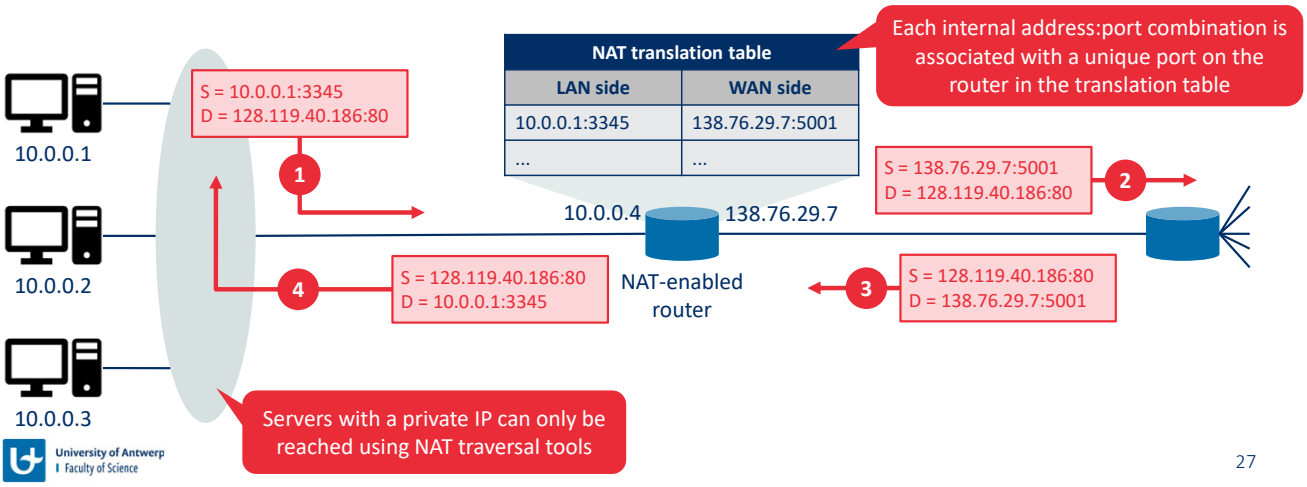
client with a **DHCP offer message** that is broadcast to all nodes on the subnet, again using the IP broadcast address of 255.255.255.255. Since several DHCP servers can be present on the subnet, the client may find itself in the enviable position of being able to choose from among several offers. Each server offer message contains the transaction ID of the received discover message, the proposed IP address for the client, the network mask, and an IP **address lease time**—the amount of time for which the IP address will be valid.

- ***DHCP request.*** The newly arriving client will choose from among one or more server offers and respond to its selected offer with a **DHCP request message**, echoing back the configuration parameters.
- ***DHCP ACK.*** The server responds to the DHCP request message with a **DHCP ACK message**, confirming the requested parameters.

Since a client may want to use its address beyond the lease's expiration, DHCP also provides a mechanism that allows a client to renew its lease on an IP address.

Network Address Translation (NAT)

- There are not enough IPv4 addresses to give every host in every network a unique IP address
- NAT offers a solution by associating a set of private addresses with a single public IP address



With the proliferation of small office, home office (SOHO) subnets, this would seem to imply that whenever a SOHO wants to install a LAN to connect multiple machines, a range of addresses would need to be allocated by the ISP to cover all of the SOHO's IP devices (including phones, tablets, gaming devices, IP TVs, printers and more). If the subnet grew bigger, a larger block of addresses would have to be allocated. But what if the ISP had already allocated the contiguous portions of the SOHO network's current address range? And what typical homeowner wants (or should need) to know how to manage IP addresses in the first place? Fortunately, there is a simpler approach to address allocation that has found increasingly widespread use in such scenarios: **network address translation (NAT)**.

The figure shows the operation of a NAT-enabled router. The NAT-enabled router, residing in the home, has an interface that is part of the home network on the left. Addressing within the home network is exactly as we have seen above—all four interfaces in the home network have the same subnet address of 10.0.0.0/24. The address space 10.0.0.0/8 is one of three portions of the IP address space that is reserved in [RFC 1918] for a **private network** (the others are 172.16.0.0/12 and 192.168.0.0/16).

To see why this is important, consider the fact that there are hundreds of thousands of home networks, many using the same address space, 10.0.0.0/24. Devices within a given home network can send packets to each other using 10.0.0.0/24 addressing. However, packets forwarded *beyond* the home network into the larger global Internet clearly cannot use these addresses (as either a source or a destination address) because there are hundreds of thousands of networks using this block of addresses. That is, the 10.0.0.0/24 addresses can only have meaning within the given home network. But if private addresses only have

meaning within a given network, how is addressing handled when packets are sent to or received from the global Internet, where addresses are necessarily unique? The answer lies in understanding NAT.

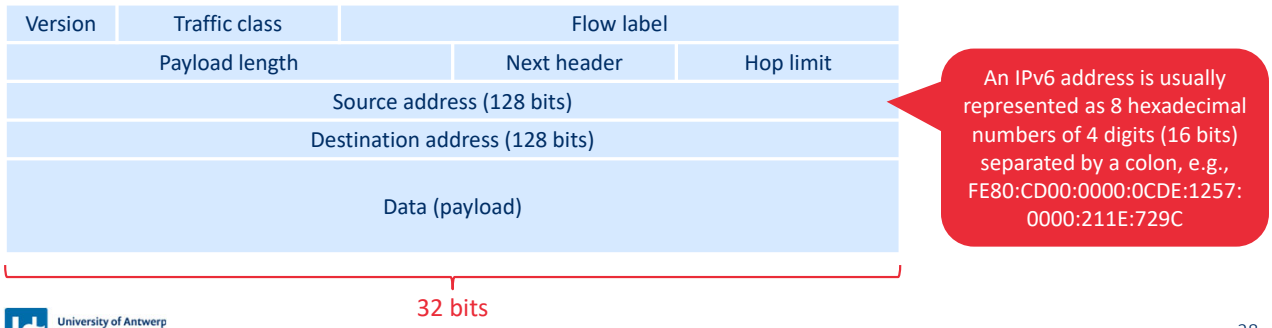
The NAT-enabled router does not *look* like a router to the outside world. Instead the NAT router behaves to the outside world as a *single* device with a *single* IP address. In the figure, all traffic leaving the home router for the larger Internet has a source IP address of 138.76.29.7, and all traffic entering the home router must have a destination address of 138.76.29.7. In essence, the NAT-enabled router is hiding the details of the home network from the outside world.

If all datagrams arriving at the NAT router from the WAN have the same destination IP address (specifically, that of the WAN-side interface of the NAT router), then how does the router know the internal host to which it should forward a given datagram? The trick is to use a **NAT translation table** at the NAT router, and to include port numbers as well as IP addresses in the table entries.

NAT has enjoyed widespread deployment in recent years. But NAT is not without detractors. First, one might argue that, port numbers are meant to be used for addressing processes, not for addressing hosts. This violation can indeed cause problems for servers running on the home network, since server processes wait for incoming requests at well-known port numbers and peers in a P2P protocol need to accept incoming connections when acting as servers. How can one peer connect to another peer that is behind a NAT server, and has a DHCP-provided NAT address? Technical solutions to these problems include **NAT traversal** tools [RFC 5389].

IPv6

- The last block of IPv4 addresses was assigned to a regional registry in February 2011
- IPv6 development began in the mid-1990s to prepare for this event
- Main improvements
  - Expanded addressing capabilities: IP address size is now 128 instead of 32 bits
  - Streamlined 40 byte header: Fixed length header is easier to process by routers
  - Flow labelling: IPv6 supports the concept of flows (i.e., packets that belong together at the application layer)



University of Antwerp | Faculty of Science

28

In the early 1990s, the Internet Engineering Task Force began an effort to develop a successor to the IPv4 protocol. A prime motivation for this effort was the realization that the 32-bit IPv4 address space was beginning to be used up, with new subnets and IP nodes being attached to the Internet (and being allocated unique IP addresses) at a breath-taking rate. To respond to this need for a large IP address space, a new IP protocol, IPv6, was developed. The designers of IPv6 also took this opportunity to tweak and augment other aspects of IPv4, based on the accumulated operational experience with IPv4.

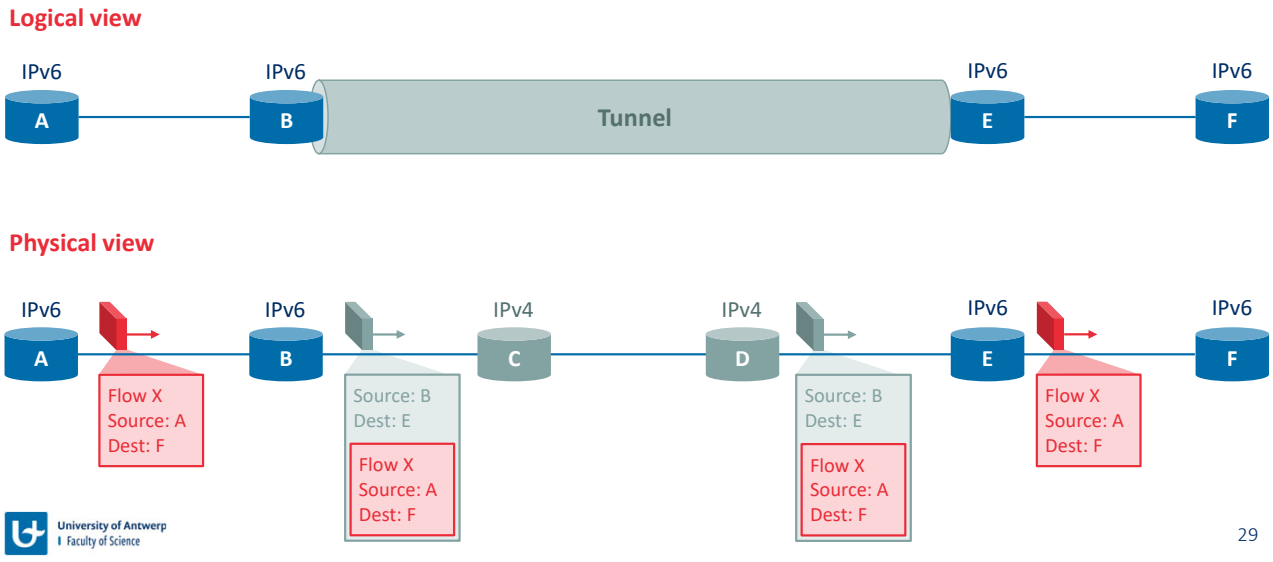
Although the mid-1990s estimates of IPv4 address depletion suggested that a considerable amount of time might be left until the IPv4 address space was exhausted, it was realized that considerable time would be needed to deploy a new technology on such an extensive scale, and so the process to develop IP version 6 (IPv6) [RFC 2460] was begun [RFC 1752]. An often-asked question is what happened to IPv5? It was initially envisioned that the ST-2 protocol would become IPv5, but ST-2 was later dropped.

The following fields are defined in IPv6:

- ***Version.*** This 4-bit field identifies the IP version number. Not surprisingly, IPv6 carries a value of 6 in this field. Note that putting a 4 in this field does not create a valid IPv4 datagram.
- ***Traffic class.*** The 8-bit traffic class field, like the TOS field in IPv4, can be used to give priority to certain datagrams within a flow, or it can be used to give priority to datagrams from certain applications (for example, voice-over-IP) over datagrams from other applications (for example, SMTP e-mail).
- ***Flow label.*** This 20-bit field is used to identify a flow of datagrams.

- ***Payload length.*** This 16-bit value is treated as an unsigned integer giving the number of bytes in the IPv6 datagram following the fixed-length, 40-byte datagram header.
- ***Next header.*** This field identifies the protocol to which the contents (data field) of this datagram will be delivered (for example, to TCP or UDP). The field uses the same values as the protocol field in the IPv4 header.
- ***Hop limit.*** The contents of this field are decremented by one by each router that forwards the datagram. If the hop limit count reaches zero, a router must discard that datagram.
- ***Source and destination addresses.*** The various formats of the IPv6 128-bit address are described in RFC 4291.
- ***Data.*** This is the payload portion of the IPv6 datagram. When the datagram reaches its destination, the payload will be removed from the IP datagram and passed on to the protocol specified in the next header field.

Tunnelling: IPv4 and IPv6 co-existence



The approach to IPv4-to-IPv6 transition that has been most widely adopted in practice involves **tunnelling** [RFC 4213]. The basic idea behind tunnelling is the following. Suppose two IPv6 nodes (in this example, B and E) want to interoperate using IPv6 datagrams but are connected to each other by intervening IPv4 routers. We refer to the intervening set of IPv4 routers between two IPv6 routers as a **tunnel**. With tunnelling, the IPv6 node on the sending side of the tunnel (in this example, B) takes the *entire* IPv6 datagram and puts it in the data (payload) field of an IPv4 datagram. This IPv4 datagram is then addressed to the IPv6 node on the receiving side of the tunnel (in this example, E) and sent to the first node in the tunnel (in this example, C). The intervening IPv4 routers in the tunnel route this IPv4 datagram among themselves, just as they would any other datagram, blissfully unaware that the IPv4 datagram itself contains a complete IPv6 datagram. The IPv6 node on the receiving side of the tunnel eventually receives the IPv4 datagram (it is the destination of the IPv4 datagram!), determines that the IPv4 datagram contains an IPv6 datagram (by observing that the protocol number field in the IPv4 datagram is 41 [RFC 4213], indicating that the IPv4 payload is a IPv6 datagram), extracts the IPv6 datagram, and then routes the IPv6 datagram exactly as it would if it had received the IPv6 datagram from a directly connected IPv6 neighbour.

Routing algorithms

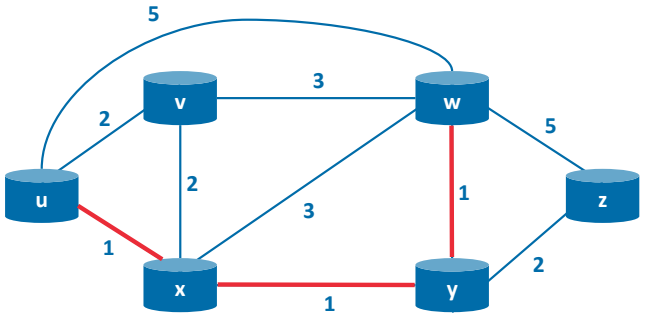
Routing algorithms

- **Goal:** Determine good paths (or routes), from senders to receivers, through the network of routers
- Routing problems are formulated using graph theory
  - A graph $G = (N, E)$ consists of a set of nodes (also called vertices) ***N*** and a set of edges ***E***
  - In network-layer routing, the nodes are routers, and the edges are physical links between these routers
  - Each edge between any pair of nodes (x, y) has a cost $c(x, y)$ associated with it (e.g., representing its length, monetary cost)
  - A routing algorithm tries to find the **least-cost path** of edges (x_1, x_2, ..., x_p) between the source x_1 and destination x_p

Example:

Least-cost path from u to w is (u, x, y, w)

Cost: $c(u, x) + c(x, y) + c(y, w) = 3$



University of Antwerp
Faculty of Science

In this section, we'll study **routing algorithms**, whose goal is to determine good paths (equivalently, routes), from senders to receivers, through the network of routers. Typically, a "good" path is one that has the least cost. We'll see that in practice, however, real-world concerns such as policy issues (for example, a rule such as "router *x*, belonging to organization *Y*, should not forward any packets originating from the network owned by organization *Z* ") also come into play. We note that whether the network control plane adopts a per-router control approach or a logically centralized approach, there must always be a well-defined sequence of routers that a packet will cross in traveling from sending to receiving host.

A graph is used to formulate routing problems. Recall that a **graph** *G* = (*N*, *E*) is a set *N* of nodes and a collection *E* of edges, where each edge is a pair of nodes from *N*. In the context of network-layer routing, the nodes in the graph represent routers—the points at which packet-forwarding decisions are made—and the edges connecting these nodes represent the physical links between these routers. Such a graph abstraction of a computer network is shown in the figure. When we study the BGP inter-domain routing protocol, we'll see that nodes represent networks, and the edge connecting two such nodes represents direct connectivity (known as peering) between the two networks.

An edge also has a value representing its cost. Typically, an edge's cost may reflect the physical length of the corresponding link (for example, a transoceanic link might have a higher cost than a short-haul terrestrial link), the link speed, or the monetary cost associated with a link. For our purposes, we'll simply take the edge costs as a given and won't worry about how they are determined. For any edge (*x*, *y*) in *E*, we denote *c*(*x*, *y*) as the cost of the

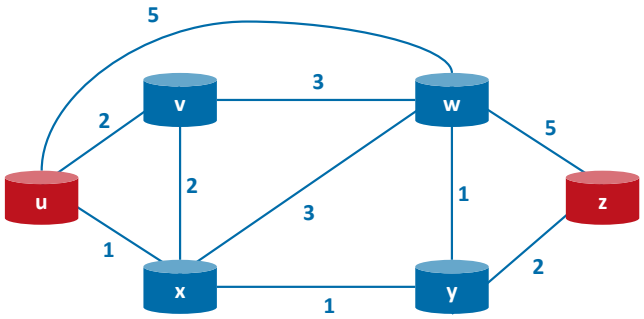
edge between nodes x and y. If the pair (x, y) does not belong to E, we set $c(x, y) = \infty$. Also, we'll only consider undirected graphs (i.e., graphs whose edges do not have a direction) in our discussion here, so that edge (x, y) is the same as edge (y, x) and that $c(x, y) = c(y, x)$; however, the algorithms we'll study can be easily extended to the case of directed links with a different cost in each direction. Also, a node y is said to be a **neighbour** of node x if (x, y) belongs to E.

Given that costs are assigned to the various edges in the graph abstraction, a natural goal of a routing algorithm is to identify the least costly paths between sources and destinations. To make this problem more precise, recall that a **path** in a graph $G = (N, E)$ is a sequence of nodes $(x_1, x_2, ..., x_p)$ such that each of the pairs (x_1, x_2), (x_2, x_3), ..., (x_{p-1}, x_p) are edges in E. The cost of a path $(x_1, x_2, ..., x_p)$ is simply the sum of all the edge costs along the path, that is, $c(x_1, x_2) + c(x_2, x_3) + ... + c(x_p-1, x_p)$. Given any two nodes x and y, there are typically many paths between the two nodes, with each path having a cost. One or more of these paths is a **least-cost path**. The least-cost problem is therefore clear: Find a path between the source and destination that has least cost.

For example, the least-cost path between source node u and destination node w is (u, x, y, w) with a path cost of 3. Note that if all edges in the graph have the same cost, the least-cost path is also the **shortest path** (that is, the path with the smallest number of links between the source and the destination).

Exercise: Find the least-cost path

Exercise: What is the **least-cost path** from node u to node z? What is the **least-hops path**?



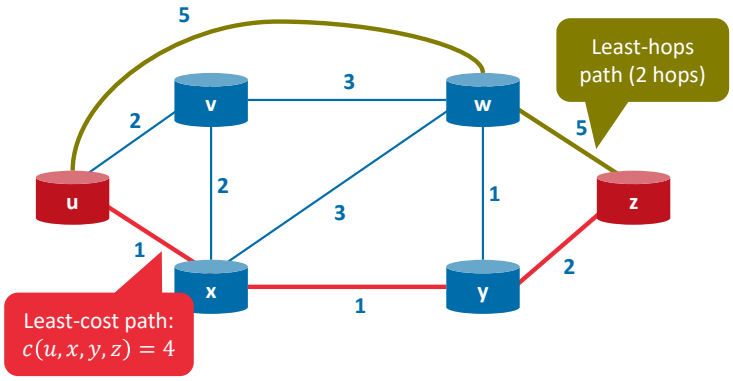
University of Antwerp
Faculty of Science

32

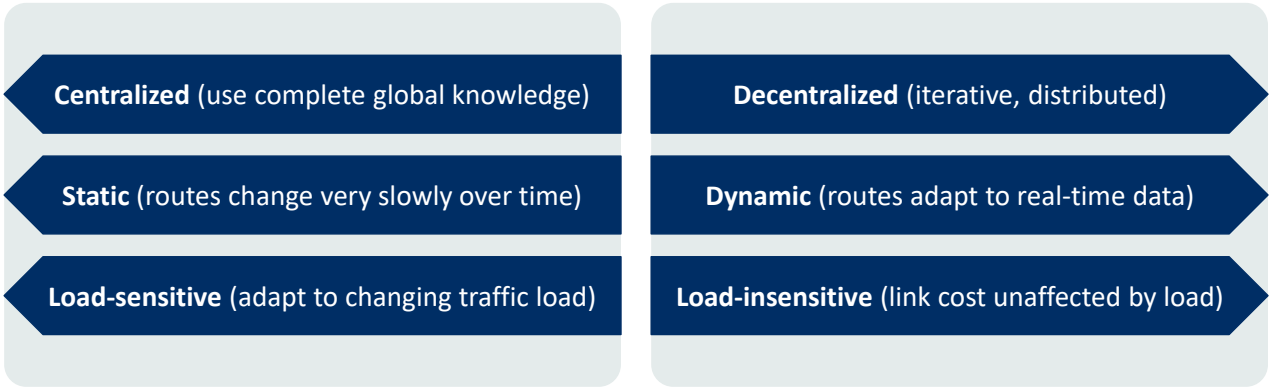
As a simple exercise, try finding the least-cost path from node *u* to *z* and reflect for a moment on how you calculated that path. If you are like most people, you found the path from *u* to *z* by examining Figure 5.3, tracing a few routes from *u* to *z*, and somehow convincing yourself that the path you had chosen had the least cost among all possible paths. (Did you check all of the 17 possible paths between *u* and *z*? Probably not!) Such a calculation is an example of a centralized routing algorithm—the routing algorithm was run in one location, your brain, with complete information about the network.

Solution: Find the least-cost path

Exercise: What is the **least-cost path** from node u to node z? What is the **shortest path**?



Classification of routing algorithms



University of Antwerp
Faculty of Science

34

Broadly, one way in which we can classify routing algorithms is according to whether they are centralized or decentralized:

- A **centralized routing algorithm** computes the least-cost path between a source and destination using complete, global knowledge about the network. That is, the algorithm takes the connectivity between all nodes and all link costs as inputs. This then requires that the algorithm somehow obtains this information before it performs the calculation. The calculation itself can be run at one site (e.g., a logically centralized controller) or could be replicated in the routing component of each and every router. The key distinguishing feature here, however, is that the algorithm has complete information about connectivity and link costs. Algorithms with global state information are often referred to as **link-state (LS) algorithms**, since the algorithm must be aware of the cost of each link in the network.
- In a **decentralized routing algorithm**, the calculation of the least-cost path is carried out in an iterative, distributed manner by the routers. No node has complete information about the costs of all network links. Instead, each node begins with only the knowledge of the costs of its own directly attached links. Then, through an iterative process of calculation and exchange of information with its neighboring nodes, a node gradually calculates the least-cost path to a destination or set of destinations. The decentralized routing algorithm we'll study is called a **distance-vector (DV) algorithm**, because each node maintains a vector of estimates of the costs (distances) to all other nodes in the network. Such decentralized algorithms, with interactive message exchange between neighboring routers is perhaps more naturally suited to control planes where the routers interact directly with each other.

A second broad way to classify routing algorithms is according to whether they are static or

dynamic:

- In **static routing algorithms**, routes change very slowly over time, often as a result of human intervention (for example, a human manually editing a link cost).
- **Dynamic routing algorithms** change the routing paths as the network traffic loads or topology change. A dynamic algorithm can be run either periodically or in direct response to topology or link cost changes. While dynamic algorithms are more responsive to network changes, they are also more susceptible to problems such as routing loops and route oscillation.

A third way to classify routing algorithms is according to whether they are load-sensitive or load-insensitive:

- In a **load-sensitive algorithm**, link costs vary dynamically to reflect the current level of congestion in the underlying link. If a high cost is associated with a link that is currently congested, a routing algorithm will tend to choose routes around such a congested link. While early ARPAnet routing algorithms were load-sensitive, a number of difficulties were encountered (e.g., massive route oscillations).
- Today's Internet routing algorithms (such as RIP, OSPF, and BGP) are **load-insensitive**, as a link's cost does not explicitly reflect its current (or recent past) level of congestion. Additional protocols are used then in case load balancing is required.

The (centralized) link-state (LS) routing algorithm

- Centralized routing algorithm that assumes the entire network topology and edge costs are known
- Can apply known least-cost path algorithms (e.g., Dijkstra's algorithm)
- Notations
  - N: set of all nodes
  - s: source node
  - $D(v)$: cost of the currently known least-cost path from the source node s to destination v
  - $p(v)$: previous node along the current least-cost path from the source s to v
  - N': subset of nodes for which the least-cost path from the source s is known
- **Dijkstra's algorithm:**

Initialization: $N' = \{s\}$

$\forall v \in N: D(v) = c(s,v)$

$c(s,v) = \infty$ if s and v are not direct neighbours

Repeat: $w = \underset{v \notin N'}{\operatorname{argmin}} D(v)$

$N' = N' \cup \{w\}$

$\forall$ neighbours v *of* $w \wedge v \notin N': D(v) = \min\big(D(v), D(w) + c(w,v)\big)$

Until: $N' = N$

University of Antwerp | Faculty of Science

35

In a link-state algorithm, the network topology and all link costs are known, that is, available as input to the LS algorithm. In practice, this is accomplished by having each node broadcast link-state packets to *all* other nodes in the network, with each link-state packet containing the identities and costs of its attached links. In practice (for example, with the Internet's OSPF routing protocol), this is often accomplished by a **link-state broadcast** algorithm. The result of the nodes' broadcast is that all nodes have an identical and complete view of the network. Each node can then run the LS algorithm and compute the same set of least-cost paths as every other node.

The link-state routing algorithm we present below is known as *Dijkstra's algorithm*, named after its inventor. Dijkstra's algorithm computes the least-cost path from one node (the source, which we will refer to as *u*) to all other nodes in the network. Dijkstra's algorithm is iterative and has the property that after the *k*th iteration of the algorithm, the least-cost paths are known to *k* destination nodes, and among the least-cost paths to all destination nodes, these *k* paths will have the *k* smallest costs. Let us define the following notation:

- *D*(*v*): cost of the least-cost path from the source node to destination *v* as of this iteration of the algorithm.
- *p*(*v*): previous node (neighbor of *v*) along the current least-cost path from the source to *v*.
- *N'*: subset of nodes; *v* is in *N* if the least-cost path from the source to *v* is definitively known.

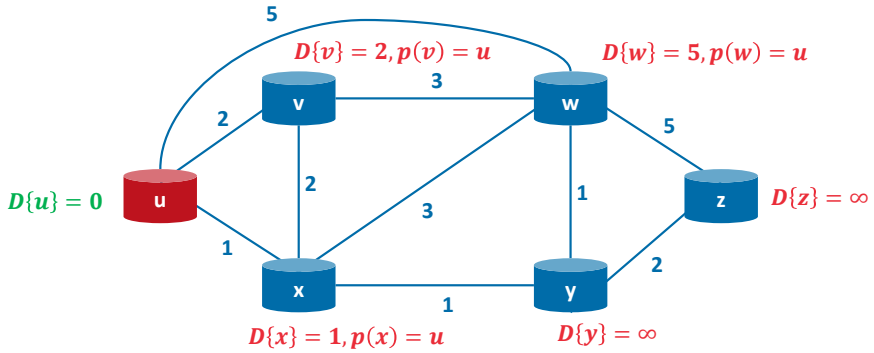
The centralized routing algorithm consists of an initialization step followed by a loop. The number of times the loop is executed is equal to the number of nodes in the network. Upon

termination, the algorithm will have calculated the shortest paths from the source node *u* to every other node in the network.

Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Initialization

$N' = \{u\}$



University of Antwerp
Faculty of Science

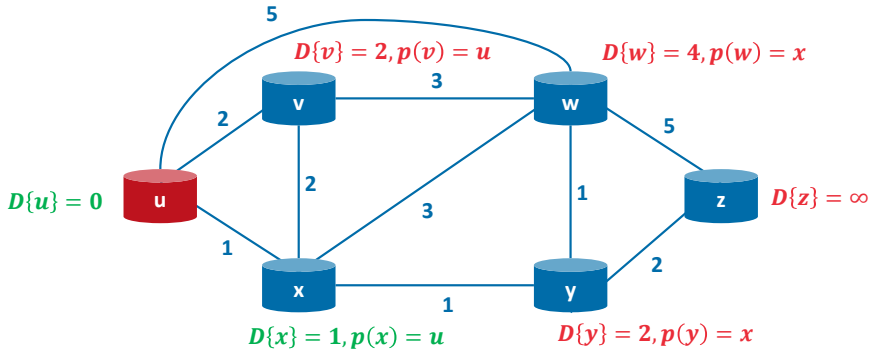
36

As an example, let's consider the network from before and compute the least-cost paths from *u* to all possible destinations. In the initialization step, the currently known least-cost paths from *u* to its directly attached neighbours, *v, x*, and *w*, are initialized to 2, 1, and 5, respectively. Note in particular that the cost to *w* is set to 5 (even though we will soon see that a lesser-cost path does indeed exist) since this is the cost of the direct (one hop) link from *u* to *w.* The costs to *y* and *z* are set to infinity because they are not directly connected to *u*.

Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 1

$N' = \{u, x\}$



University of Antwerp
Faculty of Science

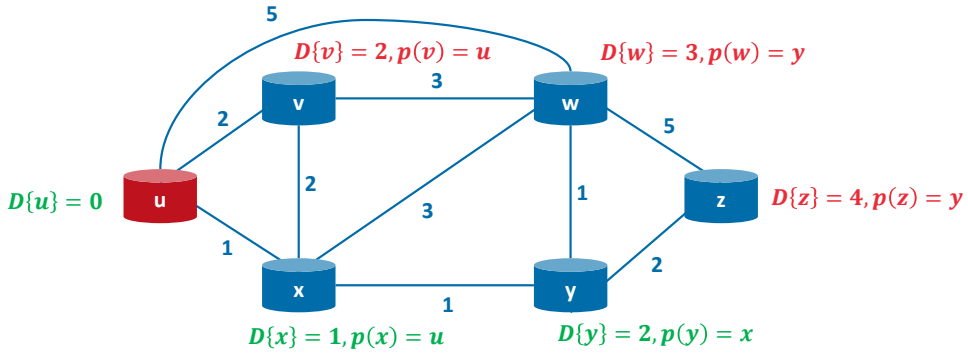
37

In the first iteration, we look among those nodes not yet added to the set *N* and find that node with the least cost as of the end of the previous iteration. That node is *x*, with a cost of 1, and thus *x* is added to the set *N*. We then update *D*(*v*) for all nodes *v*, yielding the shown results. The cost of the path to *v* is unchanged. The cost of the path to *w* (which was 5 at the end of the initialization) through node *x* is found to have a cost of 4. Hence this lower-cost path is selected and *w*'s predecessor along the shortest path from *u* is set to *x*. Similarly, the cost to *y* (through *x*) is computed to be 2, and the table is updated accordingly.

Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 2

$N' = \{u, x, y\}$



University of Antwerp
Faculty of Science

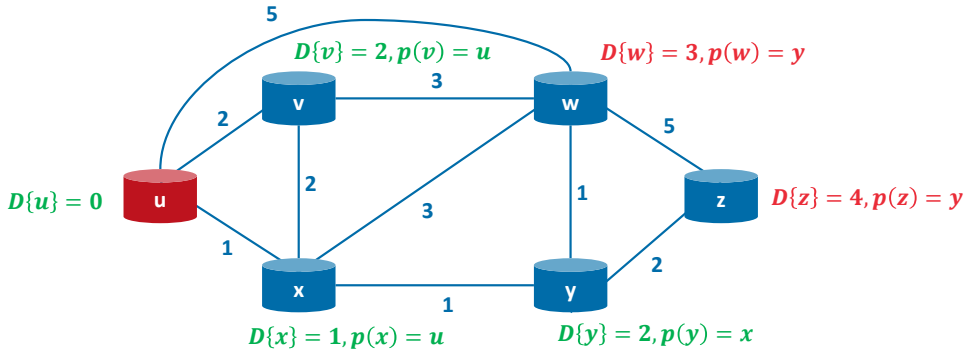
38

In the second iteration, nodes *v* and *y* are found to have the least-cost paths (2), and we break the tie arbitrarily and add *y* to the set *N'* so that *N'* now contains *u, x*, and *y.* The cost to the remaining nodes not yet in *N*, that is, nodes *v, w*, and *z*, are updated, yielding the results shown in the figure.

Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 3

$N' = \{u, x, y, v\}$



University of Antwerp
Faculty of Science

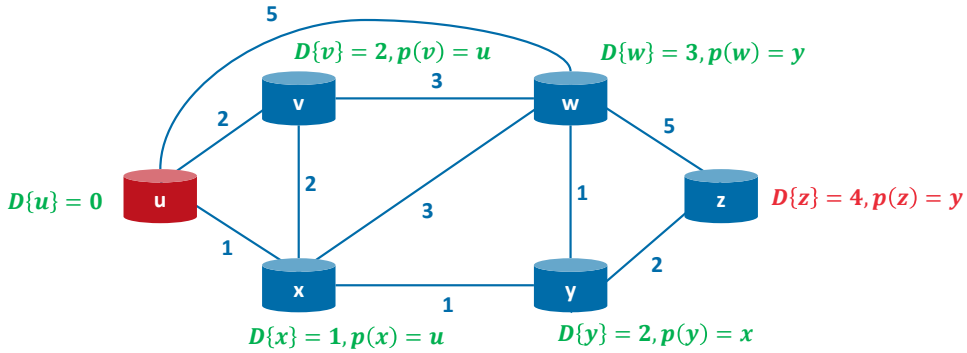
39

In the third iteration, node v has the least-cost path (2), and is added to N'. This does not result in a shorter cost path being found for nodes w and z, so their value D() and p() remains the same.

Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 4

$N' = \{u, x, y, v, w\}$



University of Antwerp
Faculty of Science

40

In the fourth iteration, node w has the least-cost path (3), and is added to N'. This does not result in a shorter cost path being found for nodes z, so its value D(z) and p(z) remains the same.

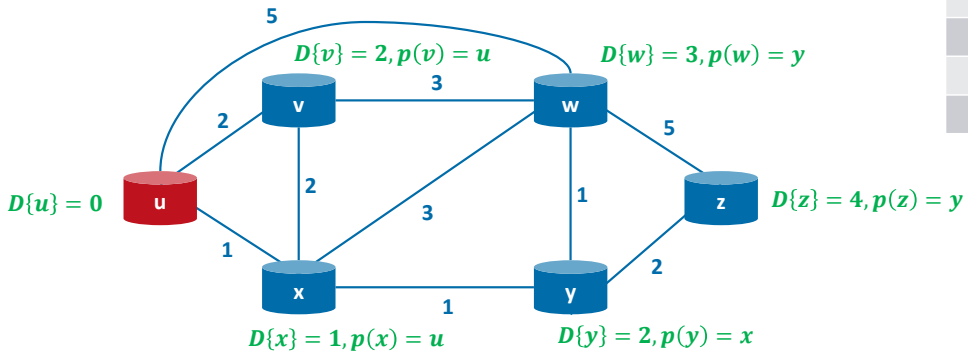
Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 5

$N' = \{u, x, y, v, w, z\} = N$

Forwarding table for node u:

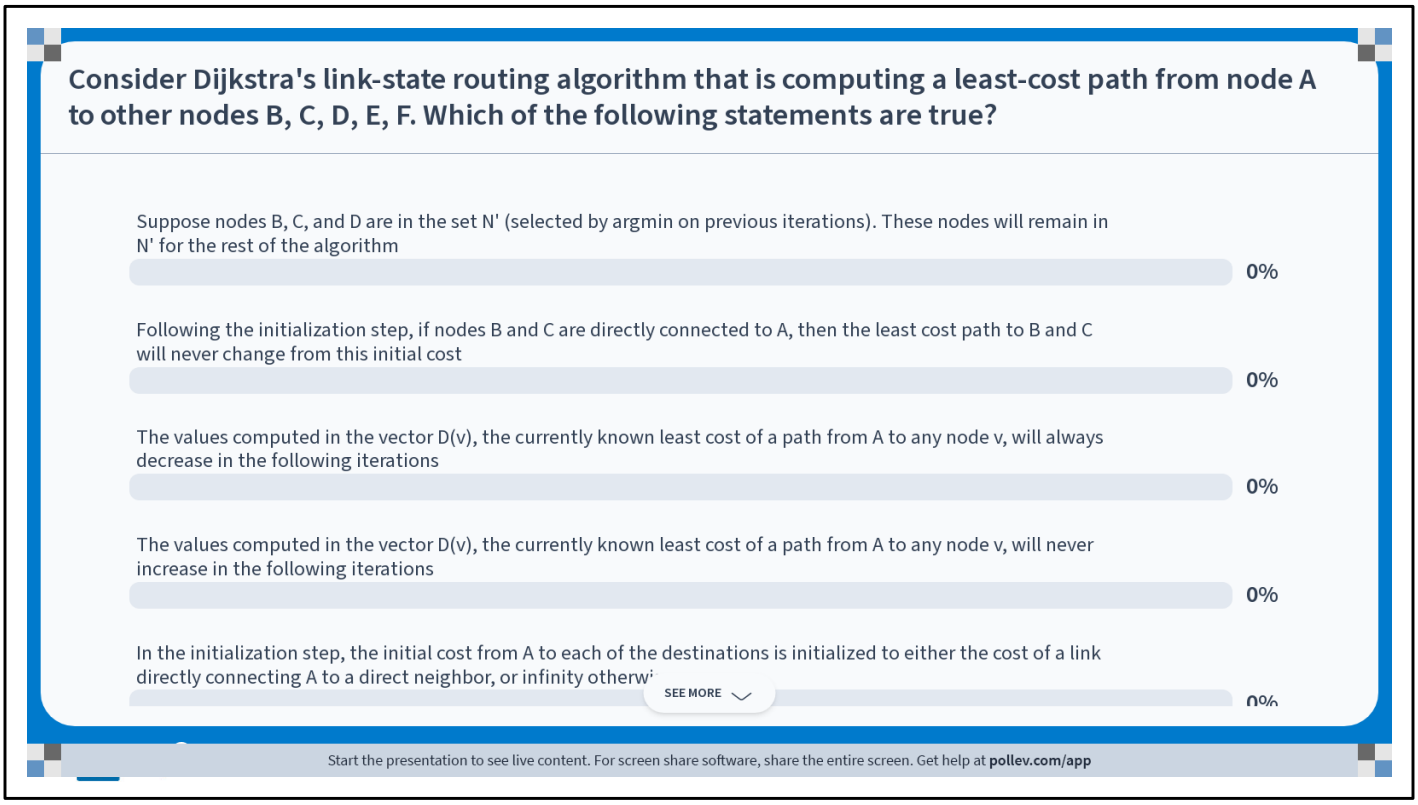
Destination	Next Hop
v	v
w	x
x	x
y	x
z	x



University of Antwerp
Faculty of Science

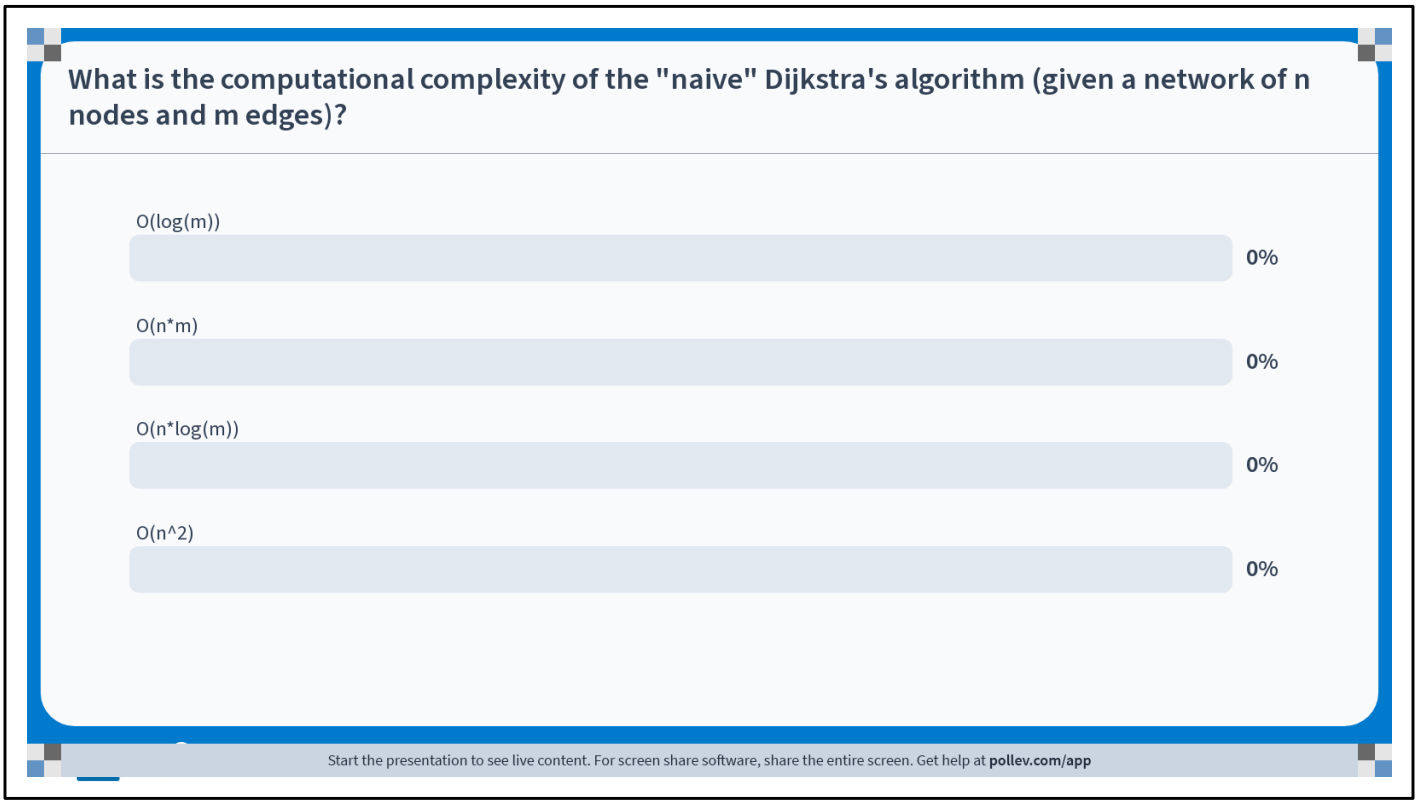
41

In the fifth and final iteration, node z has the least-cost path (4), and is added to N'. Now N' = N and the algorithm finishes. When the LS algorithm terminates, we have, for each node, its predecessor along the least-cost path from the source node. For each predecessor, we also have *its* predecessor, and so in this manner we can construct the entire path from the source to all destinations. The forwarding table in a node, say node *u*, can then be constructed from this information by storing, for each destination, the next-hop node on the least-cost path from *u* to the destination.



Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity. More info at polleverywhere.com/support

Consider Dijkstra's link-state routing algorithm that is computing a least-cost path from node A to other nodes B, C, D, E, F. Which of the following statements are true?
https://www.polleverywhere.com/multiple_choice_polls/ktEDp7FDxp0bWf0UkuRFw?state=opened&flow=Default&onscreen=persist



Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity. More info at polleverywhere.com/support

What is the computational complexity of the "naive" Dijkstra's algorithm (given a network of n nodes and m edges)?
https://www.polleverywhere.com/multiple_choice_polls/KEcle04iitx1G2Mhfl1tV?state=opened&flow=Default&onscreen=persist

Computational complexity of naive Dijkstra's algorithm

Initialization:

$N' = \{s\}$

$\forall v \in N: D(v) = c(s, v)$

Repeat:

$w = \underset{v \notin N'}{\operatorname{argmin}} D(v)$

In iteration i, we need to search n-i nodes

$N' = N' \cup \{w\}$

$\forall$ neighbours v *of* $w \wedge v \notin N': D(v) = \min\big(D(v), D(w) + c(w, v)\big)$

Until:

$N' = N$

Total number of edges considered is *m*

In worst case, $\mathrm{m} = \frac{n \times (n-1)}{2} = O(n^2)$

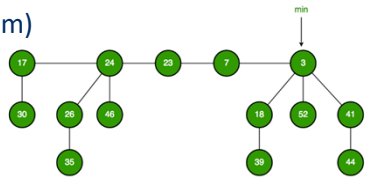
Total number of nodes visited is

$(n-1) + (n-2) + \cdots + 1 = \frac{n \times (n-1)}{2} = O(n^2)$

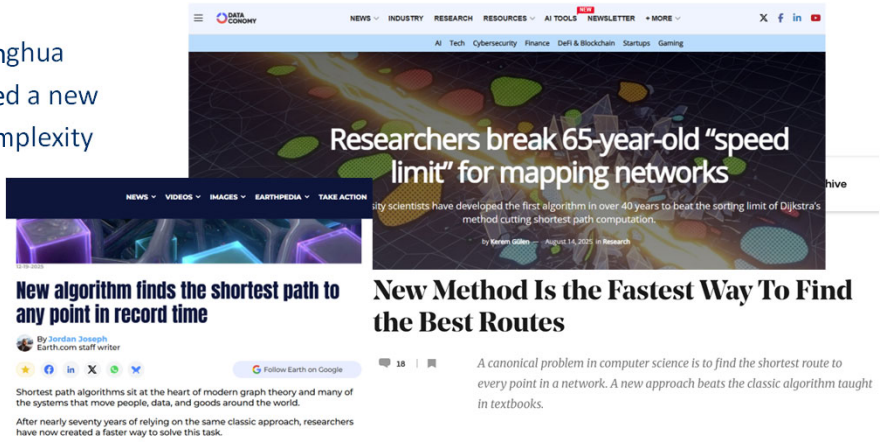
What is the computational complexity of this algorithm? That is, given *n* nodes, how much computation must be done in the worst case to find the least-cost paths from the source to all destinations? In the first iteration, we need to search through *n - 1* nodes (we exclude only the source node) to determine the node *w*, which is not in *N'* and has the minimum cost *D*. In the second iteration, we need to check *n* - 2 nodes to determine the minimum cost; in the third iteration *n* - 3 nodes, and so on. On each iteration, we also go through the edges that start at node *w* and end at it's neighbours v, which are not in *N'*. Overall, the total number of nodes we need to search through over all the iterations is *n**(*n* - 1)/2, and through all edges *m,* and thus, we say that the "naive" implementation of the LS algorithm has worst-case complexity of $O(m+n^2)$. In a worst case, for a fully-connected graph, m = n*(n-1)/2 ~ n^2 => complexity can be rewritten as simply $O(n^2)$.

Computational complexity of Dijkstra's algorithm

- Implementation using Fibonacci heap has lower complexity: O(n * log n + m)
  - O(log n) to extract minimum value
  - O(1) to update element
  - Considered as "speed limit" for > 40 years
  - Usage of binary heap instead of Fibonacci heap is widely used due to lower constants



- In 2025, scientists from Tsinghua University in Beijing developed a new algorithm: O(m * $\log^{2/3}$ n) complexity
  - Becomes better than classical implementations for graphs with more than 10^{19} nodes (according to L. Castro et al. "Implementation and brief experimental analysis of the Duan et al. (2025) algorithm for single-source shortest paths")



The (decentralized) distance vector (DV) routing algorithm

- Distributed routing algorithm where each node only relies on information obtained from neighbours
- Relies on the Bellman-Ford equation:

$$d_x(y) = \min_v\{c(x,v) + D_v(y)\}$$

where $D_x(y)$ is the least-cost distance from x to y

- The solution to the Bellman-Ford equation provides node x's forwarding table.
- Each node x keeps track of
  - An estimate distance vector $\boldsymbol{D}_x = [D_x(y): \forall\, y \in N]$
  - The cost $c(x,v)$ to each of its direct neighbours v
  - The distance vector of each of its neighbours v, $\boldsymbol{D}_v = [D_v(y): \forall\, y \in N]$

Whereas the LS algorithm is an algorithm using global information, the **distance-vector (DV)** algorithm is iterative, asynchronous, and distributed. It is *distributed* in that each node receives some information from one or more of its *directly attached* neighbours, performs a calculation, and then distributes the results of its calculation back to its neighbours. It is *iterative* in that this process continues on until no more information is exchanged between neighbours. The algorithm is *asynchronous* in that it does not require all of the nodes to operate in lockstep with each other.

Before we present the DV algorithm, it will prove beneficial to discuss an important relationship that exists among the costs of the least-cost paths. Let $d_x(y)$ be the cost of the least-cost path from node *x* to node *y*. Then the least costs are related by the celebrated Bellman-Ford equation, namely, $d_x(y) = \min_v(c(x, v) + d_v(y))$, where the min_v in the equation is taken over all of *x*'s neighbours. The Bellman-Ford equation is rather intuitive. Indeed, after traveling from *x* to *v*, if we then take the least-cost path from *v* to *y*, the path cost will be $c(x, v) + d_v(y)$. Since we must begin by traveling to some neighbour *v*, the least cost from *x* to *y* is the minimum of $c(x, v) + d_v(y)$ taken over all neighbours *v*.

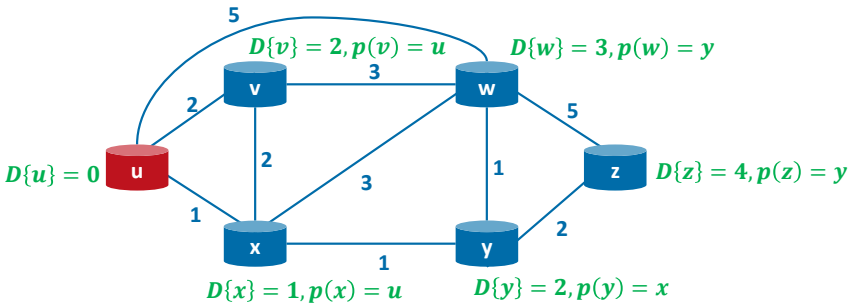
The Bellman-Ford equation is not just an intellectual curiosity. It actually has significant practical importance: the solution to the Bellman-Ford equation provides the entries in node *x*'s forwarding table. To see this, let *v** be any neighbouring node that achieves the minimum in Bellman-Ford's equation. Then, if node *x* wants to send a packet to node *y* along a least-cost path, it should first forward the packet to node *v**. Thus, node *x*'s forwarding table would specify node *v** as the next-hop router for the ultimate destination *y*.

The (decentralized) distance vector (DV) routing algorithm

Relies on the Bellman-Ford equation:

$$d_x(y) = \min_v\{c(x,v) + d_v(y)\}$$

where $d_x(y)$ is the least-cost distance from x to y



Homework: Check if the Bellman-Ford equation holds for source node u and destination node z in the previous example graph.

DV algorithm pseudocode (at each node x)

Initialization:

$\forall\, y \in N\colon\ D_x(y) = c(x,y)$

$\forall$ neighbours of w: send $D_x = [D_x(y)\colon\ \forall\, y \in N]$ to w

Repeat: (whenever link cost changes, or distance vector received from neighbour)

$\forall\, y \in N\colon\ D_x(y) = \min_v\{c(x,v) + D_v(y)\}$

set $v^* = \text{argmin}\{c(x,v) + D_v(y)\}$ as next hop for dest y

if $D_x(y)$ changed for any destination y:

$\forall$ neighbours of w: send $D_x = [D_x(y)\colon\ \forall\, y \in N]$ to w

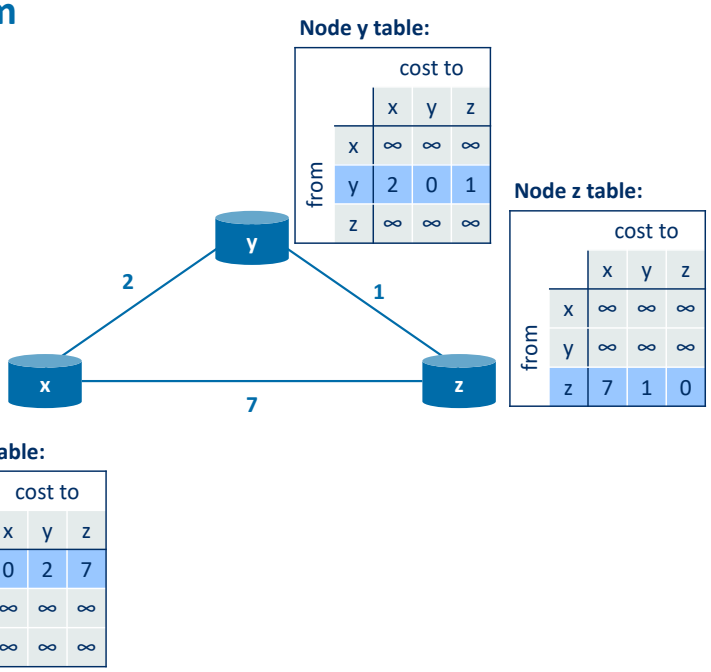
forever

In the DV algorithm, a node *x* updates its distance-vector estimate when it either sees a cost change in one of its directly attached links or receives a distance-vector update from some neighbour. But to update its own forwarding table for a given destination *y*, what node *x* really needs to know is not the shortest-path distance to *y* but instead the neighbouring node *v**(*y*) that is the next-hop router along the shortest path to *y*. As you might expect, the next-hop router *v**(*y*) is the neighbour *v* that achieves the minimum. Thus, for each destination *y*, node *x* also determines *v**(*y*) and updates its forwarding table for destination *y*.

Recall that the LS algorithm is a centralized algorithm in the sense that it requires each node to first obtain a complete map of the network before running the Dijkstra algorithm. The DV algorithm is *decentralized* and does not use such global information. Indeed, the only information a node will have is the costs of the links to its directly attached neighbours and information it receives from these neighbours. Each node waits for an update from any neighbour, calculates its new distance vector when receiving an update, and distributes its new distance vector to its neighbours.

Example: DV algorithm

Initialization

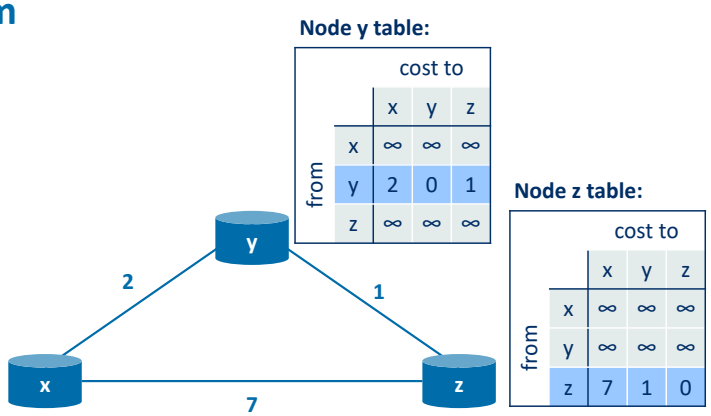


Let's illustrates the operation of the DV algorithm for the simple three-node network. The figure displays three initial **routing tables** for each of the three nodes. Within a specific routing table, each row is a distance vector—specifically, each node's routing table includes its own distance vector and that of each of its neighbours. Thus, the first row in node *x*'s initial routing table is $\boldsymbol{D}_x = [D_x(x), D_x(y), D_x(z)] = [0, 2, 7]$. The second and third rows in this table are the most recently received distance vectors from nodes *y* and *z*, respectively. Because at initialization node *x* has not received anything from node *y* or *z*, the entries in the second and third rows are initialized to infinity.

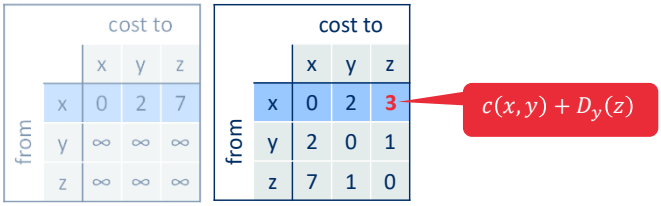
Example: DV algorithm

Step 1

Node x received $\boldsymbol{D}_y$ and $\boldsymbol{D}_z$



Node x table:



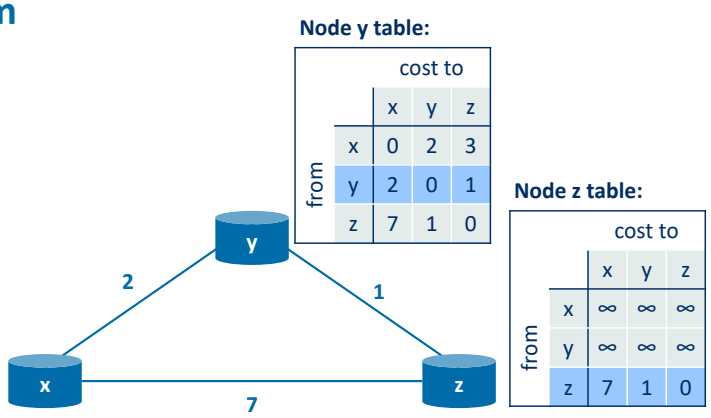
After initialization, each node sends its distance vector to each of its two neighbours. For example, node *x* receives distance vectors $\boldsymbol{D}_y$ = [2, 0, 1] and $\boldsymbol{D}_z$ = [7, 1, 0] from both nodes *y* and *z*. After receiving the updates, node x recomputes its own distance vector:

$$D_x(x) = 0$$
$$D_x(y) = \min\{c(x,y) + D_y(y),\ c(x,z) + D_z(y)\} = \min\{2+0, 7+1\} = 2$$
$$D_x(z) = \min\{c(x,z) + D_z(z),\ c(x,y) + D_y(z)\} = \min\{7+0, 2+1\} = 3$$

Example: DV algorithm

Step 2

Node y receives D_x and D_z

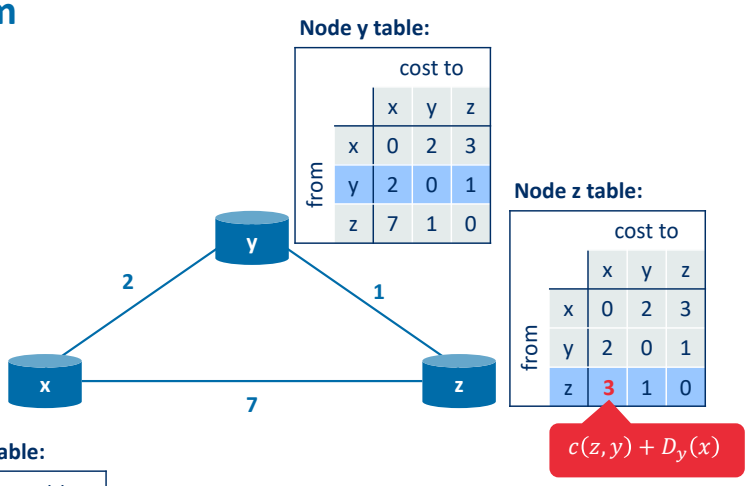


Subsequently, node y receives the distance vectors from nodes x and z. It recomputes its own distance vector, but nothing changes.

Example: DV algorithm

Step 3

Node z receives $\boldsymbol{D}_x$ and $\boldsymbol{D}_y$



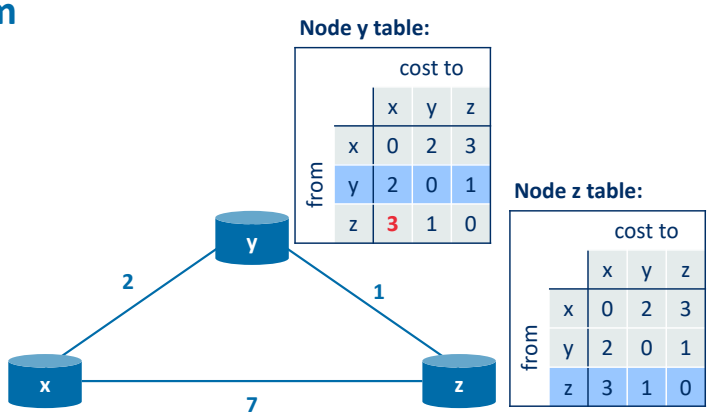
University of Antwerp
Faculty of Science

Next, node z receives the distance vectors from nodes x and y. It also recalculates its own distance vector, and discovers a change in $D_z(x)$, as $D_z(x)=c(z, y)+D_y(x) = 3$, which is less than the previous value 7.

Example: DV algorithm

Step 4

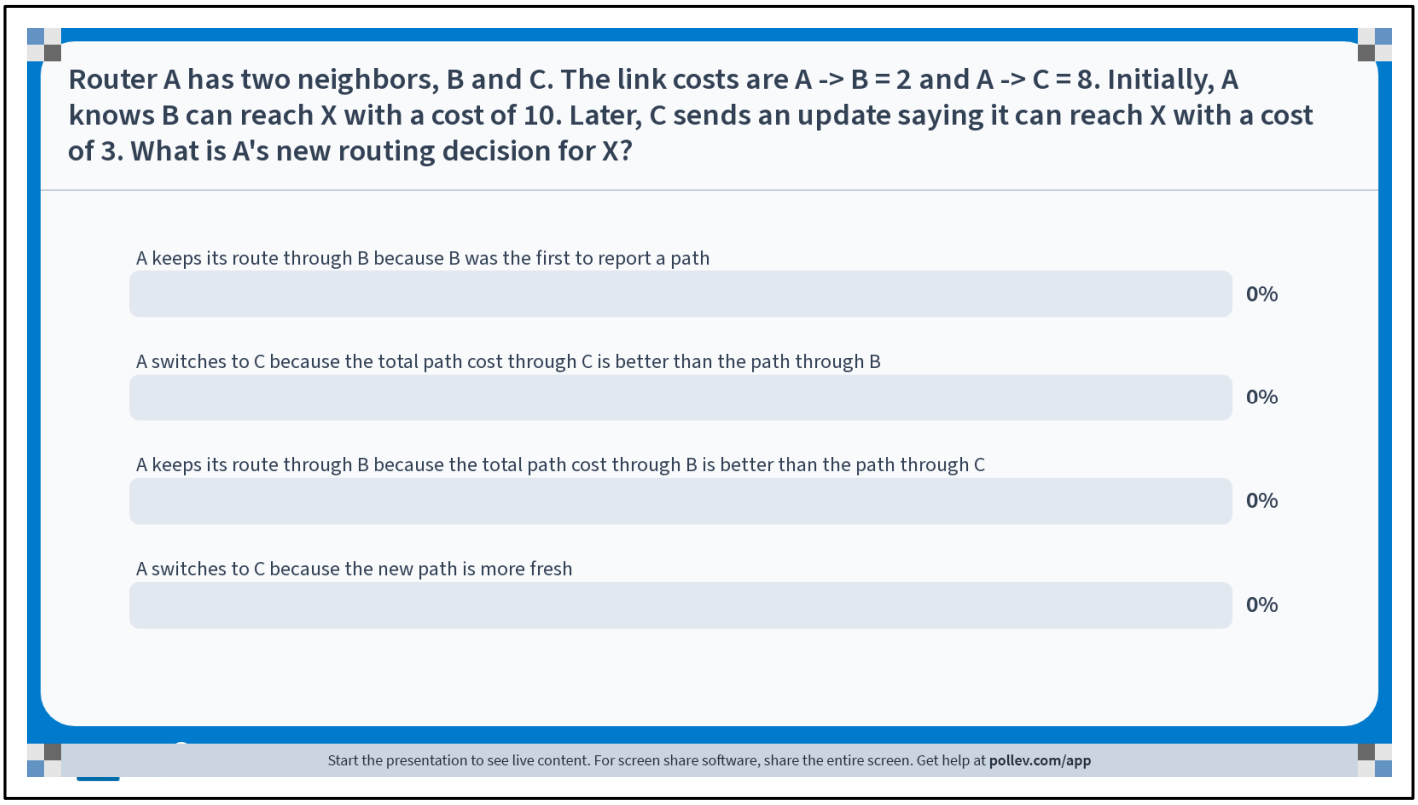
Node x and y receive D_z



University of Antwerp
Faculty of Science

55

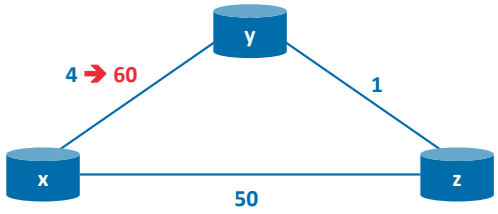
Finally, node z sends out its distance vector to x and y again, as it changed in step 3. Nodes x and y update their forwarding table, but do not need to make any more changes to their own distance vectors. All three nodes have the same information in their forwarding tables. At this point, since no update messages are sent, no further routing table calculations will occur and the algorithm will enter a quiescent state; that is, all nodes will be performing the wait of the DV algorithm. The algorithm remains in the quiescent state until a link cost changes, as discussed next.



Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity. More info at polleverywhere.com/support

Router A has two neighbors, B and C. The link costs are A -> B = 2 and A -> C = 8. Initially, A knows B can reach X with a cost of 10. Later, C sends an update saying it can reach X with a cost of 3. What is A's new routing decision for X?
https://www.polleverywhere.com/multiple_choice_polls/mUdkXHFApBbHuv0Xkymar?state=opened&flow=Default&onscreen=persist

Routing loops in DV due to link cost increase (count-to-infinity problem)



Node y table:

Before change:

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

Step 1: cost change, advertise D_y to x and z

from \ cost to	x	y	z
x	0	4	5
y	6	0	1
z	5	1	0

Routing loop!

Step 2: receive D_x and D_z

from \ cost to	x	y	z
x	0	51	50
y	6	0	1
z	7	1	0

Step 3: recalculate DVs, advertise D_y to x and z

from \ cost to	x	y	z
x	0	51	50
y	8	0	1
z	7	1	0

...

Step 46: self-correction

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

University of Antwerp | Faculty of Science

58

Let's now consider what can happen when a link cost *increases.* Suppose that the link cost between *x* and *y* increases from 4 to 60:

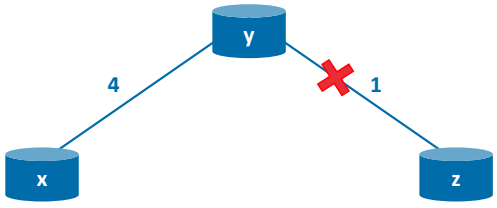
- Before the link cost changes, $D_y(x) = 4$, $D_y(z) = 1$, $D_z(y) = 1$, and $D_z(x) = 5$. At time t_0, *y* detects the link-cost change (the cost has changed from 4 to 60). *y* computes its new minimum-cost path to *x* to have a cost of:

$$D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60 + 0, 1 + 5\} = 6$$

- Of course, with our global view of the network, we can see that this new cost via *z* is *wrong.* But the only information node *y* has is that its direct cost to *x* is 60 and that *z* has last told *y* that *z* could get to *x* with a cost of 5. So in order to get to *x, y* would now route through *z*, fully expecting that *z* will be able to get to *x* with a cost of 5. As of *t*1 we have a **routing loop**—in order to get to *x, y* routes through *z*, and *z* routes through *y*. A routing loop is like a black hole—a packet destined for *x* arriving at *y* or *z* as of t_1 will bounce back and forth between these two nodes forever (or until the forwarding tables are changed).
- Since node *y* has computed a new minimum cost to *x*, it informs *z* of its new distance vector at time t_1. Sometime after t_1, *z* receives *y*'s new distance vector, which indicates that *y*'s minimum cost to *x* is 6. *z* knows it can get to *y* with a cost of 1 and hence computes a new least cost to *x* of $D_z(x) = \min\{50 + 0, 1 + 6\} = 7$. Since *z*'s least cost to *x* has increased, it then informs *y* of its new distance vector at t_2.
- In a similar manner, after receiving *z*'s new distance vector, *y* determines $D_y(x) = 8$ and sends *z* its distance vector. *z* then determines $D_z(x) = 9$ and sends *y* its distance vector, and so on.

How long will the process continue? You should convince yourself that the loop will persist for 44 more iterations (message exchanges between *y* and *z*)—until *z* eventually computes the cost of its path via *y* to be greater than 50. At this point, *z* will (finally!) determine that its least-cost path to *x* is via its direct connection to *x*. *y* will then route to *x* via *z*. The result of the bad news about the increase in link cost has indeed travelled slowly! What would have happened if the link cost $c(y, x)$ had changed from 4 to 10,000 and the cost $c(z, x)$ had been 9,999? Because of such scenarios, the problem we have seen is sometimes referred to as the **count-to-infinity problem**.

Routing loops in DV due to link loss (count-to-infinity problem)



Node y table:

Before loss:

from \ cost to	x	y	z
x	0	4	5
y	4	0	1
z	5	1	0

Step 1: link loss, advertise D_y to x

from \ cost to	x	y	z
x	0	4	5
y	4	0	9
z	5	1	0

Step 2: receive D_x, recalculate DVs

from \ cost to	x	y	z
x	0	4	13
y	4	0	9
z	5	1	0

...

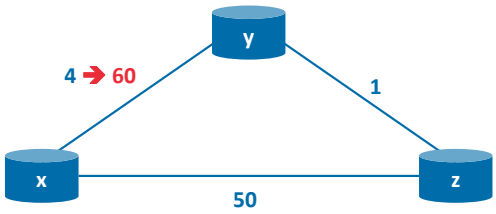
University of Antwerp | Faculty of Science

59

The situation can be come even worse in case of link loss. Let's consider what will happen when a link between y and z is lost, e.g., because of the interfaces went down.

- Before the link cost changes, $D_y(x) = 4$, $D_y(z) = 1$, $D_z(y) = 1$, and $D_z(x) = 5$. At time t_0, *y* detects the link loss. *y* computes its new minimum-cost path to *z* to have a cost of $D_y(z) = c(y,x) + D_x(x) = 4 + 5 = 9$.
- As of *t*1 we have a **routing loop** again — in order to get to *z, y* routes through *x*, and *x* still thinks that it can route through *y* to *z.*
- Since node *y* has computed a new minimum cost to *z*, it informs *x* of its new distance vector at time t_1. Sometime after t_1, *x* receives *y*'s new distance vector, which indicates that *y*'s minimum cost to *z* is 9. *x* knows it can get to *y* with a cost of 4 and hence computes a new least cost to *z* of $D_x(z) = 4 + 9 = 13$. Since *x*'s least cost to *z* has increased, it then informs *y* of its new distance vector at t_2.
- Unlike the previous situation, in this case the process can never stop, so it becomes literally "counting to infinity". To resolve this issue, an arbitrarily selected big enough value can be selected to represent infinity. When the cost reaches this value, it is assumed that the node is unreachable.

Poisoned reverse



Solution – Poisoned Reverse:
If a node x routes through y to get to z, then x will always tell y that $D_x(z)=\infty$

Node y table

Before change:

from \ cost to	x	y	z
x	0	4	∞
y	4	0	1
z	∞	1	0

Poisoned reverse!

Node y table

Step 1: cost change

from \ cost to	x	y	z
x	0	4	∞
y	60	0	1
z	5	1	0

Advertisement D_y

Step 2: Advertise D_y to z

from \ cost to	x	y	z
y	60	0	1

Node z table

Step 3: receive D_y, recalculate DVs

from \ cost to	x	y	z
x	0	∞	50
y	60	0	1
z	50	1	0

Node y table

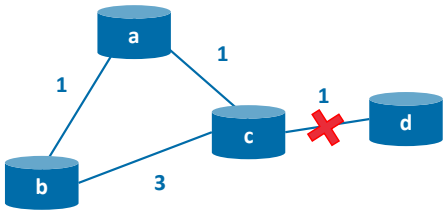
Step 4: receive D_z, self-correction

from \ cost to	x	y	z
x	0	51	50
y	51	0	1
z	50	1	0

60

The specific looping scenario just described can be avoided using a technique known as ***poisoned reverse***. The idea is simple—if *x* routes through *y* to get to destination *z*, then *x* will advertise to *y* that its distance to *z* is infinity, that is, *x* will advertise to *y* that $D_x(z) = \infty$ (even though *x* thinks/knows the value of $D_x(z)$). *x* will continue telling this little white lie to *y* as long as it routes to *z* via *y*. Since *y* believes that *x* has no path to *z*, *y* will never attempt to route to *z* via *x*, as long as *x* continues to route to *z* via *y* (and lie about doing so).

Poisoned reverse



Poisoned Reverse does not always work

If a node y routes through z to get to x, then y will always tell z that $D_y(x)=\infty$

Node c table

Before change:

from \ cost to	a	b	c	d
a	0	1	1	∞
b	1	0	2	3
c	1	2	0	1
d	∞	∞	1	0

Node c table

Step 1: link loss, update DVs

from \ cost to	a	b	c	d
a	0	1	1	∞
b	1	0	2	3
c	1	2	0	6
d	∞	∞	1	0

Node a table

Step 2: update DVs at node a after D_c

from \ cost to	a	b	c	d
a	0	1	1	7
b	1	0	∞	∞
c	1	∞	0	6

Node b table

Step 3: update DVs at node b after D_c and D_a

from \ cost to	a	b	c	d
a	0	1	1	7
b	1	0	2	8
c	1	2	0	6

Node c table

Step 4: update DVs at node b after D_a and D_b

from \ cost to	a	b	c	d
a	0	1	1	∞
b	1	0	2	8
c	1	2	0	11
d	∞	∞	1	0

...

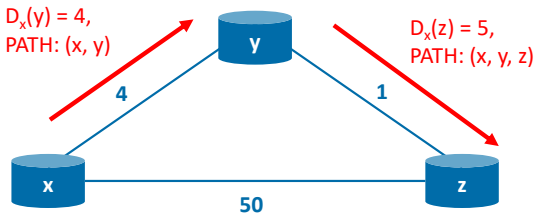
University of Antwerp
Faculty of Science

61

Does poisoned reverse solve the general count-to-infinity problem? It does not. You should convince yourself that loops involving three or more nodes (rather than simply two immediately neighboring nodes) will not be detected by the poisoned reverse technique.

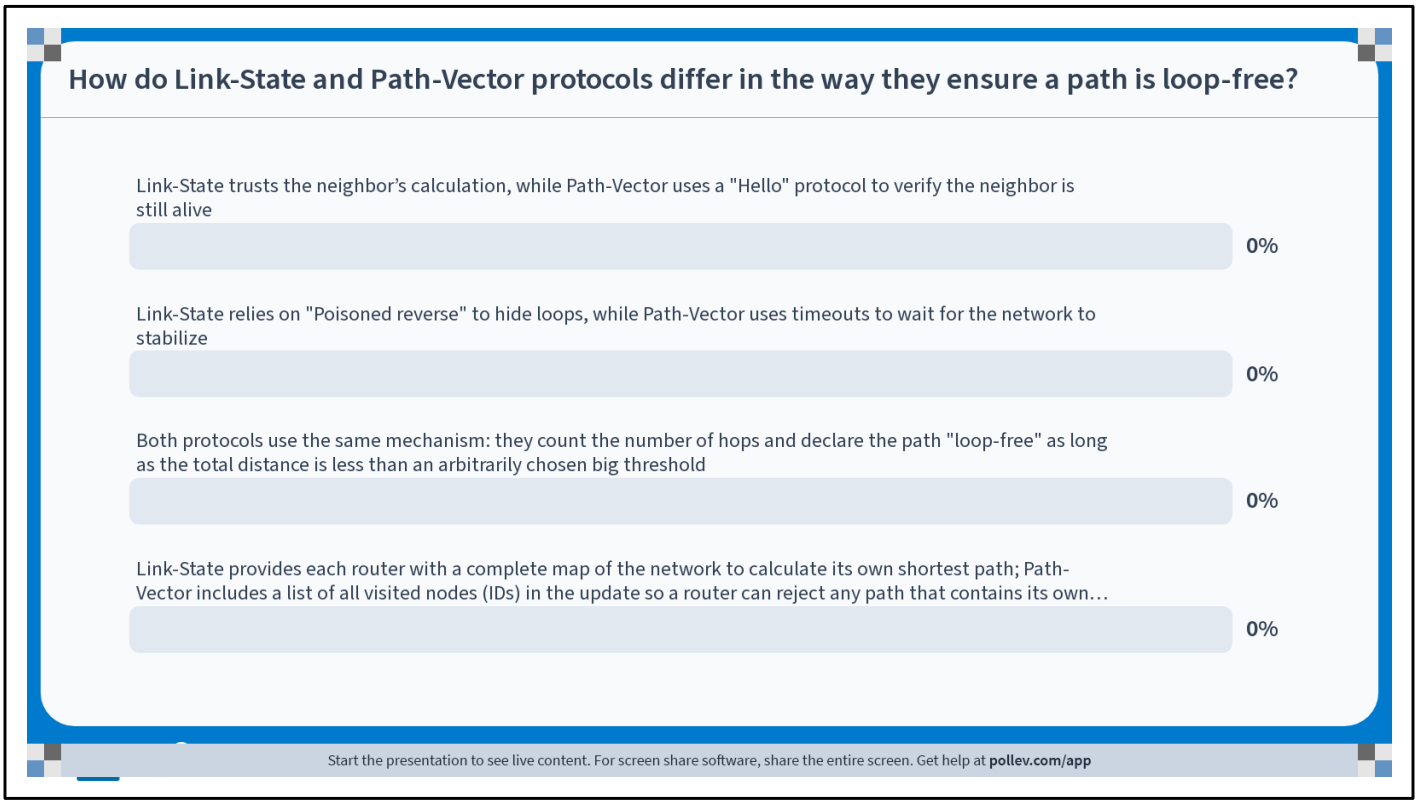
Path-vector routing

- **Key idea**: advertise the entire path to the destination node along with distance metric
- **Advantages:**
  - avoid count-to-infinity problem due to fast loop detection (node cannot be twice in the path)
  - support of flexible routing policies based on local preference of one path over another



One of the solutions to the problem of counting to infinity is to report not only the distance metric, but also the entire path to the node. This approach is called path-vector routing. With the knowledge of the entire path, the loop detection becomes very easy: the node cannot be indicated in the path twice.

Besides the loop detection, path-vector routing enables more flexibility in path selection. For example, the network manager can decide that a certain path is preferable than the other based on the knowledge of the path the packets will have to take to reach the destination.



Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity. More info at polleverywhere.com/support

How do Link-State and Path-Vector protocols differ in the way they ensure a path is loop-free?
https://www.polleverywhere.com/multiple_choice_polls/rHOCWGEBdlEbTl10z6CXQ?state=opened&flow=Default&onscreen=persist

Routing protocols summary

- In **Link-State protocols**, all routers have complete information about connectivity and link costs. Routers calculate shortest paths using Dijkstra's algorithm. The algorithm ensures that the chosen paths are loop-free.
- In **Distance-Vector protocols**, routers rely on the knowledge provided by their neighbors. Each router advertises the vector of current shortest distances to all reachable nodes in the network. Special mechanisms, such as Poisoned Reverse, are used to avoid loops. To overcome count-to-infinity, they need a threshold value.
- In **Path-Vector protocols**, a list of all visited nodes is advertised together with the distance vector. Routers reject the paths that already contain their own ID.

Routing on the Internet: OSPF & BGP

University of Antwerp
Faculty of Science

Sections 5.3 and 5.4

Routing on the Internet happens on two levels

- The Internet is split into **autonomous systems** (ASs)
  - Each AS consists of a group of routers that are under the same administrative control (e.g., owned by one ISP)
  - Each AS has a globally unique autonomous system number (ASN)
- Two different types of routing algorithms
  - Within an AS (**intra-AS**): Interior Gateway Protocol (IGP), such as Open Shortest Path First (OSPF)
  - Across ASs (**inter-AS**): Exterior Gateway Protocol (EGP), such as Border Gateway Protocol (BGP)
- Advantages of a two-stage approach
  - **Scalability**: The number of routers on the Internet is too big for a single routing algorithm to handle
  - **Administrative autonomy**: Each ISP wants to enforce its own policies on the routing algorithms it uses

University of Antwerp
Faculty of Science

In our study of routing algorithms so far, we've viewed the network simply as a collection of interconnected routers. One router was indistinguishable from another in the sense that all routers executed the same routing algorithm to compute routing paths through the entire network. In practice, this model and its view of a homogenous set of routers all executing the same routing algorithm is simplistic for two important reasons:

- ***Scale.*** As the number of routers becomes large, the overhead involved in communicating, computing, and storing routing information becomes prohibitive. Today's Internet consists of hundreds of millions of routers. Storing routing information for possible destinations at each of these routers would clearly require enormous amounts of memory. The overhead required to broadcast connectivity and link cost updates among all of the routers would be huge! A distance-vector algorithm that iterated among such a large number of routers would surely never converge. Clearly, something must be done to reduce the complexity of route computation in a network as large as the Internet.
- ***Administrative autonomy.*** The Internet is a network of ISPs, with each ISP consisting of its own network of routers. An ISP generally desires to operate its network as it pleases (for example, to run whatever routing algorithm it chooses within its network) or to hide aspects of its network's internal organization from the outside. Ideally, an organization should be able to operate and administer its network as it wishes, while still being able to connect its network to other outside networks.

Both of these problems can be solved by organizing routers into **autonomous systems (ASs)**, with each AS consisting of a group of routers that are under the same administrative control. Often the routers in an ISP, and the links that interconnect them, constitute a single AS. Some

ISPs, however, partition their network into multiple ASs. In particular, some tier-1 ISPs use one gigantic AS for their entire network, whereas others break up their ISP into tens of interconnected ASs. An autonomous system is identified by its globally unique autonomous system number (ASN) [RFC1930]. AS numbers, like IP addresses, are assigned by ICANN regional registries. Routers within the same AS all run the same routing algorithm and have information about each other. The routing algorithm running within an autonomous system is called an **intra-autonomous system routing protocol**.

Interior Gateway Protocols (IGPs)

- Open Shortest Path First (OSPF)
  - Based on link-state routing and Dijkstra's shortest path algorithm
  - Standardized as RFC 2328 (IPv4) and RFC 5340 (IPv6)
  - Widely used in large enterprise networks
- Intermediate System to Intermediate System (IS-IS)
  - Conceptually very similar to OSPF (also based on link-state routing and Dijkstra's shortest path algorithm)
  - Standardized as ISO/IEC 10589:2002
  - Widely used in large service provider networks
- Enhanced Interior Gateway Routing Protocol (EIGRP)
  - Based on distance-vector routing
  - Developed as a proprietary protocol by Cisco, but eventually standardized as RFC 7868 in 2016

OSPF routing and its closely related cousin, IS-IS, are widely used for intra-AS routing in the Internet. The Open in OSPF indicates that the routing protocol specification is publicly available (for example, as opposed to Cisco's EIGRP protocol, which was only recently became open, after roughly 20 years as a Cisco-proprietary protocol). The most recent version of OSPF, version 2, is defined in [RFC 2328], a public document.

Open Shortest Path First (OSPF)

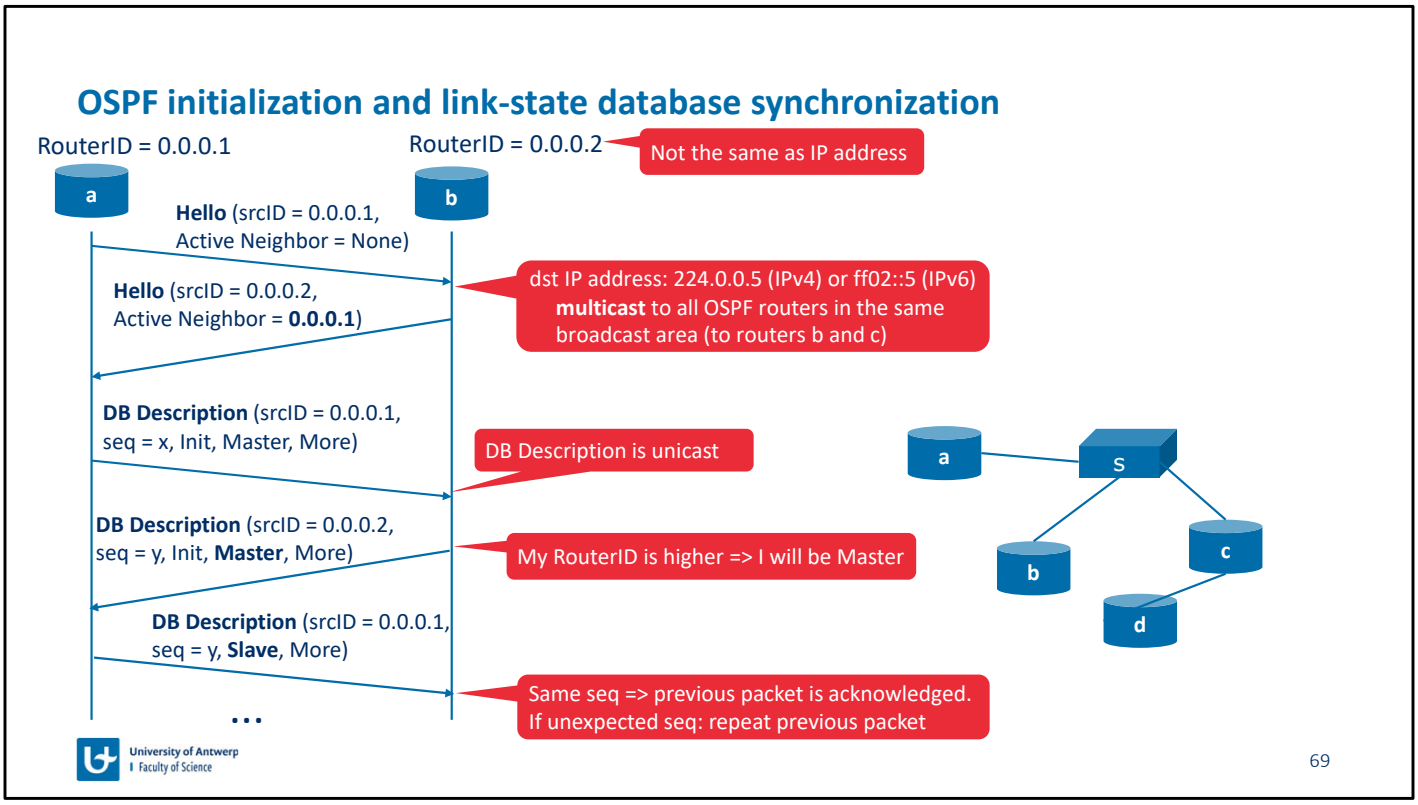
- Based on link-state routing and Dijkstra's shortest path algorithm
  - Routers **flood** link-state information [i.e., current knowledge of the entire AS] **to all other routers in the autonomous system**, not only direct neighbours
  - Link-state is advertised when a link changes (cost change, comes up, or goes down)
  - Link-state is also advertised periodically (at least once very 30 minutes)
  - Link availability is checked by sending periodic HELLO messages to all direct neighbours
- Works directly on top of IP, i.e., none of the transport layer protocols are used. OSPF should take care of reliable delivery of its messages on its own.
- Link weights are manually configured by the administrator
- Other features
  - Security: Exchanges between OSPF routers can be authenticated (when IPv6 is used, can rely on IP Authentication Header)
  - Multi-path: If multiple same-cost paths exist, they can be used simultaneously to each carry part of the traffic
  - Multicast support
  - Hierarchical division of AS into multiple independent routing areas

OSPF is a link-state protocol that uses flooding of link-state information and a Dijkstra's least-cost path algorithm. With OSPF, each router constructs a complete topological map (that is, a graph) of the entire autonomous system. Each router then locally runs Dijkstra's shortest-path algorithm to determine a shortest-path tree to all *subnets*, with itself as the root node. Individual link costs are configured by the network administrator. The administrator might choose to set all link costs to 1, thus achieving minimum-hop routing, or might choose to set the link weights to be inversely proportional to link capacity in order to discourage traffic from using low-bandwidth links. OSPF does not mandate a policy for how link weights are set (that is the job of the network administrator), but instead provides the mechanisms (protocol) for determining least-cost path routing for the given set of link weights.

With OSPF, a router **floods** link-state information to all other routers in the autonomous system, by passing it through its neighboring routers. A router transmits a link-state advertisement whenever there is a change in a link's state (for example, a change in cost or a change in up/down status). It also transmits link-state advertisement periodically (at least once every 30 minutes), even if the link's state has not changed. RFC 2328 notes that "this periodic updating of link state advertisements adds robustness to the link state algorithm." OSPF advertisements are contained in OSPF messages that are carried directly by IP, with an upper-layer protocol of 89 for OSPF. Thus, the OSPF protocol must itself implement functionality such as reliable message transfer and link-state broadcast. The OSPF protocol also checks that links are operational (via a HELLO message that is sent to an attached neighbor) and allows an OSPF router to obtain a neighboring router's database of network-wide link state.

Some of the advances embodied in OSPF include the following:

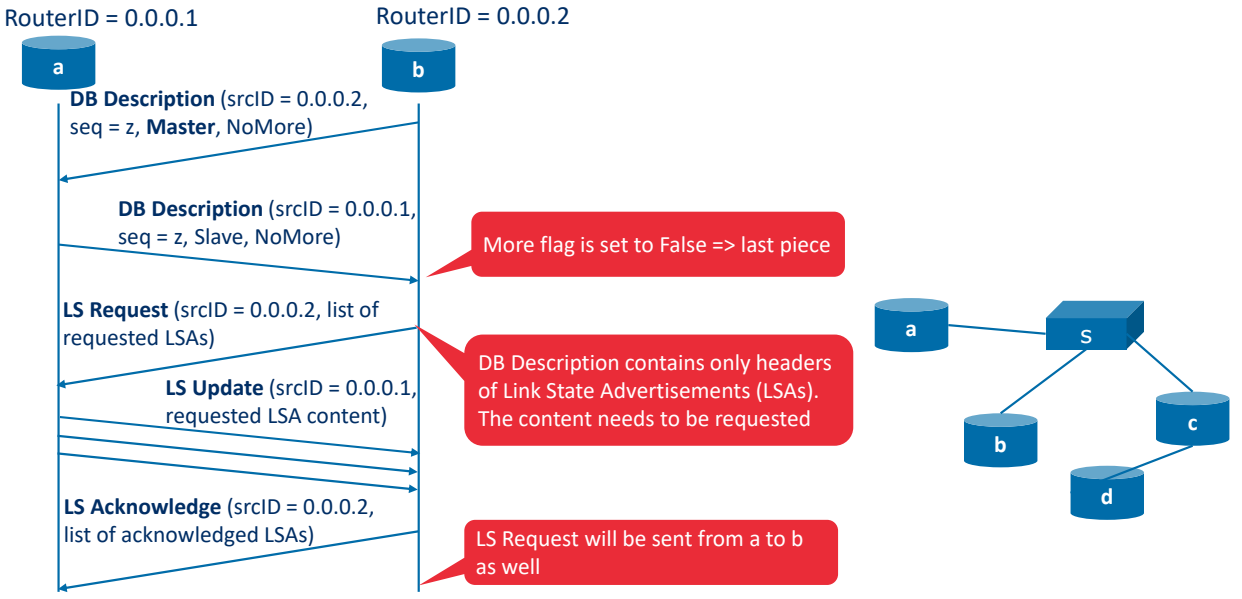
- *Security.* Exchanges between OSPF routers (for example, link-state updates) can be authenticated. With authentication, only trusted routers can participate in the OSPF protocol within an AS, thus preventing malicious intruders (or networking students taking their newfound knowledge out for a joyride) from injecting incorrect information into router tables. By default, OSPF packets between routers are not authenticated and could be forged. Two types of authentication can be configured—simple and MD5 (see Chapter 8 for a discussion on MD5 and authentication in general). With simple authentication, the same password is configured on each router. When a router sends an OSPF packet, it includes the password in plaintext. Clearly, simple authentication is not very secure. MD5 authentication is based on shared secret keys that are configured in all the routers. For each OSPF packet that it sends, the router computes the MD5 hash of the content of the OSPF packet appended with the secret key. (See the discussion of message authentication codes in Chapter 8.) Then the router includes the resulting hash value in the OSPF packet. The receiving router, using the preconfigured secret key, will compute an MD5 hash of the packet and compare it with the hash value that the packet carries, thus verifying the packet's authenticity. Sequence numbers are also used with MD5 authentication to protect against replay attacks.
- *Multiple same-cost paths.* When multiple paths to a destination have the same cost, OSPF allows multiple paths to be used (that is, a single path need not be chosen for carrying all traffic when multiple equal-cost paths exist).
- *Integrated support for unicast and multicast routing.* Multicast OSPF (MOSPF) [RFC 1584] provides simple extensions to OSPF to provide for multicast routing. MOSPF uses the existing OSPF link database and adds a new type of link-state advertisement to the existing OSPF link-state broadcast mechanism.
- *Support for hierarchy within a single AS.* An OSPF autonomous system can be configured hierarchically into areas. Each area runs its own OSPF link-state routing algorithm, with each router in an area broadcasting its link state to all other routers in that area. Within each area, one or more area border routers are responsible for routing packets outside the area. Lastly, exactly one OSPF area in the AS is configured to be the backbone area. The primary role of the backbone area is to route traffic between the other areas in the AS. The backbone always contains all area border routers in the AS and may contain non-border routers as well. Inter-area routing within the AS requires that the packet be first routed to an area border router (intra-area routing), then routed through the backbone to the area border router that is in the destination area, and then routed to the final destination.



In OSPF, the **Hello Protocol** facilitates neighbor discovery via multicast (to 224.0.0.5 or ff02::5). In a **broadcast area,** a logical segment where such a multicast packet is automatically delivered to every connected OSPF router, OSPF elects a **Designated Router (DR)** to manage the exchange and prevent redundant traffic between every possible pair. A bidirectional "2-Way" state is reached only when a router sees its own 32-bit **RouterID** (not necessarily the same as its IP address) in a neighbor's packet. To sync data, routers negotiate a Master/Slave relationship; the router with the higher RouterID becomes the Master to coordinate the exchange. This relationship is indicated within the DBD packet header using the MS (Master/Slave) bit: the Master sets this bit to 1, while the Slave sets it to 0. If a router receives a DBD from a neighbor with a higher Router ID, it immediately yields its "Master" status and becomes the Slave, allowing the higher-ID router to take over and drive the exchange.

Once established, routers exchange **DataBase Description (DBD)** packets as a "Table of Contents" for their Link-State Databases. DBD packets are sent via unicast (i.e., directly to the neighbor) because each router already learned its neighbor's IP address from the initial Hello packets. The Master initiates the exchange by sending a packet with a sequence number, which the Slave must then use in its own response to acknowledge receipt. To ensure reliability, OSPF performs its own **error recovery**: if a router receives an unexpected sequence number (indicating a lost or out-of-order packet), it does not acknowledge the new data. The missed packet is repeated.

OSPF initialization and link-state database synchronization



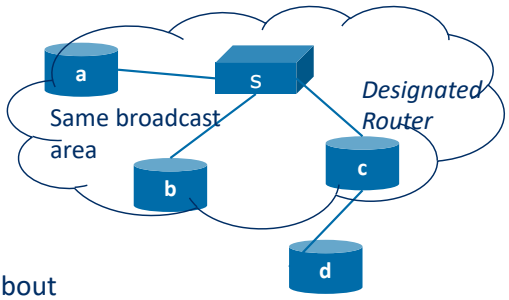
The synchronization process concludes with the exchange of **Database Description (DBD)** packets until both routers set the **"More" flag** (M-bit) to 0, indicating the entire "Table of Contents" has been shared. Because these initial summaries contain only LSA headers rather than full topology data, routers must identify which specific entries are missing or more recent than their own. For that, a router sends **Link State Request (LSR)** to its neighbor, explicitly asking for the full details of the specific link-state advertisements.

In response to a request, the neighbor transmits a **Link State Update (LSU)**, which contains the full content and attributes of the requested Link State Advertisements. These updates are the most critical packets in OSPF, as they provide the actual data used to build the network map. In case the topology changed (link went up/down, cost changed), LSU packet will be sent, and the information about the change will be flooded throughout the AS. To control the flood within the broadcast area, routers send updates to the designated router, which in its turn shares the update with all routers in the broadcast area. To guarantee reliability, the receiving router must return a **Link State Acknowledgment (LSAck)**. After the routers get the full information about the network topology, they independently run the Dijkstra algorithm to determine the best paths through the network.

OSPF Link-State Advertisements (LSAs)

- **Router LSA** contains information about interfaces of the router. For each interface, it provides a RouterID of the neighboring router connected through this interface and link cost (metric).
  In *OSPFv2*, also provides the IP address of the Designated Router in the broadcast area.

- **Network LSA** is generated by designated router, and contains information about all attached routers in this broadcast area. Designated router is responsible for other routers in this area.
  In *OSPFv2,* also provides the network mask.

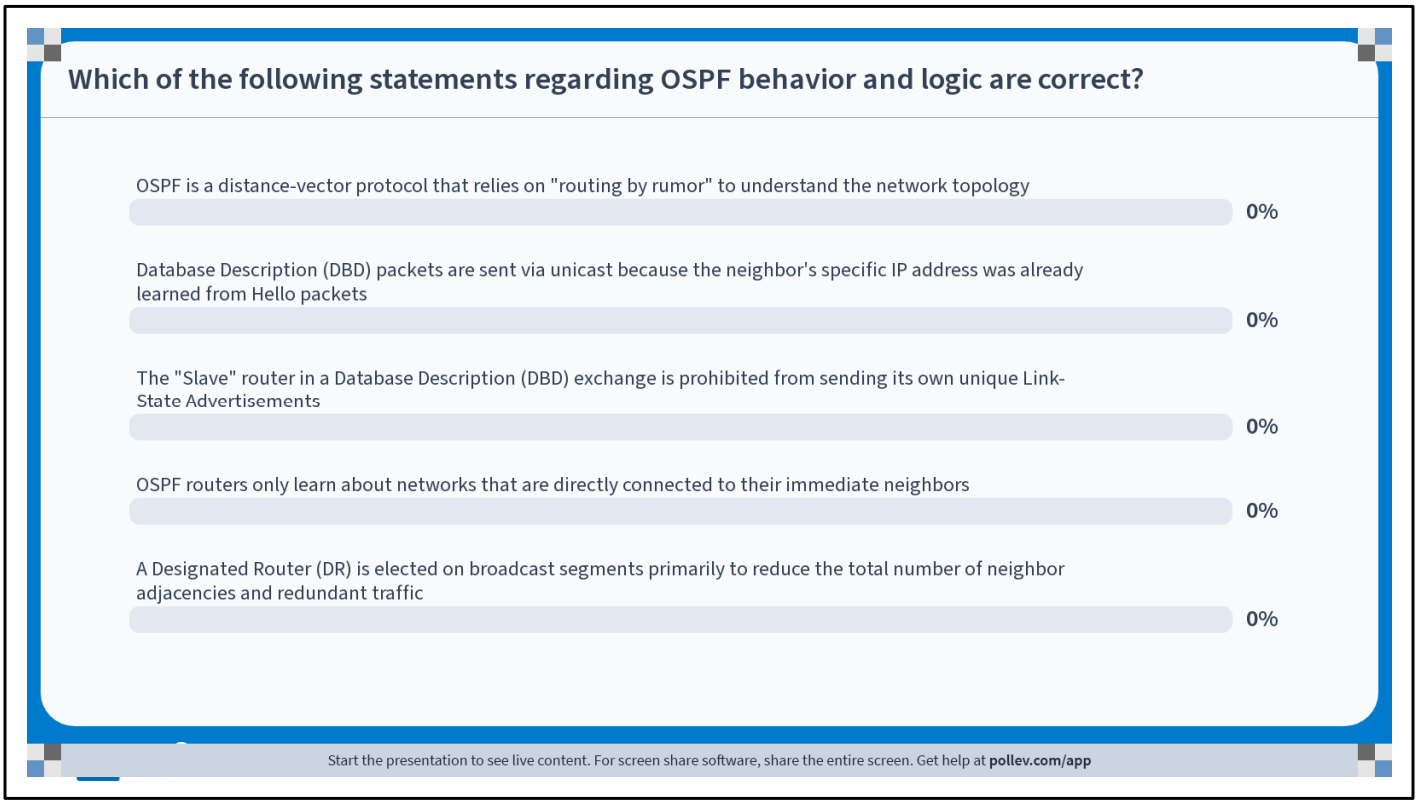


- **Intra-area prefix LSA** (*OSPFv3* only) contains information about network IP addresses (prefixes) reachable through a router.

In OSPF, the "map" of the network is built using different types of **Link-State Advertisements (LSAs)**. The **Router LSA (Type 1)** lists each active interface on a router, the cost to use the link attached to the interface, and the Router IDs of neighbors found on those links. In OSPFv2, this LSA also includes the IP address of the **Designated Router (DR)** if the interface is part of a broadcast segment. By collecting these Type 1 LSAs from every router, the network can map out exactly which routers are physically connected to one another.

To reduce the number of generated advertisements, the **Network LSA (Type 2)** is generated exclusively by the Designated Router, which represents the entire segment as a single "node." This LSA lists all routers currently attached to that specific broadcast area and, in OSPFv2, includes the network mask.

OSPFv3 (OSPF with IPv6) introduces the **Intra-area prefix LSA (Type 9)**. Unlike the earlier versions where addressing was bundled into the topology LSAs, OSPFv3 separates them; Type 1 and 2 LSAs now only describe the physical "skeleton" of the network, while the Type 9 LSA carries the actual IPv6 prefixes reachable through those routers.

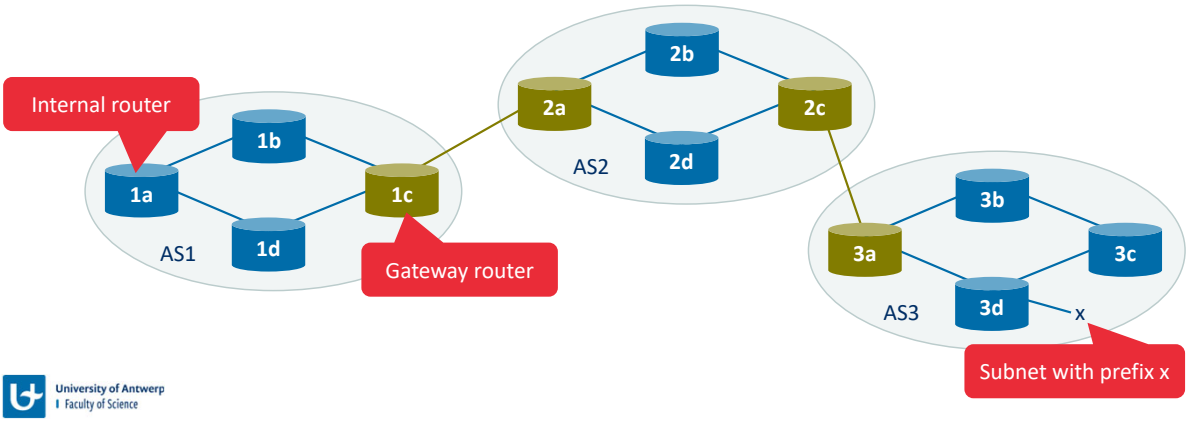


Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity. More info at polleverywhere.com/support

Which of the following statements regarding OSPF behavior and logic are correct? https://www.polleverywhere.com/multiple_choice_polls/ivL7ItUPCpHJfTfzmMdBr?state=opened&flow=Default&onscreen=persist

Border Gateway Protocol (BGP)

- BGP is used for routing between all ASs on the Internet
- Decentralized and asynchronous, based on **path-vector** routing
- Relies on TCP for delivery of its messages
- Routing is done based on CIDR prefixes (e.g., 138.16.68/22). Summary prefixes can be advertised instead of many specific prefixes from the AS (e.g., one /22 prefix instead of multiple /24 prefixes)



University of Antwerp
Faculty of Science

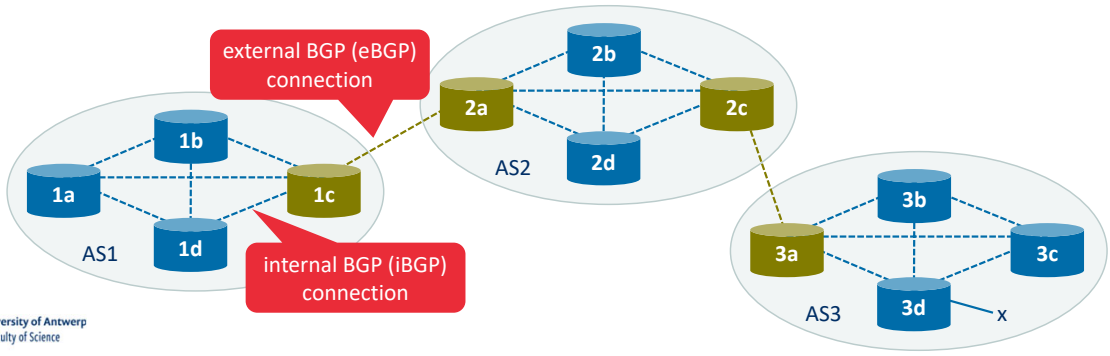
73

To route a packet across multiple ASs, say from a smartphone in Antwerp to a server in a datacenter of Google, we need an **inter-autonomous system routing protocol**. You can check the AS number by the IP address using online BGP looking glasses, e.g., https://bgp.he.net/. Since an inter-AS routing protocol involves coordination among multiple ASs, communicating ASs must run the same inter-AS routing protocol. In fact, in the Internet, all ASs run the same inter-AS routing protocol, called the Border Gateway Protocol, more commonly known as **BGP** [RFC 4271]. BGP is a decentralized and asynchronous protocol in the vein of distance-vector routing. More specifically, BGP can be considered as a path-vector routing protocol.

The figure shows a simple network with three autonomous systems: AS1, AS2, and AS3. As shown, AS3 includes a subnet with prefix x. For each AS, each router is either a **gateway router** or an **internal router**. A gateway router is a router on the edge of an AS that directly connects to one or more routers in other ASs. An **internal router** connects only to hosts and routers within its own AS. In AS1, for example, router 1c is a gateway router; routers 1a, 1b, and 1d are internal routers.

BGP connections

- How is information about subnet x in AS3 advertised to the Internet?
- Routers maintain a semi-permanent TCP connection on port 179 to exchange BGP information
  - **iBGP connection**: TCP connection between two routers within the same AS
  - **eBGP connection**: TCP connection between two routers in different ASs
- Within an AS, all routers maintain an iBGP connection to each other (in a full mesh or using Route Reflectors), while eBGP connections are only maintained between direct neighbours in different ASs.

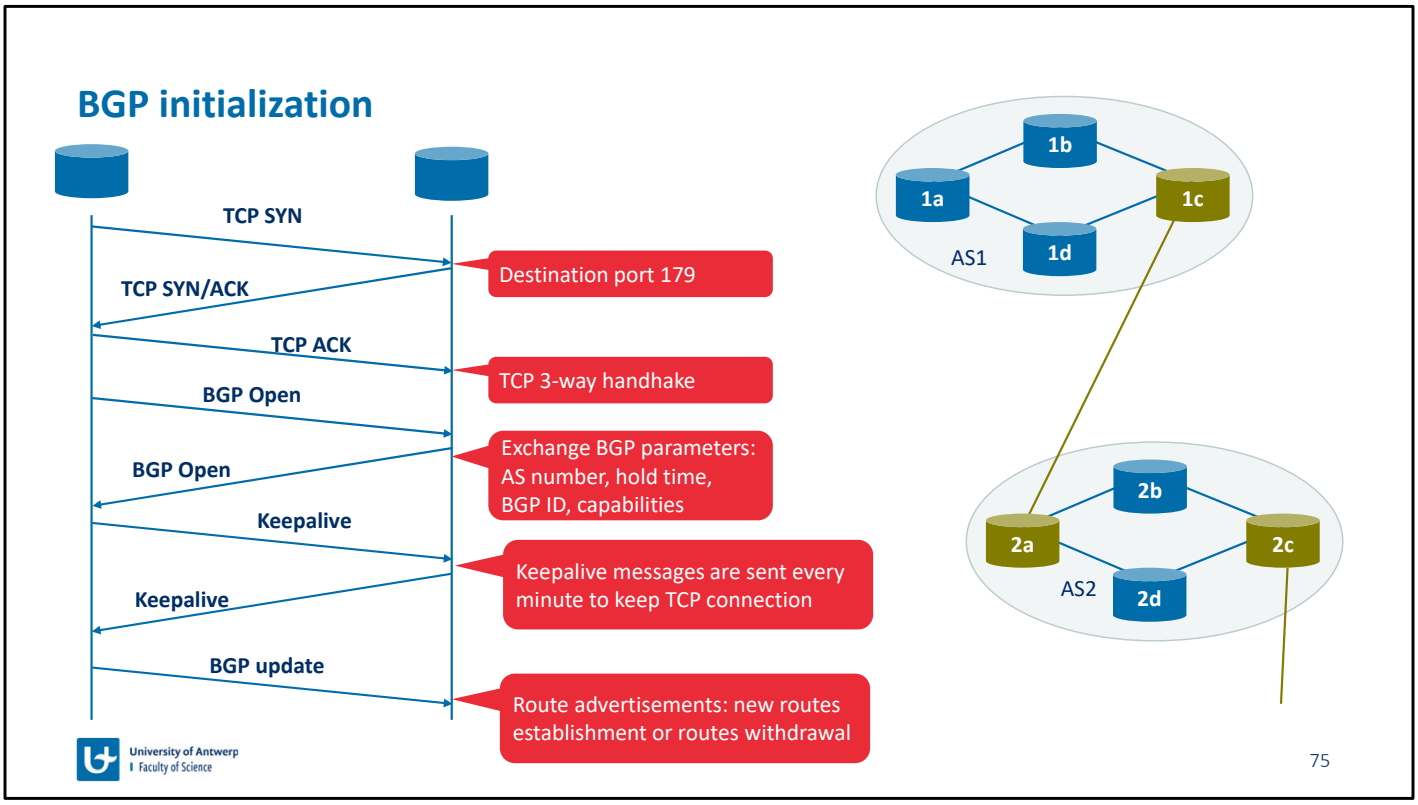


University of Antwerp
Faculty of Science

74

In BGP, pairs of routers exchange routing information over semi-permanent TCP connections (maintained by periodic transmission of keepalive messages) using port 179. Each such TCP connection, along with all the BGP messages sent over the connection, is called a **BGP connection**. Furthermore, a BGP connection that spans two ASs is called an **external BGP (eBGP)** connection, and a BGP session between routers in the same AS is called an **internal BGP (iBGP)** connection.

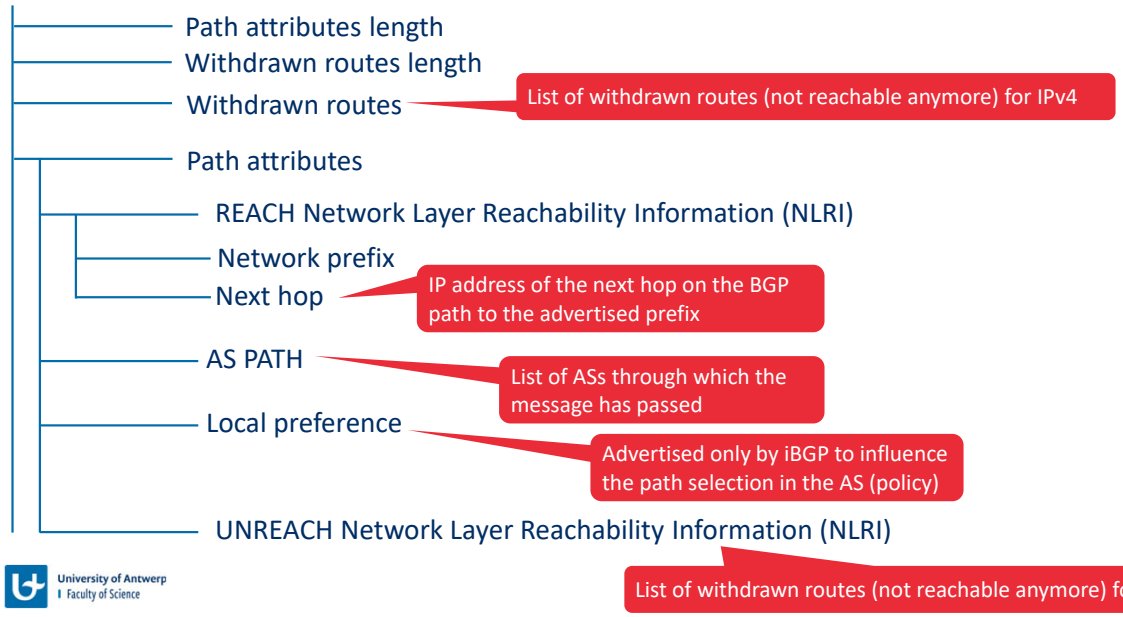
There is typically one eBGP connection for each link that directly connects gateway routers in different ASs; thus, there is an eBGP connection between gateway routers 1c and 2a and an eBGP connection between gateway routers 2c and 3a. There are also iBGP connections between routers within each of the ASs. In particular, a common configuration for small-scale networks is to create one BGP connection for each pair of routers internal to an AS, creating a mesh of TCP connections within each AS. In larger networks, Route Reflectors are designated to redistribute information received from all peers, while all the routers establish TCP connections only with Route Reflectors. Note that these iBGP connections do not always correspond to direct physical links.



BGP follows multi-step initialization process: first, the underlying network must complete a **TCP 3-way handshake**, providing the reliable connection for data exchange. Once connected, routers exchange **BGP Open** messages to negotiate critical session parameters, such as the **AS Number**, **Hold Time** (how long to wait before declaring a neighbor dead), and the **BGP Identifier** (similar to an OSPF Router ID).

After the session parameters are agreed upon, routers enter a steady state maintained by periodic **Keepalive messages**. These small packets are sent every minute (by default) to confirm that the neighbor is still active and the TCP connection is healthy, preventing the session from timing out. Using the established connection, routers transmit **BGP Updates** containing the actual path information. These updates are incremental: routers only advertise new network prefixes or withdrawals of failed routes.

BGP Update messages

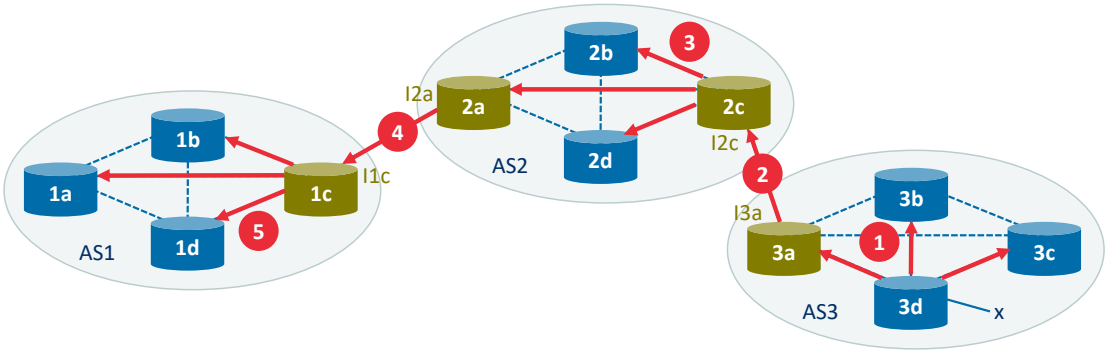


BGP uses BGP Update messages to describe the "path" to a destination. The message is split into two main functions: telling neighbors about routes that no longer exist (**Withdrawn Routes or UNREACH NLRI**) and advertising new ones (**REACH NLRI**). BGP uses a Path Vector approach to determine the best route, where the primary metric is the **AS_PATH length.** The AS_PATH attribute records every Autonomous System number the update has passed through; BGP prefers the route with the fewest number of AS hops. Next hop attribute points to an IP address

Local Preference allows for a granular policy control by setting a preference to certain paths, even if their AS_PATH is longer.

BGP advertisement messages

- BGP advertisements (also called routes) consist of an address prefix and several attributes
  - AS-PATH: List of ASs through which the message has passed
  - NEXT-HOP: IP address of the router interface that begins the AS-PATH
- **Example (advertising subnet x):**



University of Antwerp
Faculty of Science

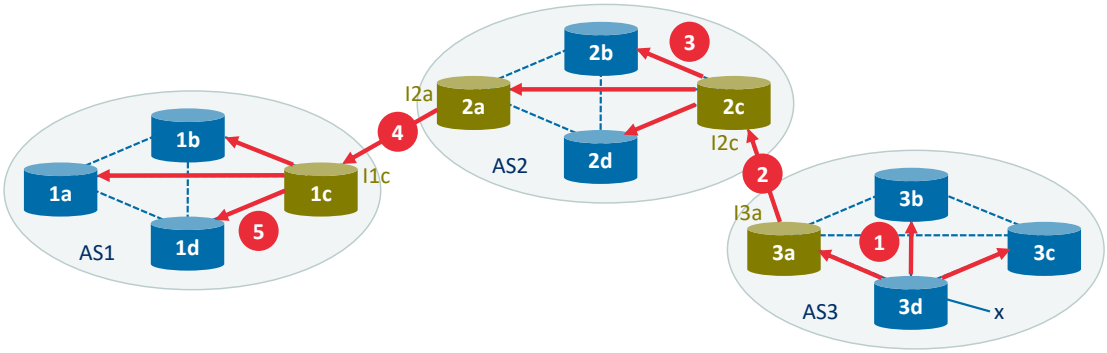
77

In order to propagate the reachability information, both iBGP and eBGP sessions are used. Consider again advertising the reachability information for prefix x to all routers in AS1 and AS2. In this process, gateway router 3a first sends an eBGP message "AS3 x" to gateway router 2c. Gateway router 2c then sends the iBGP message "AS3 x" to all of the other routers in AS2, including to gateway router 2a. Gateway router 2a then sends the eBGP message "AS2 AS3 x" to gateway router 1c. Finally, gateway router 1c uses iBGP to send the message "AS2 AS3 x" to all the routers in AS1. After this process is complete, each router in AS1 and AS2 is aware of the existence of x and is also aware of an AS path that leads to x.

When a router advertises a prefix across a BGP connection, it includes with the prefix several **BGP attributes**. In BGP jargon, a prefix along with its attributes is called a **route**. Two of the more important attributes are AS-PATH and NEXT-HOP. The AS-PATH attribute contains the list of ASs through which the advertisement has passed. To generate the ASPATH value, when a prefix is passed to an AS, the AS adds its ASN to the existing list in the AS-PATH. BGP routers also use the AS-PATH attribute to detect and prevent looping advertisements; specifically, if a router sees that its own AS is contained in the path list, it will reject the advertisement. Providing the critical link between the inter-AS and intra-AS routing protocols, the NEXT-HOP attribute has a subtle but important use. The NEXT-HOP is the *IP address of the router interface that begins the AS-PATH*.

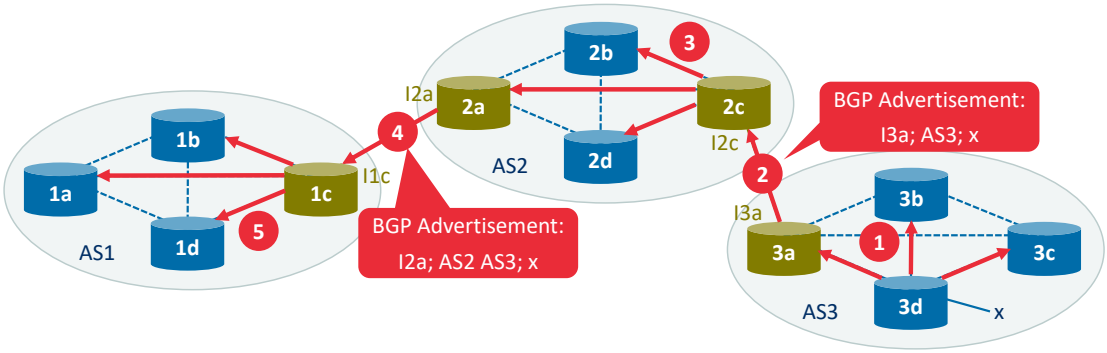
Exercise: BGP advertisement messages

Exercise: What is the content of the BGP advertisements sent at steps 2 and 4, assuming a structure "NEXT-HOP; AS-PATH; prefix"?



Solution: BGP advertisement messages

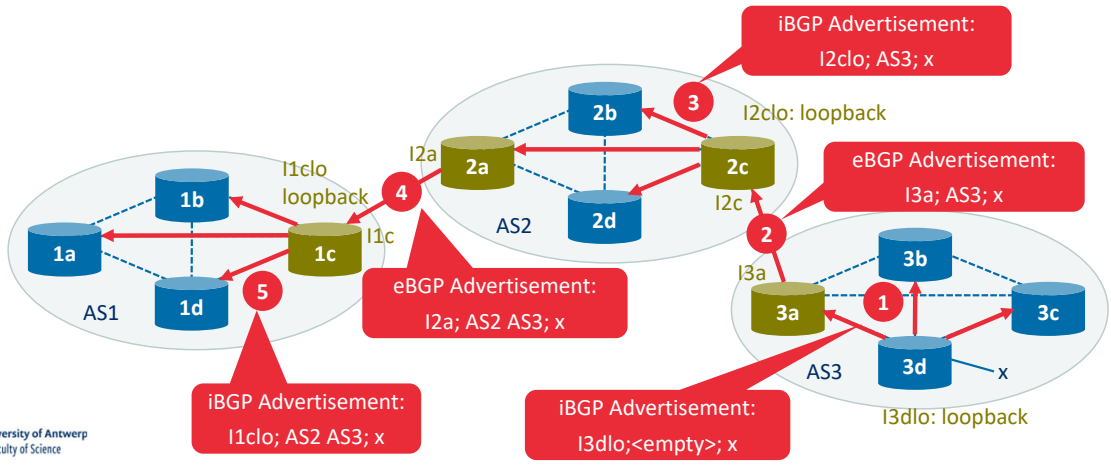
Exercise: What is the content of the BGP advertisements sent at steps 2 and 4, assuming a structure "NEXT-HOP; AS-PATH; prefix"?



BGP advertisement messages: iBGP next hop self

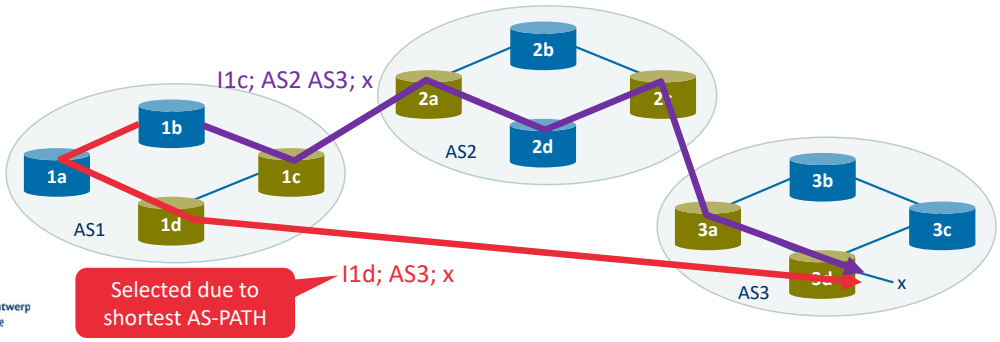
IP address of interface I3a is unknown inside AS2. To be able to route to next hop inside AS2, iBGP substitutes IP address of the next hop with internal IP address.

A good practice is to use loopback interface.



BGP route-selection algorithm

- BGP sequentially invokes elimination rules until only 1 route remains
  1. If any routes have the **local preference value** set, the route with the highest local preference is selected
  2. From the remaining routes, the one with the **shortest AS-PATH** is selected
  3. From the remaining routes, the one with the **closest NEXT-HOP** router is selected (determined using intra-AS routing, e.g., OSPF)
  4. If more than 1 route remains, a route is selected based on the BGP identifier
- **Example**: Which of the 2 routes would be selected?



University of Antwerp | Faculty of Science

81

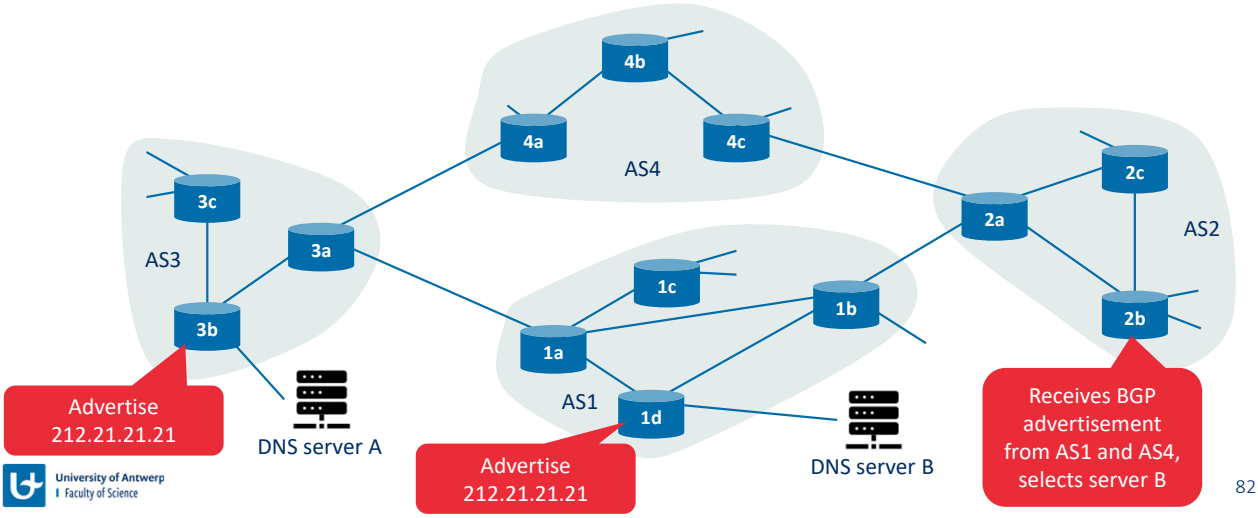
For any given destination prefix, the input into BGP's route-selection algorithm is the set of all routes to that prefix that have been learned and accepted by the router. If there is only one such route, then BGP obviously selects that route. If there are two or more routes to the same prefix, then BGP sequentially invokes the following elimination rules until one route remains (we list here 4 rules only, but in total BGP applies 10 rules sequentially):

1. A route is assigned a **local preference** value as one of its attributes (in addition to the AS-PATH and NEXT-HOP attributes). The local preference of a route could have been set by the router or could have been learned from another router in the same AS. The value of the local preference attribute is a policy decision that is left entirely up to the AS's network administrator. (We will shortly discuss BGP policy issues in some detail.) The routes with the highest local preference values are selected.
2. From the remaining routes (all with the same highest local preference value), the route with the shortest AS-PATH is selected. If this rule were the only rule for route selection, then BGP would be using a DV algorithm for path determination, where the distance metric uses the number of AS hops rather than the number of router hops.
3. From the remaining routes (all with the same highest local preference value and the same AS-PATH length), the route with the closest NEXT-HOP router is selected.
4. If more than one route still remains, the router uses BGP identifiers to select the route;

As an example, let's consider router 1b. There are exactly two BGP routes to prefix x, one that passes through AS2 and one that bypasses AS2. In the above route-selection algorithm, rule 2 is applied before rule 3, causing BGP to select the route that bypasses AS2, since that route has a shorter AS PATH. So we see that with the above route-selection algorithm, BGP is no longer a selfish algorithm—it first looks for routes with short AS paths (thereby likely reducing end-to-end delay).

IP-Anycast

- BGP can be used to advertise the same IP address for multiple servers, allowing routers to automatically select the nearest one (e.g., DNS server replication)

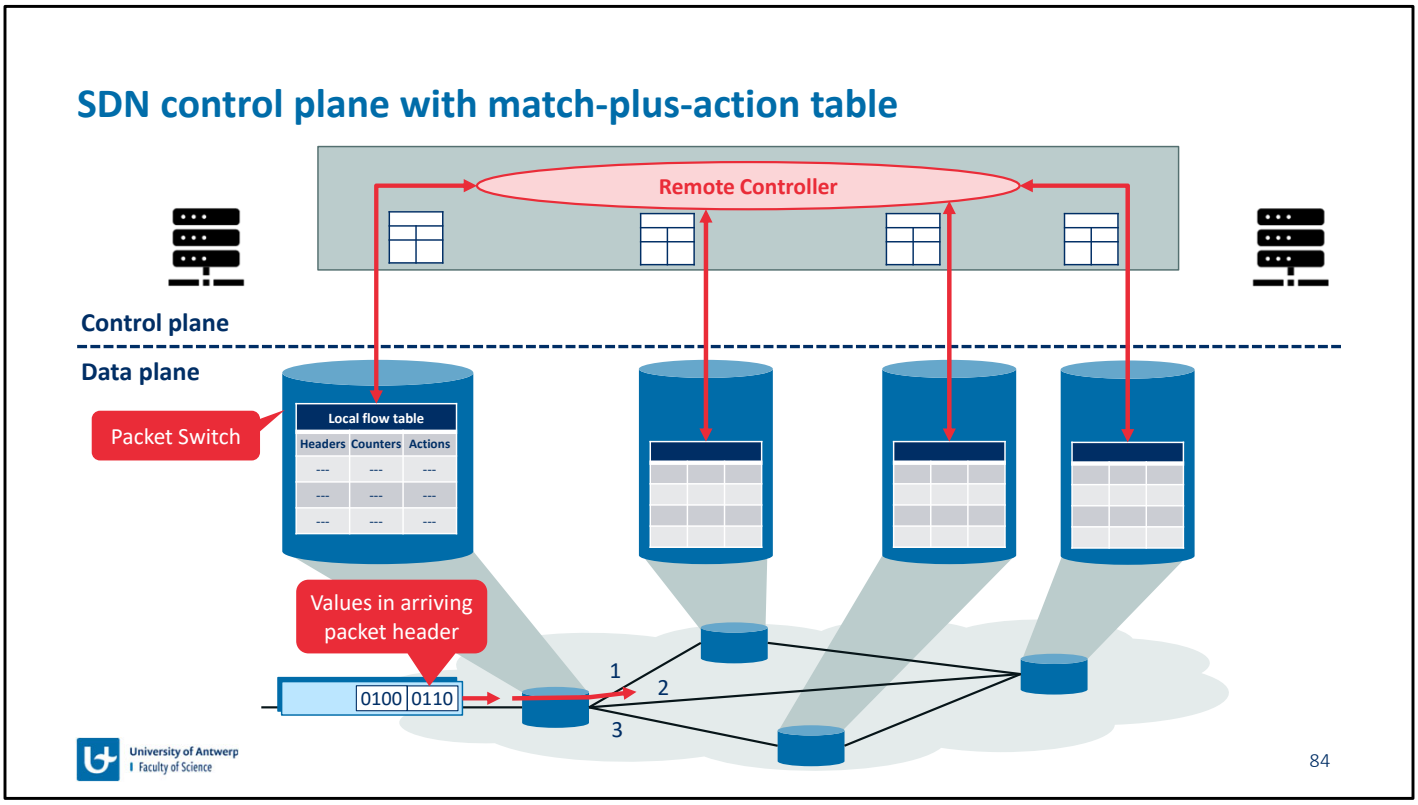


In addition to being the Internet's inter-AS routing protocol, BGP is often used to implement the IP-anycast service [RFC 1546, RFC 7094], which is commonly used in DNS. To motivate IP-anycast, consider that in many applications, we are interested in (1) replicating the same content on different servers in many different dispersed geographical locations, and (2) having each user access the content from the server that is closest. BGP's route-selection algorithm provides an easy and natural mechanism for doing this.

To make our discussion concrete, let's describe how a DNS might use IP-anycast. During the IP-anycast configuration stage, the DNS operator assigns the *same* IP address to each of its servers, and uses standard BGP to advertise this IP address from each of the servers. When a BGP router receives multiple route advertisements for this IP address, it treats these advertisements as providing different paths to the same physical location (when, in fact, the advertisements are for different paths to different physical locations). When configuring its routing table, each router will locally use the BGP route-selection algorithm to pick the "best" (for example, closest, as determined by AS-hop counts) route to that IP address. IP-anycast is extensively used by the DNS system to direct DNS queries to the closest root DNS server.

Software defined networking (SDN)

Sections 4.4 and 5.5



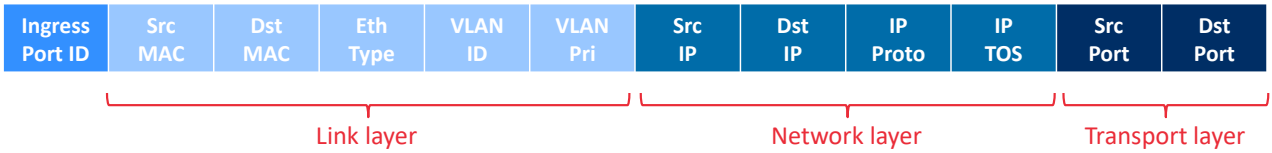
Recall destination-based forwarding as the two steps of looking up a destination IP address ("match"), then sending the packet into the switching fabric to the specified output port ("action"). Let's now consider a significantly more general "match-plus-action" paradigm, where the "match" can be made over multiple header fields associated with different protocols at different layers in the protocol stack. The "action" can include forwarding the packet to one or more output ports (as in destination-based forwarding), load balancing packets across multiple outgoing interfaces that lead to a service (as in load balancing), rewriting header values (as in NAT), purposefully blocking/dropping a packet (as in a firewall), sending a packet to a special server for further processing and action (as in deep packet inspection), and more.

In generalized forwarding, a match-plus-action table generalizes the notion of the destination-based forwarding table. Because forwarding decisions may be made using network-layer and/or link-layer source and destination addresses, the forwarding devices are more accurately described as "packet switches" rather than layer 3 "routers" or layer 2 "switches." Thus, we'll refer to these devices as packet switches, adopting the terminology that is gaining widespread adoption in SDN literature.

The figure shows a match-plus-action table in each packet switch, with the table being computed, installed, and updated by a remote controller. We note that while it is possible for the control components at the individual packet switches to interact with each other, in practice, generalized match-plus-action capabilities are implemented via a remote controller that computes, installs, and updates these tables.

Generalized forwarding in OpenFlow (the most used SDN protocol)

- **Flow table** consists of 3 columns:
  - Header field values to match (including link layer, network layer and transport layer fields)
  - Counters keeping track of number of matches and time table entry was last updated
  - Actions to be taken (forwarding, load balancing, rewriting header fields, dropping packets, etc.)
- Matching is based on 11 packet-header fields and the incoming port ID



- Header values can have wildcards (e.g., 128.119.*.* matches all IP addresses starting with 128.119)
- Each flow table entry has a priority, used when multiple entries match with a single packet header

University of Antwerp | Faculty of Science

85

Our following discussion of generalized forwarding will be based on OpenFlow—a highly visible standard that has pioneered the notion of the match-plus-action forwarding abstraction and controllers, as well as the SDN revolution more generally. Each entry in the match-plus-action forwarding table, known as a **flow table** in OpenFlow, includes:

- *A set of **header field values*** to which an incoming packet will be matched. A packet that matches no flow table entry can be dropped or sent to the remote controller for more processing. In practice, a flow table may be implemented by multiple flow tables for performance or cost
- reasons, but we'll focus here on the abstraction of a single flow table.
- *A **set of counters*** that are updated as packets are matched to flow table entries. These counters might include the number of packets that have been matched by that table entry, and the time since the table entry was last updated.
- *A **set of actions to be taken*** when a packet matches a flow table entry. These actions might be to forward the packet to a given output port, to drop the packet, makes copies of the packet and sent them to multiple output ports, and/or to rrewrite selected header fields.

11 packet-header fields and the incoming port ID that can be matched in an OpenFlow 1.0 match-plus-action rule. The first observation we make is that OpenFlow's match abstraction allows for a match to be made on selected fields from *three* layers of protocol headers (thus rather brazenly defying the layering principle). The ingress port refers to the input port at the packet switch on which a packet is received. Since we've not yet covered the link layer, suffice it to say that the source and destination MAC addresses shown are the link-layer addresses associated with the frame's sending and receiving interfaces; by forwarding on the

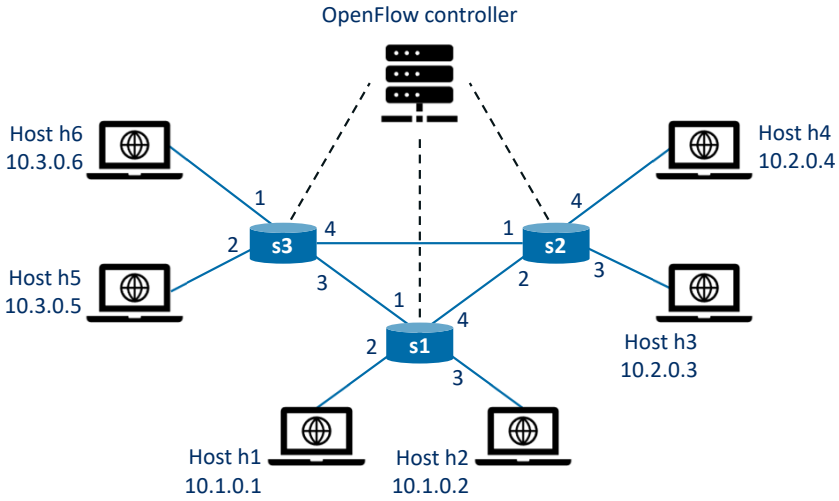
basis of Ethernet addresses rather than IP addresses, we can see that an OpenFlow-enabled device can equally perform as a router (layer-3 device) forwarding datagrams as well as a switch (layer-2 device) forwarding frames. The Ethernet type field corresponds to the upper layer protocol (e.g., IP) to which the frame's payload will be de-multiplexed, and the VLAN fields are concerned with so-called virtual local area networks.

Flow table entries may also have wildcards. For example, an IP address of 128.119.*.* in a flow table will match the corresponding address field of any datagram that has 128.119 as the first 16 bits of its address. Each flow table entry also has an associated priority. If a packet matches multiple flow table entries, the selected match and corresponding action will be that of the highest priority entry with which the packet matches.

Each flow table entry has a list of zero or more actions that determine the processing that is to be applied to a packet that matches a flow table entry. If there are multiple actions, they are performed in the order specified in the list. Among the most important possible actions are:

- ***Forwarding.*** An incoming packet may be forwarded to a particular physical output port, broadcast over all ports (except the port on which it arrived) or multicast over a selected set of ports. The packet may be encapsulated and sent to the remote controller for this device. That controller then may (or may not) take some action on that packet, including installing new flow table entries, and may return the packet to the device for forwarding under the updated set of flow table rules.
- ***Dropping.*** A flow table entry with no action indicates that a matched packet should be dropped.
- ***Modify-field.*** The values in 10 packet-header fields (all layer 2, 3, and 4 fields except the IP Protocol field) may be re-written before the packet is forwarded to the chosen output port.

OpenFlow example network topology

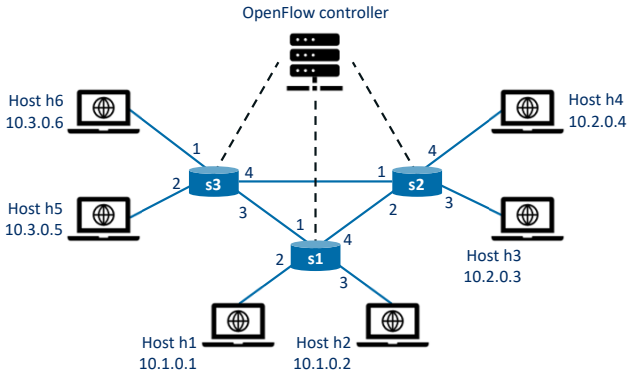


University of Antwerp
Faculty of Science

Having now considered both the match and action components of generalized forwarding, let's put these ideas together in the context of the sample network shown in the figure. The network has 6 hosts (h1, h2, h3, h4, h5 and h6) and three packet switches (s1, s2 and s3), each with four local interfaces (numbered 1 through 4). We'll consider a number of network-wide behaviours that we'd like to implement, and the flow table entries in s1, s2 and s3 needed to implement this behaviour.

Example 1: Simple forwarding

- Datagrams from h5 or h6 towards h3 or h4 should be forwarded via s1



s1 flow table	
Match	**Action**
Ingress Port 1; IP Src = 10.3.0.*; IP Dst = 10.2.0.*	Forward(4)
...	...

s3 flow table	
Match	**Action**
IP Src = 10.3.0.*; IP Dst = 10.2.0.*	Forward(3)
...	...

s2 flow table	
Match	**Action**
Ingress port = 2; IP Dst = 10.2.0.3	Forward(3)
Ingress port = 2; IP Dst = 10.2.0.4	Forward(4)
...	...

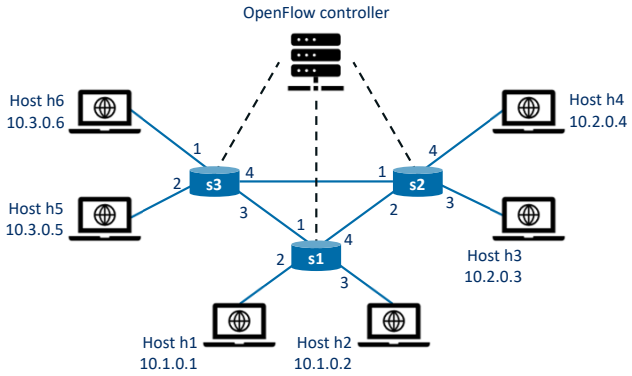
University of Antwerp
Faculty of Science

87

As a very simple example, suppose that the desired forwarding behaviour is that packets from h5 or h6 destined to h3 or h4 are to be forwarded from s3 to s1, and then from s1 to s2 (thus completely avoiding the use of the link between s3 and s2). A flow table entry needs to be added to s1 to forward traffic coming from 10.3.0.* towards 10.2.0.* via interface 4. Of course, we'll also need a flow table entry in s3 so that datagrams sent from h5 or h6 are forwarded to s1 over outgoing interface 3. Lastly, we'll also need a flow table entry in s2 to complete this first example, so that datagrams arriving from s1 are forwarded to their destination, either host h3 or h4.

Example 2: Load balancing

- Datagrams from h3 to 10.1.0.* should pass to s1
- Datagrams from h4 to 10.1.0.* should pass to s3



s2 flow table	
Match	**Action**
Ingress Port 3; IP Dst = 10.1.0.*	Forward(2)
Ingress Port 4; IP Dst = 10.1.0.*	Forward(1)
...	...

Homework: Write out the flow table entries for s1 and s3

University of Antwerp
Faculty of Science

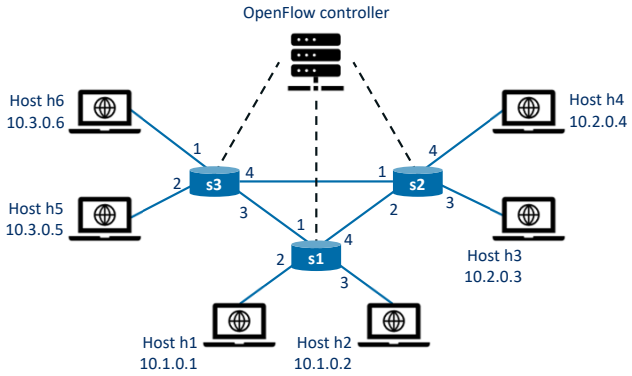
88

As a second example, let's consider a load-balancing scenario, where datagrams from h3 destined to 10.1.0.* are to be forwarded over the direct link between s2 and s1, while datagrams from h4 destined to 10.1.0.* are to be forwarded over the link between s2 and s3 (and then from s3 to s1). Note that this behaviour couldn't be achieved with IP's destination-based forwarding. In this case, the flow table in s2 would be configured to forward traffic coming from interface 3 towards IP destination 10.1.0.* on interface 2, and traffic coming from interface 4 towards IP destination 10.1.0.* on interface 1. Flow table entries are also needed at s1 to forward the datagrams received from s2 to either h1 or h2; and flow table entries are needed at s3 to forward datagrams received on interface 4 from s2 over interface 3 toward s1. See if you can figure out these flow table entries at s1 and s3.

Example 3: Firewalling

- s2 only wants to receive traffic from hosts attached to s3

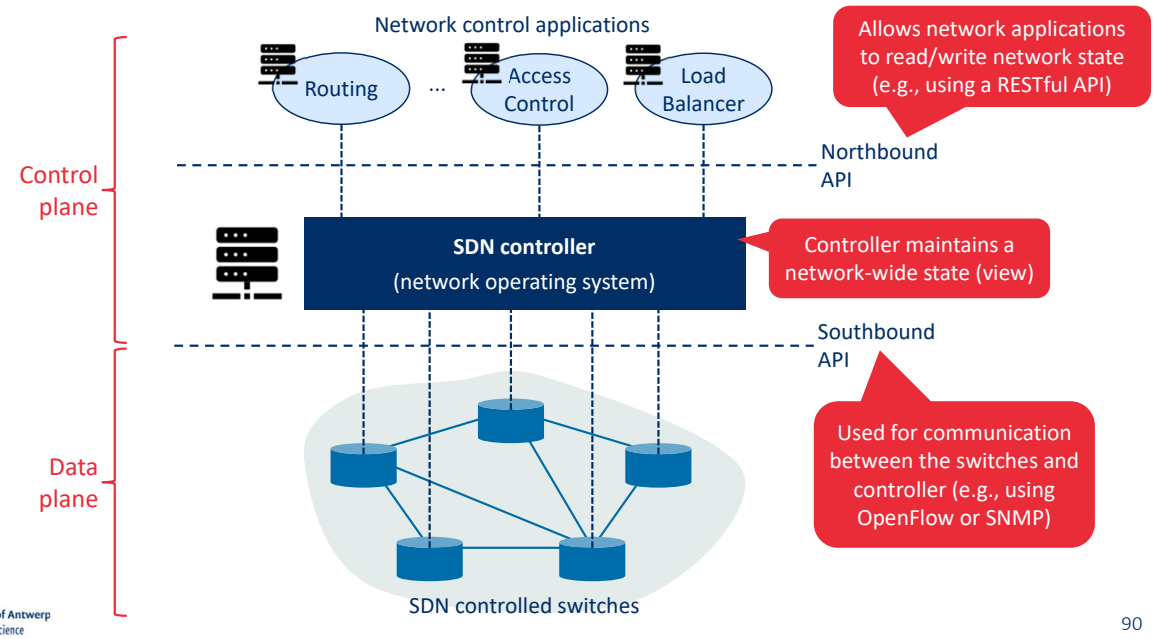
s2 flow table	
Match	**Action**
IP Src = 10.3.0.*; IP Dst = 10.2.0.3	Forward(3)
IP Src = 10.3.0.*; IP Dst = 10.2.0.4	Forward(4)



University of Antwerp
Faculty of Science

As a third example, let's consider a firewall scenario in which s2 wants only to receive (on any of its interfaces) traffic sent from hosts attached to s3. If there were no other entries in s2's flow table, then only traffic from 10.3.0.* would be forwarded to the hosts attached to s2.

SDN architecture

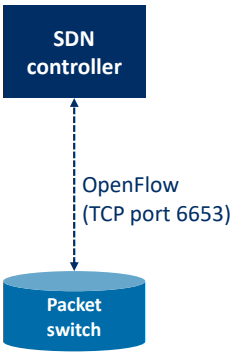


The SDN control plane divides broadly into two components—the SDN controller and the SDN network-control applications. controller's functionality can be broadly organized into three layers:

- ***A communication layer: communicating between the SDN controller and controlled network devices.*** Clearly, if an SDN controller is going to control the operation of a remote SDN-enabled switch, host, or other device, a protocol is needed to transfer information between the controller and that device. In addition, a device must be able to communicate locally-observed events to the controller (for example, a message indicating that an attached link has gone up or down, that a device has just joined the network, or a heartbeat indicating that a device is up and operational). These events provide the SDN controller with an up-to-date view of the network's state. The communication between the controller and the controlled devices cross what has come to be known as the controller's "southbound" interface. OpenFlow is a specific protocol that provides this communication functionality.
- ***A network-wide state-management layer.*** The ultimate control decisions made by the SDN control plane—for example, configuring flow tables in all switches to achieve the desired end-end forwarding, to implement load balancing, or to implement a particular firewalling capability—will require that the controller have up-to-date information about state of the networks' hosts, links, switches, and other SDN-controlled devices. A switch's flow table contains counters whose values might also be profitably used by network-control applications; these values should thus be available to the applications. Since the ultimate aim of the control plane is to determine flow tables for the various controlled devices, a controller might also maintain a copy of these tables.
- ***The interface to the network-control application layer.*** The controller interacts with

network-control applications through its “northbound” interface. This API allows network-control applications to read/write network state and flow tables within the state-management layer. Applications can register to be notified when state-change events occur, so that they can take actions in response to network event notifications sent from SDN-controlled devices. Different types of APIs may be provided, such as REST or request-response interfaces.

OpenFlow Protocol



- Messages from **SDN controller** to **packet switch**
  - **Configuration**: Query and set the switch's configuration
  - **Modify-State**: Add/delete/modify entries in the switch's flow table and port properties
  - **Read-State**: Collect statistics and counter values from the switch's flow table and ports
  - **Send-Packet**: Send a specific packet via a specific port of the controlled switch
- Messages from **packet switch** to **SDN controller**
  - **Flow-Removed**: Sent when a flow table entry is removed (e.g., due to time-out or Modify-State)
  - **Port-Status**: Inform controller of a change in port status
  - **Packet-In**: Send a packet to the controller that does not match any flow table entry

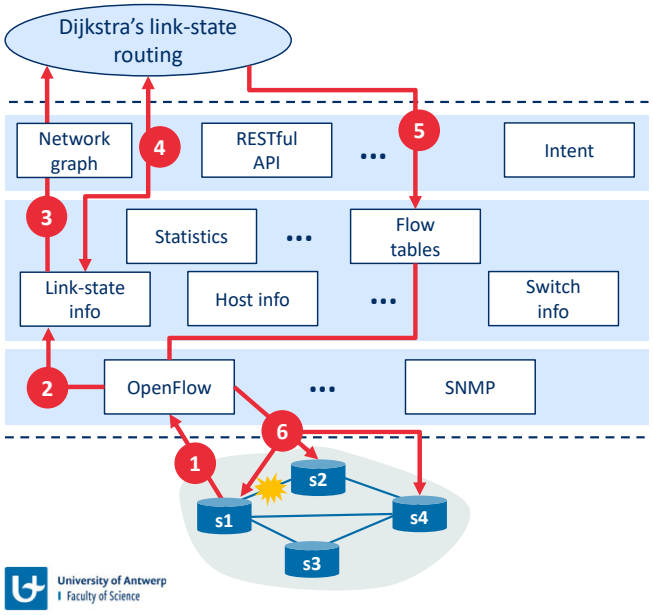
The OpenFlow protocol operates between an SDN controller and an SDN-controlled switch or other device implementing the OpenFlow API that we studied earlier. The OpenFlow protocol operates over TCP, with a default port number of 6653. Among the important messages flowing from the controller to the controlled switch are the following:

- *Configuration*. This message allows the controller to query and set a switch's configuration parameters.
- *Modify-State*. This message is used by a controller to add/delete or modify entries in the switch's flow table, and to set switch port properties.
- *Read-State*. This message is used by a controller to collect statistics and counter values from the switch's flow table and ports.
- *Send-Packet*. This message is used by the controller to send a specific packet out of a specified port at the controlled switch. The message itself contains the packet to be sent in its payload.

Among the messages flowing from the SDN-controlled switch to the controller are the following:

- *Flow-Removed*. This message informs the controller that a flow table entry has been removed, for example by a timeout or as the result of a received modify-state message.
- *Port-status*. This message is used by a switch to inform the controller of a change in port status.
- *Packet-in*. A packet arriving at a switch port and not matching any flow table entry is sent to the controller for additional processing. Matched packets may also be sent to the controller, as an action to be taken on a match. The packet-in message is used to send such packets to the controller.

Example: Dijkstra's shortest path routing using SDN



1. s1 detects link failure to s2 and notifies SDN controller using OpenFlow protocol
2. OpenFlow controller updates link-state database
3. Link-state routing application receives notification
4. Routing application calculates new routes based on updated link-state database
5. Routing application updates flow table database
6. OpenFlow is used to update flow tables in switches

92

In order to solidify our understanding of the interaction between SDN-controlled switches and the SDN controller, let's consider an example in which Dijkstra's algorithm is used to determine shortest path routes. The SDN scenario has two important differences from the earlier per-router-control scenario, where Dijkstra's algorithm was implemented in each and every router and link-state updates were flooded among all network routers:

- Dijkstra's algorithm is executed as a separate application, outside of the packet switches.
- Packet switches send link updates to the SDN controller and not to each other.

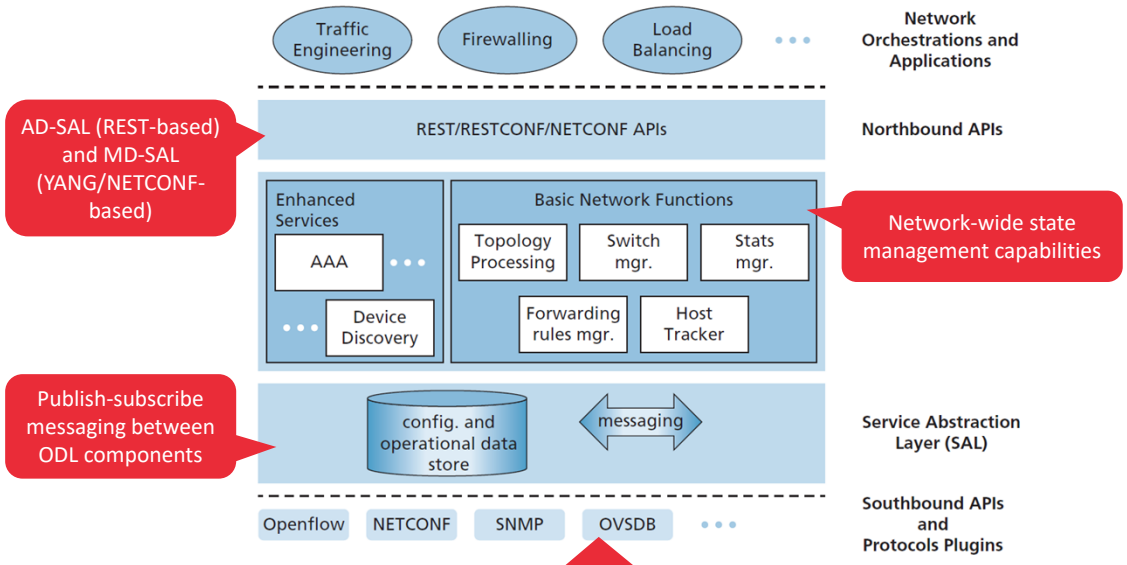
In this example, let's assume that the link between switch s1 and s2 goes down; that shortest path routing is implemented, and consequently and that incoming and outgoing flow forwarding rules at s1, s3, and s4 are affected, but that s2's operation is unchanged. Let's also assume that OpenFlow is used as the communication layer protocol, and that the control plane performs no other function other than link-state routing.

1. Switch s1, experiencing a link failure between itself and s2, notifies the SDN controller of the link-state change using the OpenFlow port-status message.
2. The SDN controller receives the OpenFlow message indicating the link-state change, and notifies the link-state manager, which updates a link-state database.
3. The network-control application that implements Dijkstra's link-state routing has previously registered to be notified when link state changes. That application receives the notification of the link-state change.
4. The link-state routing application interacts with the link-state manager to get updated link state; it might also consult other components in the state-management layer. It then computes the new least-cost paths.

5. The link-state routing application then interacts with the flow table manager, which determines the flow tables to be updated.
6. The flow table manager then uses the OpenFlow protocol to update flow table entries at affected switches—s1 (which will now route packets destined to s2 via s4), s2 (which will now begin receiving packets from s1 via intermediate switch s4), and s4 (which must now forward packets from s1 destined to s2).

This example is simple but illustrates how the SDN control plane provides control plane services (in this case, network-layer routing) that had been previously implemented with per-router control exercised in each and every network router. One can now easily appreciate how an SDN-enabled ISP could easily switch from least-cost path routing to a more hand-tailored approach to routing. Indeed, since the controller can tailor the flow tables as it pleases, it can implement any form of forwarding that it pleases—simply by changing its application-control software. This ease of change should be contrasted to the case of a traditional per-router control plane, where software in all routers (which might be provided to the ISP by multiple independent vendors) must be changed.

SDN Controllers in practice: OpenDaylight



The figure presents a simplified view of the OpenDaylight (ODL) controller platform. ODL's Basic Network Functions are at the heart of the controller, and correspond closely to the network-wide state management capabilities. The Service Abstraction Layer (SAL) is the controller's nerve center, allowing controller components and applications to invoke each other's services, access configuration and operational data, and to subscribe to events they generate. The SAL also provides a uniform abstract interface to specific protocols operating between the ODL controller and the controlled devices. These protocols include OpenFlow, and the Simple Network Management Protocol (SNMP) and the Network Configuration (NETCONF) protocol. The Open vSwitch Database Management Protocol (OVSDB) is used to manage data center switching, an important application area for SDN technology.

Network Orchestrations and Applications determine how data-plane forwarding and other services, such as firewalling and load balancing, are accomplished in the controlled devices. ODL provides two ways in which applications can interoperate with native controller services (and hence devices) and with each other. In the API-Driven (AD-SAL) approach, applications communicate with controller modules using a REST request-response API running over HTTP. Initial releases of the OpenDaylight controller provided only the AD-SAL. As ODL became increasingly used for network configuration and management, later ODL releases introduced a Model- Driven (MD-SAL) approach. Here, the YANG data modeling language defines models of device, protocol, and network configuration and operational state data. Devices are then configured and managed by manipulating this data using the NETCONF protocol.

Network management protocols

Sections 5.6 and 5.7

Internet Control Message Protocol (ICMP) – RFC 792

- Used by hosts and routers to communicate network-related information to each other
- ICMP messages are carried as IP payload (protocol number 1)
- Example messages:

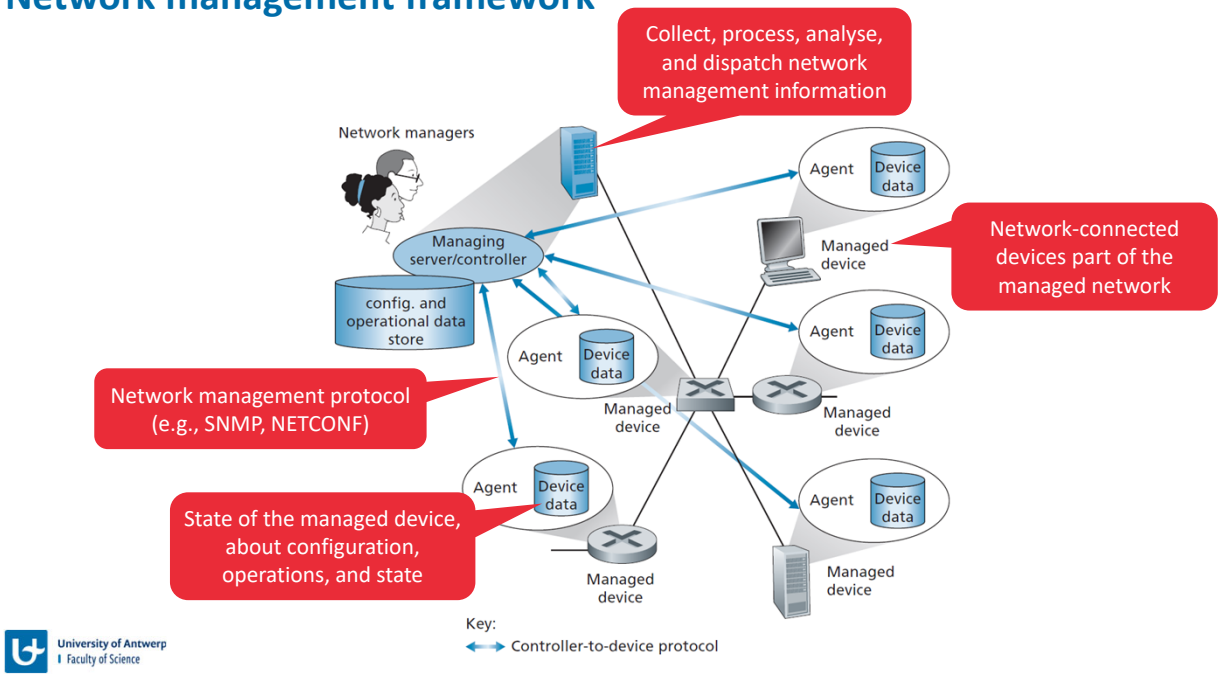
Type	Code	Description
0	0	Echo reply (to ping)
3	0	Destination network unreachable
3	1	Destination host unreachable
3	2	Destination protocol unreachable
3	3	Destination port unreachable
3	6	Destination network unknown
3	7	Destination host unknown
8	0	Echo request
9	0	Router advertisement
10	0	Router discovery
11	0	TTL expired

- Ping makes use of ICMP (Type=8 and Type=0 messages)
- Traceroute captures TTL expired (Type=11, Code=0) and destination port unreachable (Type=3, Code=6) messages combined UDP datagrams with increasing TTL values

The Internet Control Message Protocol (ICMP), specified in [RFC 792], is used by hosts and routers to communicate network-layer information to each other. The most typical use of ICMP is for error reporting. For example, when running an HTTP session, you may have encountered an error message such as “Destination network unreachable.” This message had its origins in ICMP. At some point, an IP router was unable to find a path to the host specified in your HTTP request. That router created and sent an ICMP message to your host indicating the error. ICMP is often considered part of IP, but architecturally it lies just above IP, as ICMP messages are carried inside IP datagrams. That is, ICMP messages are carried as IP payload, just as TCP or UDP segments are carried as IP payload. Similarly, when a host receives an IP datagram with ICMP specified as the upper-layer protocol (an upper-layer protocol number of 1), it demultiplexes the datagram’s contents to ICMP, just as it would demultiplex a datagram’s content to TCP or UDP.

ICMP messages have a type and a code field, and contain the header and the first 8 bytes of the IP datagram that caused the ICMP message to be generated in the first place (so that the sender can determine the datagram that caused the error). Selected ICMP message types are shown in the table. Note that ICMP messages are used not only for signalling error conditions.

Network management framework



University of Antwerp
Faculty of Science

96

The key components of network management:

- **Managing server.** The managing server is an application, typically with network managers (humans) in the loop, running in a centralized network management station in the network operations center (NOC). The managing server is the locus of activity for network management: it controls the collection, processing, analysis, and dispatching of network management information and commands. It is here that actions are initiated to configure, monitor, and control the network's managed devices. In practice, a network may have several such managing servers.
- **Managed device.** A managed device is a piece of network equipment (including its software) that resides on a managed network. A managed device might be a host, router, switch, middlebox, modem, thermometer, or other network-connected device. The device itself will have many manageable components (e.g., a network interface is but one component of a host or router), and configuration parameters for these hardware and software components (e.g., an intra-AS routing protocol, such as OSPF).
- **Data.** Each managed device will have data, also known as "state," associated with it. There are several different types of data. Configuration data is device information explicitly configured by the network manager, for example, a manager-assigned/ configured IP address or interface speed for a device interface. Operational data is information that the device acquires as it operates, for example, the list of immediate neighbors in OSPF protocol. Device statistics are status indicators and counts that are updated as the device operators (e.g., the number of dropped packets on an interface, or the device's cooling fan speed). The network manager can query remote device data, and in some cases, control the remote device by writing device data values, as discussed below. The managing server also maintains its own copy of configuration, operational and statistics

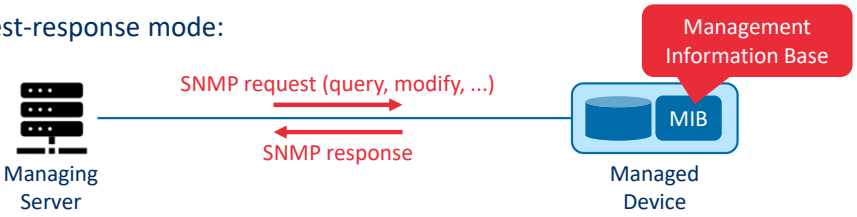
data from its managed devices as well as network-wide data (e.g., the network's topology).

- **Network management agent.** The network management agent is a software process running in the managed device that communicates with the managing server, taking local actions at the managed device under the command and control of the managing server.
- **Network management protocol.** The final component of a network management framework is the network management protocol. This protocol runs between the managing server and the managed devices, allowing the managing server to query the status of managed devices and take actions at these devices via its agents. Agents can use the network management protocol to inform the managing server of exceptional events (e.g., component failures or violation of performance thresholds).

Simple Network Management Protocol (SNMP)

- Application-layer protocol (usually over UDP) used to convey network-management control information
- SNMPv3 is standardized in IETF RFC 3410

- Request-response mode:



- Trap-mode:

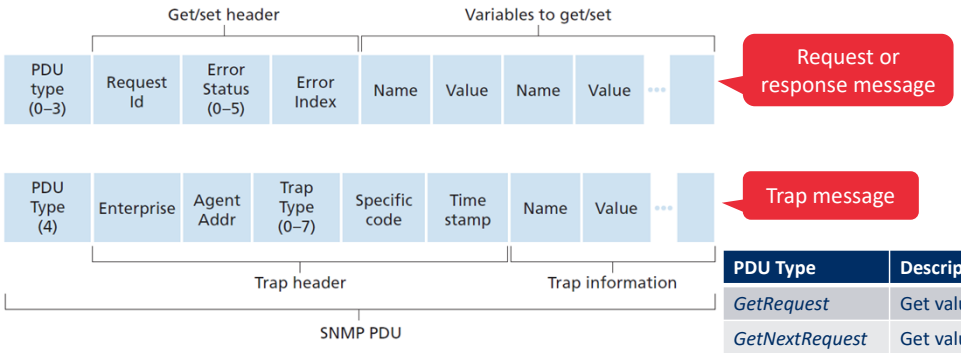
University of Antwerp | Faculty of Science

97

The Simple Network Management Protocol version 3 (SNMPv3) [RFC 3410] is an application-layer protocol used to convey network-management control and information messages between a managing server and an agent executing on behalf of that managing server. The most common usage of SNMP is in a request-response mode in which an SNMP managing server sends a request to an SNMP agent, who receives the request, performs some action, and sends a reply to the request. Typically, a request will be used to query (retrieve) or modify (set) MIB object values associated with a managed device. A second common usage of SNMP is for an agent to send an unsolicited message, known as a trap message, to a managing server. Trap messages are used to notify a managing server of an exceptional situation (e.g., a link interface going up or down) that has resulted in changes to MIB object values.

MIB objects are specified in a data description language known as SMI (Structure of Management Information) [RFC 2578; RFC 2579; RFC 2580], a rather oddly named component of the network management framework whose name gives no hint of its functionality. A formal definition language is used to ensure that the syntax and semantics of the network management data are well defined and unambiguous. Related MIB objects are gathered into MIB modules. As of late 2019, there are more than 400 MIB-related RFCs and a much larger number of vendor-specific (private) MIB modules.

SNMP messages – aka. Protocol Data Units (PDUs)



PDU Type	Description
GetRequest	Get value of MIB object instance(s)
GetNextRequest	Get value of next MIB instance in list
GetBulkRequest	Get values of large block of MIB data
InformRequest	Inform remote managing entity of MIB values
SetRequest	Set value of MIB object instance(s)
Response	Response to PDU request messages
Trap	Device informs manager of exceptional event

University of Antwerp
Faculty of Science

98

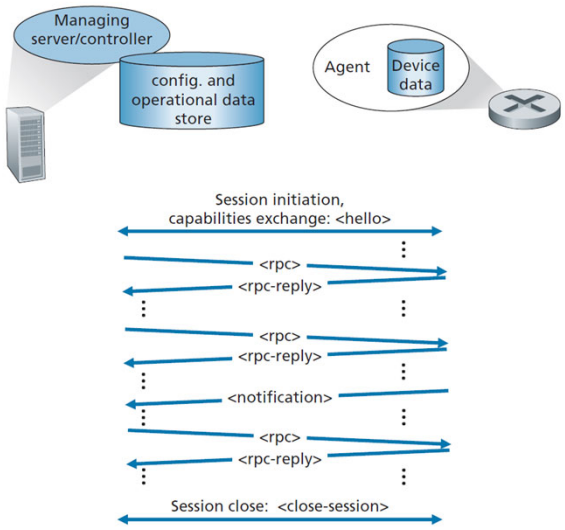
SNMPv3 defines seven types of messages, known generically as protocol data units—PDUs:

- The **GetRequest**, **GetNextRequest**, and **GetBulkRequest** PDUs are all sent from a managing server to an agent to request the value of one or more MIB objects at the agent's managed device. The MIB objects whose values are being requested are specified in the variable binding portion of the PDU. GetRequest, GetNextRequest, and GetBulkRequest differ in the granularity of their data requests. GetRequest can request an arbitrary set of MIB values; multiple GetNextRequests can be used to sequence through a list or table of MIB objects; GetBulkRequest allows a large block of data to be returned, avoiding the overhead incurred if multiple GetRequest or GetNextRequest messages were to be sent. In all three cases, the agent responds with a Response PDU containing the object identifiers and their associated values.
- The **SetRequest** PDU is used by a managing server to set the value of one or more MIB objects in a managed device. An agent replies with a Response PDU with the "noError" error status to confirm that the value has indeed been set.
- The **InformRequest** PDU is used by a managing server to notify another managing server of MIB information that is remote to the receiving server.
- The **Response** PDU is typically sent from a managed device to the managing server in response to a request message from that server, returning the requested information.
- The final type of SNMPv3 PDU is the **trap message**. Trap messages are generated asynchronously; that is, they are not generated in response to a received request but rather in response to an event for which the managing server requires notification. RFC 3418 defines well-known trap types that include a cold or warm start by a device, a link going up or down, the loss of a neighbor, or an authentication failure event. A received trap request has no required response from a managing server.

Given the request-response nature of SNMP, it is worth noting here that although SNMP PDUs can be carried via many different transport protocols, the SNMP PDU is typically carried in the payload of a UDP datagram.

Network Configuration Protocol (NETCONF) – RFC 6241

- Messaging protocol between managing server and managed network devices, that allows:
  - Retrieving, setting, and modifying configuration data
  - Querying operational data and statistics
  - Subscribe to notifications generated by devices
- Configurations are structured in XML documents
- NETCONF uses remote procedure calls (RPCs)
- Runs on top of TLS/TCP
- Semantics of the data are captured using the YANG data modelling language



The NETCONF protocol operates between the managing server and the managed network devices, providing messaging to (i) retrieve, set, and modify configuration data at managed devices; (ii) to query operational data and statistics at managed devices; and (iii) to subscribe to notifications generated by managed devices. The managing server actively controls a managed device by sending it configurations, which are specified in a structured XML document, and activating a configuration at the managed device. NETCONF uses a remote procedure call (RPC) paradigm, where protocol messages are also encoded in XML and exchanged between the managing server and a managed device over a secure, connection-oriented session such as the TLS (Transport Layer Security) protocol over TCP.

The figure on the right shows an example NETCONF session. First, the managing server establishes a secure connection to the managed device. Once a secure connection has been established, the managing server and the managed device exchange <hello> messages, declaring their "capabilities"—NETCONF functionality that supplements the base NETCONF specification in [RFC 6241]. Interactions between the managing server and managed device take the form of a remote procedure call, using the <rpc> and <rpc-response> messages. These messages are used to retrieve, set, query and modify device configurations, operational data and statistics, and to subscribe to device notifications. Device notifications themselves are proactively sent from managed device to the managing server using NETCONF <notification> messages. A session is closed with the <session-close message>.

NETCONF operations

NETCONF operation	Description
<get-config>	Retrieve all or part of a configuration
<get>	Retrieve all or part of a configuration state and operational state data
<edit-config>	Change all or part of a configuration
<lock>, <unlock>	Lock or unlock the configuration of a device, to avoid race conditions
<create-subscription>	Initiate even notification subscription
<notification>	Event notification for subscribers

The table shows a number of the important NETCONF operations that a managing server can perform at a managed device. As in the case of SNMP, we see operations for retrieving operational state data (<get>), and for event notification. However, the <get-config>, <edit-config>, <lock> and <unlock> operation demonstrate NETCONF's particular emphasis on device configuration. Using these basic operations, it is also possible to create a set of more sophisticated network management transactions that either complete atomically (i.e., as a group) and successfully on a set of devices, or are fully reversed and leave the devices in their pre-transaction state. Such multi-device transactions enabling operators to concentrate on the configuration of the network as a whole rather than individual devices" was an important operator requirement put forth in [RFC 3535].

Summary

University of Antwerp
Faculty of Science

Summary

- The Internet's network layer is best-effort and provides **addressing** (using IPv4 or IPv6)
- Every router performs **forwarding** (data plane) based on a forwarding table populated by a **routing** algorithm (control plane)
- Router **queueing** can occur both at the input and output ports
  - Queuing policies: First-in-first-out (FIFO), priority queuing, weighted fair queuing (and round robin)
- **IPv4** uses 32-bit addresses, while **IPv6** uses 128-bit addresses
  - Each IP address has a network prefix and a host identifier (marked using the subnet mask /x notation)
  - **DHCP** automatically assigns IP addresses, and discovers the default gateway, subnet mask, and local DNS server
  - **NAT** is used to translate between public and private IP addresses
- **Routing algorithms** determine the path from the source to destination host in a network
  - Centralized (**link-state**) vs. distributed (**distance vector/path vector**) routing
  - The Internet splits routing in two levels: within and between autonomous systems (ASs)
  - Several algorithms exist for intra-AS routing, such as **OSPF** (based on the link-state method)
  - Inter-AS routing is done using **BGP**, which is based on distance-vectors
- **SDN** supports more fine-grained packet processing, using multi-header matching rules and flexible actions
- ~~Network management protocols (**SNMP** or **NETCONF**) are used manage (configure, monitor) network devices~~
- The contents of this part is covered in the **book** in Sections 4.1—4.4 and 5.2—5.5~~5.7~~

End of Part 4

University of Antwerp
Faculty of Science